408 详解 · 2020 年选择题 · 从零版

只含选择题第 1–40 题，不含综合应用题。

每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。

说明：凡题干问"错误的是"的题（第 15、18、21、22、29、30、34 题），第 4 节说明正确选项（即那个错述）为什么成立，第 5 节说明其余选项为什么是对的（配例子），这样 6 节结构全卷统一。

默认读者是零基础跨考学生：不默认任何 408 前置知识，每题自成体系，不依赖其他题的解析；推理和计算的每一步都写出，不用"显然""易得""同理"这类跳步词；错误选项的例子选最直观的小数字、小例子、生活类比。

本版每题含完整题干与选项，可脱离真题册独立使用。

本版语言以大白话为主：每个术语第一次出现都配一句大白话解释；顺序按"从具体到抽象"——先讲生活例子、小数字、具体场景，让人先懂意思，再贴术语名（考试卷子用的就是术语名，认得它也是必须的）。

本版讲解比 2022 版更深：考查知识点与错误选项两节把相关定义、适用条件、边界情况、易混对子讲到位；计组、寻址、流水线等弱项处多讲一步。

图题说明：第 5、7、33、35、37、40 题题干依赖图片，图片未能直接查看，这 6 题标注"部分核对"，答案按答案表写，图中细节以原卷为准。

升级版说明：每题在原有 6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）；原有题干、选项、答案与 6 节内容未作改动。

第 1 题 · 对称矩阵上三角列优先存储（正确答案：C，22）

题干与选项：

将一个 10×10 对称矩阵 M 的上三角部分的元素 m_{ij}（$1 \le i \le j \le 10$），按列优先存入 C 语言的一维数组 N 中，元素 $m_{7,2}$ 在 N 中的下标是（）

- A. 15
- B. 16
- C. 22
- D. 23

入手点： 先把 $m_{7,2}$ 换成对角线右上方那个对应的元素，再数它前面有几个元素。

题干专有名词从零解释：

- 先想一张方形的数表，对折之后两边的数完全一样——比如左上角的数等于右下角对称位置的数。这样的表在卷子上叫"对称矩阵"，意思是 $m_{ij} = m_{ji}$。
- 只要对角线右上方那一半（包括对角线本身），就叫"上三角部分"，意思是行号 ⩽ 列号（$i \le j$）的那些元素。
- "列优先"是存的顺序：先把第 1 列从上到下存完，再存第 2 列，依此类推；一列内部按行号从小到大。
- C 语言里数组的"下标"是从 0 开始数的：第一个元素下标是 0，第二个是 1，依此类推；日常说的"第几个"是从 1 开始数的，下标 = 第几个 − 1。

考查知识点：

- 先想"只存一半就够了"：对称矩阵对折两边一样，存一半就能推出另一半，所以只存上三角。10 阶的表，上三角一共有 $10 \times 11/2 = 55$ 个元素，下标从 0 到 54。
- 再想"第几个"怎么算：上三角里第 j 列有 j 个元素（第 1 列 1 个，第 2 列 2 个，依此类推）。所以对角线右上方那个位置 m_{ij}（$i \le j$）按列优先存，下标 = 前面 $j-1$ 列的总数 $j(j-1)/2$，再加它在本列里的位置偏移 $(i-1)$。验算两个边角：m_{11} 是第 1 个，下标 0；$m_{10,10}$ 是最后 1 个，下标 $10 \times 9/2 + 9 = 54$，正好是 55 个元素的最后一个。
- 对照另一种存法"行优先"：一行一行存，m_{ij} 前面有 $(i-1) \times n - (i-1)(i-2)/2$ 个（前面 $i-1$ 行的元素数）再加 $(j-i)$。行优先和列优先不能混用，本题要求列优先。
- 容易踩的两个坑：下三角的元素要先换成上三角的对应元素（$m_{ji} = m_{ij}$）再套公式，不能直接把行列号代入；下标从 0 起，不是"第几个"。

正确选项为什么对： $m_{7,2}$ 中 $7 > 2$，在对角线左下方，不在上三角里；由对称性 $m_{7,2} = m_{2,7}$，实际存的是 $m_{2,7}$。套列优先的算法：下标 = $7 \times 6/2 + (2-1) = 21 + 1 = 22$。也可以逐列数：第 7 列前面 6 列共有 $1+2+3+4+5+6 = 21$ 个元素；第 7 列内部，$m_{1,7}$ 排第 1 个，$m_{2,7}$ 排第 2 个；下标从 0 开始，所以下标 = $21 + 1 = 22$。选 C。

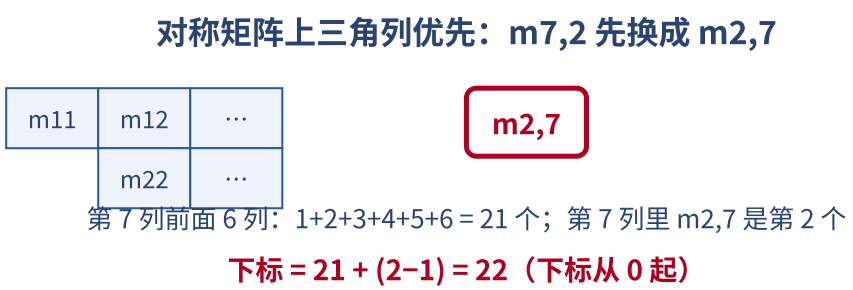
错误选项为什么错＋举例：

- A（15）：按行优先算的：第 1 行 10 个，$m_{2,7}$ 是第 2 行第 6 个，下标 $10 + 5 = 15$；题干要求列优先，方向错了。教训：看到"列优先"三个字先把方向定死，再动手数；行优先和列优先的答案通常都不一样，本题一个 15 一个 22。
- B（16）：行优先且从 1 开始数（第 16 个），既错了方向又错了起数。教训：两个错叠加时答案会离谱——平时练的时候先把"方向"和"起数"两个开关各自确认一遍。

- D（23）：列优先数对了 21 个在前、第 7 列第 2 个，但忘了下标从 0 开始，多加了 1（$21+2=23$）。验算办法：拿最小的元素验一下起数——m_{11} 是第 1 个元素，下标必须是 0；如果你的算法算出 m_{11} 的下标是 1，说明起数错了。

笔记本一句话： 第 1 题＋对称矩阵先换元（$m_{7,2}=m_{2,7}$），再套列优先通式 $j(j-1)/2+(i-1)$，下标从 0 起。

【图像】



① m7,2 中 7>2，在下三角 → 换成 m2,7

② 列优先：前面 j−1 列共 j(j−1)/2 = 21 个

③ 本列第 i 个：下标再加 (i−1) = 1

④ 验起数：m11 下标是 0（m7,2 不能直接代入！）

行优先会得 15（方向错），列优先从 1 数会得 23（起数错）

对称矩阵上三角列优先：m7,2先换成m2,7，前面6列21个，本列第2个，下标22

【定位】 数据结构 · 第 3 章栈、队列和数组 · 特殊矩阵的压缩存储

【易混对比】

- "第几个"和"下标"是两回事：第几个从 1 开始数，下标从 0 开始数，下标 = 第几个 − 1；m11 是第 1 个元素，下标是 0。
- "行优先"和"列优先"是两回事：行优先一行一行数，列优先一列一列数；本题 m2,7 按行优先下标是 15，按列优先下标是 22，方向一错答案就错。
- "m7,2"和"m2,7"是两回事：m7,2 在对角线左下方，不在上三角存储范围里；必须先用对称性换成 m2,7 再套公式，不能直接把 7、2 代入。

先换元，再定方向，最后记得下标从 0 起。

【坑位】

下三角的元素不能直接代入列优先公式，先换成上三角的对应元素。

最容易踩的坑，是看到 m7,2 就把 7 当行、2 当列直接套 j(j−1)/2 + (i−1)。m7,2 里 7 > 2，它在对角线左下方，根本不在"上三角部分"那 55 个元素里。正确做法是先用对称性 m7,2 = m2,7 换到对角线右上方，再按列优先数：前面 6 列共有 1+2+3+4+5+6 = 21 个，第 7 列里它是第 2 个，下标 = 21 + 1 = 22。

【套路】

- 第一步，看下标在不在上三角：i > j 就用对称性换成 mji。
- 第二步，定方向：列优先就一列一列数，前面 j−1 列共有 j(j−1)/2 个。

- 第三步，数本列第几个：第 i 行就是本列第 i 个，下标再加 (i−1)。
- 第四步，验起数：m11 的下标必须是 0，算出来不是 0 就回头查方向和起数。

换元 → 定方向 → 套公式 → 验 m11。

【类比】

像电影院按列卖票，只卖右上半区（含对角线）的座位。你拿着一张"7 排 2 座"的票，但 7 排 2 座在左下半区，根本没开卖——售票员先按对称换成"2 排 7 座"，再告诉你是第 22 张票（从 0 号开始数）。直接按"7 排 2 座"找，座位号全错。

【口诀】 下三角先换元，列优先往前数，下标从零起。

【考频】 「列优先」在 2009–2022 年解析中出现 3 次 · 中频

第 2 题 · 栈的操作序列（正确答案：D，b，c，e）

题干与选项：

对空栈 S 进行 Push 和 Pop 操作，入栈序列 a，b，c，d，e，经过 Push，Push，Pop，Push，Pop，Push，Push，Pop 操作后得到的出栈序列是：（）

- A. b，a，c
- B. b，a，e
- C. b，c，a
- D. b，c，e

入手点： 拿一张纸画个桶，Push 就是往桶里放东西，Pop 就是从桶口拿东西，照单操作。

题干专有名词从零解释：

- 先想摞盘子：盘子一个一个往上摞，拿的时候只能拿最上面那个，先放的反而最后才能拿到。这种"后放的先拿"的结构，在卷子上叫"栈"，拿东西的那一端叫"栈顶"。
- "Push"就是往栈顶放一个元素；"Pop"就是从栈顶拿走一个元素。
- "入栈序列"就是元素按什么顺序被放进去；"出栈序列"就是元素按什么顺序被拿出来。

考查知识点：

- 栈的规矩就一条：进出都在栈顶；Pop 永远拿走最后进来的那个；底下的元素要等它上面的全部出去后才能出来。
- 通用解法叫"照单模拟法"：操作序列给什么就做什么，Push 就把下一个入栈元素写进栈顶，Pop 就把栈顶抄下来；模拟法对一切栈操作题都适用，不用背结论。
- 边界情况：空桶不能拿东西（空栈 Pop 是非法的，叫"下溢"）；桶满了不能再放（叫"上溢"）；操作结束时桶里可能还有剩下的元素（本题最后 a、d 还在栈里，这是正常的）。
- 对照另一种结构"队列"：排队买票，先到的先走，叫"先进先出"；栈是"后进先出"。本题如果按队列算，出栈序列会是 a，b，c，选项里根本没有，那是另一道题。
- 延伸：栈的典型应用——括号匹配、表达式求值、函数调用（每次调用把返回地址压栈，返回时弹栈），都是"后进先出"这一个性质在起作用。

正确选项为什么对： 一步一步走：Push a → 桶里 [a]；Push b → [a，b]；Pop → 拿出 b；Push c → [a，c]；Pop → 拿出 c；Push d → [a，d]；Push e → [a，d，e]；Pop → 拿出 e。拿出的顺序是 b，c，e。选 D。

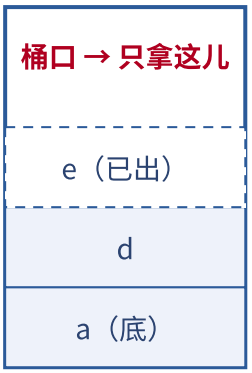
错误选项为什么错＋举例：

- A（b，a，c）：按 A 的顺序，第二拿的是 a。实际第 2 次 Pop（第 5 步）拿的是 c，因为第 4 步放了 c，桶口是 c 不是 a；a 还在桶底，要等 c、d、e 全出去才能出来，但操作在第 8 步就结束了。教训：每次拿之前先看一眼当前桶口是谁，不要凭"下一个该轮到谁"的感觉。
- B（b，a，e）：同样是把桶底的 a 当成了桶口；栈的规矩是"后放的先拿"，不是"先放的先拿"，先放的 a 恰恰最后才能出。
- C（b，c，a）：最后一步拿以为是 a，忘了第 6、7 步放了 d、e，桶口是 e；a 在最底下，第 8 步根本轮不到它。

笔记本一句话： 第 2 题＋栈的操作题照单画桶，拿只拿桶口，别拿桶底；操作结束桶里有剩余是正常的。

【图像】

栈操作模拟：Push a,b → Pop出b → Push c → Pop出c → Push d,e → Pop出e



出栈序列：b, c, e

第2次Pop拿的是c，不是a

a在桶底，要等c,d,e全出去

操作结束a,d还在桶里：正常

① Push a → [a]；Push b → [a,b]

② Pop → 拿出b（桶口）

③ Push c → [a,c]；Pop → 拿出c

④ Push d → [a,d]；Push e → [a,d,e]

⑤ Pop → 拿出e（桶口是e不是a）

栈操作模拟：Push a,b → Pop出b → Push c → Pop出c → Push d,e → Pop出e，出栈序列b,c,e

【定位】 数据结构 · 第 3 章栈、队列和数组 · 栈的操作模拟

【易混对比】

- "栈顶"和"栈底"是两回事：Pop 永远拿走最后进来的那个（桶口），不是最早进来的那个（桶底）；a 最先进来，反而最后才能出去。
- "照单模拟"和"凭感觉猜"是两回事：操作序列给什么就做什么，一步一步画；凭"下一个该轮到谁"的感觉猜，十有八九把桶底当成桶口。
- "操作结束桶里有剩余"和"出错"是两回事：8 步操作完 a、d 还在桶里，这是正常的——不是每次操作都要把桶清空。

拿只拿桶口，桶底的要等它上面的全出去。

【坑位】

每次 Pop 之前先看一眼当前桶口是谁，不要凭感觉拿。

最容易踩的坑，是第 5 步（第 2 次 Pop）拿成 a。前面是 Push a、Push b、Pop（拿出 b）、Push c——此时桶口是刚放进去的 c，不是 a。第 2 次 Pop 拿的是 c。a 在桶底，要等 c、d、e 全出去才能出来，但操作在第 8 步就结束了，根本轮不到 a。凡是选项里第 2 个是 a 的（A、B），都是把桶底当成了桶口。

【套路】

- 第一步，画个桶：Push 就把下一个入栈元素写进桶口，Pop 就把桶口那个抄下来当输出。
- 第二步，照单走：操作序列有 8 步就走 8 步，一步一画，不跳步。
- 第三步，每次拿之前看桶口：桶口是谁就拿谁，不猜"该轮到谁"。

照单画桶，拿只拿桶口。

【类比】

像摞盘子：盘子一个一个往上摞，拿只能拿最上面那个。服务员先放了 a 盘、b 盘，拿走 b 盘，又放上 c 盘——这时候要拿，只能拿最上面的 c 盘，不可能隔着 c 盘去拿底下的 a 盘。想拿 a 盘，得先把 c、d、e 全拿走。

【口诀】 照单画桶走八步，只拿桶口不拿底。

【考频】 「出栈序列」在 2009–2022 年解析中出现 24 次 · 高频

第 3 题 · 顺序存储二叉树的存储单元数（正确答案：A，31）

题干与选项：

对任意一棵高度为 5 且有 10 个节点的二叉树，若采用顺序存储结构保存，每个结点占 1 个存储单元（仅存放结点的数据信息），则存放该二叉树需要的存储单元数量至少是（ ）

- A. 31
- B. 16
- C. 15
- D. 10

入手点： 抓住"任意一棵"和"至少"：要的是保证任意一棵都装得下的最小数组，不是某一棵最省的情况。

题干专有名词从零解释：

- 先想一排格子，每个结点占一格，结点按"完全二叉树的编号"对号入座：根是 1 号，某个结点的左孩子是它编号 × 2，右孩子是 × 2 + 1。这种"用数组格子存树"的办法，在卷子上叫"顺序存储"。
- "高度"就是从根到最深的叶子有几层，根在第 1 层。
- 空缺位置也要占一格：不然编号对不上，父子关系就乱了。

考查知识点：

- 先想编号怎么排：第 k 层的结点编号从 2^{k-1} 到 2^k-1；比如第 5 层就是 16 号到 31 号。高度 5 的满树（每层都排满）一共有 $2^5-1=31$ 个位置，编号从 1 到 31。
- 再想"顺序存储适合什么样的树"：适合完全二叉树和满二叉树（没有空缺或空缺很少）；稀疏的树（比如只有一条链）用顺序存储会浪费大量空缺位置，这时更适合链式存储（一个结点连一个指针，没有空缺）。
- 对照链式存储：链式是按结点个数开空间（10 个结点就 10 个单元加指针），没有"按高度开"的问题；顺序存储是按位置开空间，位置数由高度定，不由结点数定。
- 边界情况：高度 5 的树，第 5 层结点编号范围是 16~31；最左是 16，最右是 31；"任意一棵都装得下"对应的是最右 31，不是"某一棵最省"的 16。
- 容易踩的坑：结点数（10）与位置数（31）是两回事——顺序存储开的是位置数，不是结点数；高度从 1 数（根第 1 层）与从 0 数（根第 0 层）也要分清，本题高度 5 按从 1 数，满树 31 格。

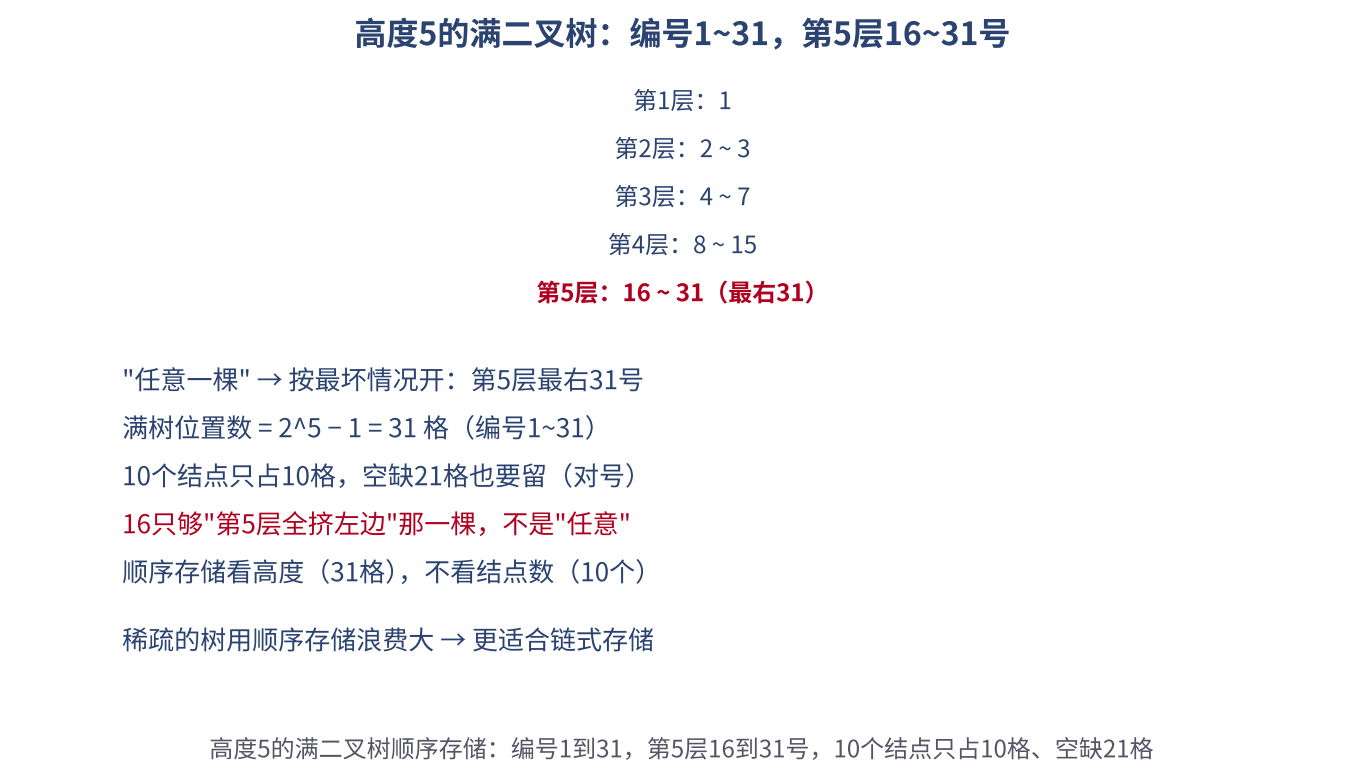
正确选项为什么对： 顺序存储按完全二叉树编号，高度为 5 的满二叉树有 $2^5-1=31$ 个位置，编号从 1 到 31。第 5 层最右的结点编号就是 31。要"任意一棵"高度为 5 的树都装得下，数组必须开到 31 格：比如一棵树在第 5 层最右边有个结点，它的编号是 31，数组小于 31 就装不下。10 个结点只说明实际用了 10 格，空缺的 21 格也要留出来对号。选 A。

错误选项为什么错＋举例：

- B（16）：16 是第 5 层最左结点的编号，只够"第 5 层结点全挤在最左边"的那一棵树；换一棵第 5 层右边有结点的树，16 格就不够了。教训："某一棵最省"不等于"任意一棵都够"，题干有"任意"二字就要按最坏情况开。
- C（15）：15 是高度为 4 的满二叉树结点数（2^4-1），第 5 层一个结点都装不下，高度数错了一层。教训：高度 5 对应 2^5-1，不是 2^4-1；先确认高度再套公式。
- D（10）：只按结点个数开数组，等于默认没有空缺；顺序存储的空缺位置必须占单元，不然父子编号对不上。这是把链式存储的算法套到了顺序存储上。

笔记本一句话： 第 3 题＋顺序存储看高度不开结点数，高度 5 就按满树 31 格开，"任意"二字对应最坏情况。

【图像】



高度5的满二叉树顺序存储：编号1到31，第5层16到31号，10个结点只占10格、空缺21格

【定位】 数据结构 · 第 5 章树与二叉树 · 树的顺序存储

【易混对比】

- "结点数"和"位置数"是两回事：顺序存储开的是位置数（由高度定），不是结点数；10 个结点不等于开 10 格。
- "某一棵最省"和"任意一棵都够"是两回事：16 格只够第 5 层结点全挤在最左边那一棵；题干有"任意"二字，就要按最坏情况（第 5 层最右边有结点）开 31 格。
- "顺序存储"和"链式存储"是两回事：顺序按位置开空间（空缺也占格），链式按结点开空间（10 个结点就 10 个单元）；稀疏的树用顺序存储浪费大，更适合链式。

顺序存储看高度不开结点数，"任意"二字对应最坏情况。

【坑位】

看到"任意一棵"就要按最坏情况开数组，不能按某一棵最省的情况开。

最容易踩的坑，是选 16（第 5 层最左结点的编号）。16 只够"第 5 层结点全挤在最左边"的那一棵树——换一棵第 5 层右边有结点的树，16 格就装不下了。顺序存储按完全二叉树编号对号入座，高度 5 的满树有 31 个位置（编号 1 到 31），第 5 层最右的结点编号就是 31。要保证任意一棵高度 5 的树都装得下，数组必须开到 31 格，空缺的 21 格也要留出来对号。

【套路】

- 第一步，定高度：高度 5，根在第 1 层。
- 第二步，算满树位置数：2^5 − 1 = 31 格。
- 第三步，看题干有没有"任意"：有"任意"就按最坏情况（最右编号）开，没有才按具体那棵树算。

高度定位置数，"任意"定最坏情况。

【类比】

像电影院对号入座：座位号是按"第几排第几座"定的，第 5 排最右边是 31 号。来了 10 个观众（结点），不能只准备 10 张票——万一有人买的是 31 号票，没给他留座就乱套了。空着的 21 个座位也得留着，保证"任意 10 个人来都有座"。

【口诀】 顺序存储看高度，满树开格别数结点。

【考频】 「顺序存储」在 2009–2022 年解析中出现 4 次 · 中频

第 4 题 · 森林转二叉树后根遍历（正确答案：C，b，f，e，d，c，a）

题干与选项：

已知森林 F 及与之对应的二叉树 T，若 F 的先根遍历序列是 a，b，c，d，e，f，后根遍历序列是 b，a，d，f，e，c，则 T 的后根遍历序列是（）

- A. b，a，d，f，e，c
- B. b，d，f，e，c，a
- C. b，f，e，d，c，a
- D. f，e，d，c，b，a

入手点： 先把森林的形状还原出来，再按"左孩子、右兄弟"变成一棵二叉树，最后按"左、右、根"的顺序走一遍。

题干专有名词从零解释：

- 先想几棵树排成一排、互不相连，好比一个院子里种了几棵独立的树——这就叫"森林"，意思是多棵树的集合。
- 再想把这几棵树"捏"成一棵树：口诀是"左孩子、右兄弟"——每个结点的第一个孩子变成它的左孩子，下一个兄弟变成它的右孩子；森林里第一棵树的根就是捏成后那棵树的根。这就是"森林转二叉树"的规则。
- 再想三种走法：从根开始，先看根、再走左、再走右，叫"先根遍历"（顺序是根、左、右）；先走左、再看根、再走右，叫"中根遍历"（左、根、右）；先走左、再走右、最后看根，叫"后根遍历"（左、右、根）。
- 森林的走法是"一棵一棵拼起来"：森林的先根就是各棵树先根拼起来，后根就是各棵树后根拼起来。

考查知识点：

- 先想"怎么从两个序列还原森林"：森林的先根以第一棵树的根开头，后根以第一棵树的根结尾。题干先根是 a 开头，后根是 b，a，d，f，e，c——后根里 a 出现在第 2 位，说明第一棵树的后根就是 b，a，先根就是 a，b：第一棵树只有 a、b 两个结点，a 是根，b 是 a 的孩子。剩下的先根 c，d，e，f 和后根 d，f，e，c 是第二棵树：根是 c；c 的第一个孩子是 d（先根 d、后根 d，单结点）；剩下的先根 e，f、后根 f，e 是第二个孩子 e，e 的孩子是 f。所以森林是：第一棵 a（孩子 b），第二棵 c（孩子 d、e，e 的孩子 f）。
- 再想"左孩子、右兄弟"怎么挂：a 的第一个孩子是 b，b 挂在 a 的左孩子上；a 的下一个兄弟是第二棵树的根 c，c 挂在 a 的右孩子上；c 的第一个孩子是 d，d 挂在 c 的左孩子上；d 的下一个兄弟是 e，e 挂在 d 的右孩子上（注意：兄弟挂在上一个兄弟的右孩子上，不是挂在父亲的右孩子上——这是本题最容易挂错的地方）；e 的第一个孩子是 f，f 挂在 e 的左孩子上。
- 捏成的二叉树 T 是：a 的左孩子 b、右孩子 c；c 的左孩子 d；d 的右孩子 e；e 的左孩子 f。
- 再想"后根怎么走"：后根是左、右、根。走 a：先走左子树 b，得到 b；再走右子树 c：c 的左子树 d 里先走 d 的左（空），再走 d 的右子树 e：e 的左子树 f 得到 f，再看 e，得到 f，e；再看 d，得到 f，e，d；再看 c，得到 f，e，d，c；最后看 a。合起来：b，f，e，d，c，a。
- 对照"森林的后根"和"二叉树的后根"：森林的后根是题干给的 b，a，d，f，e，c；二叉树 T 的后根是 b，f，e，d，c，a；两者不一样，不能直接拿森林的后根当答案。
- 边界情况：如果森林只有一棵树，转换后根的右孩子是空的；如果某个结点没有孩子，对应的左孩子就是空的，走的时候直接跳过。

正确选项为什么对： 按上面的还原和转换，T 的结构是 a（左 b，右 c）、c（左 d）、d（右 e）、e（左 f）。按左、右、根走：左子树 b 先出 b；右子树 c 里：d 的右子树 e 里 f 先出，再出 e，再出 d，再出 c；最后出 a。序列是

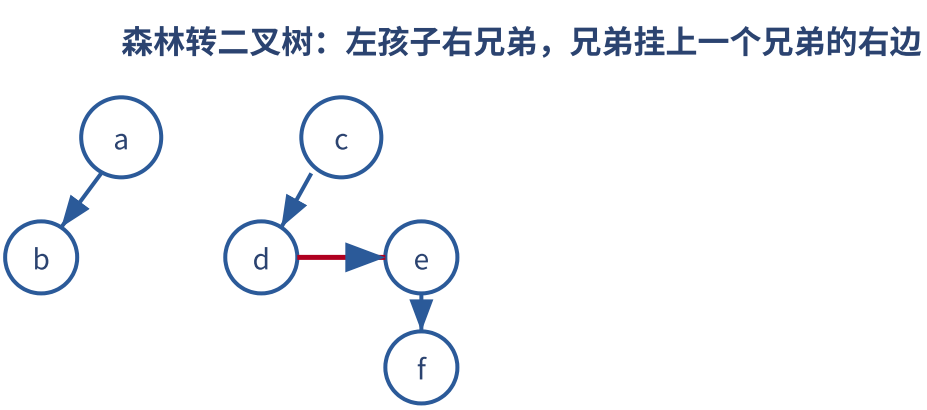
b，f，e，d，c，a。选 C。

错误选项为什么错＋举例：

- A（b，a，d，f，e，c）：这是森林 F 自己的后根（题干原样给的），不是转换后二叉树 T 的后根。教训：题干给的序列是森林的，问题问的是二叉树的——先转换再走，不能直接抄题干。
- B（b，d，f，e，c，a）：这是把 e 挂错位置的结果——把 e 当成了 c 的右孩子，而不是 d 的右孩子。按错的挂法，c 的右子树是 e，左、右、根走出来就是 b，d，f，e，c，a。教训：兄弟链要挂在"上一个兄弟"的右孩子上（e 挂在 d 右边），不是挂在父亲的右孩子上；挂错一个，整串顺序全变。
- D（f，e，d，c，b，a）：这是 T 的先根序列 a，b，c，d，e，f 整个倒过来念的，不是后根。教训：后根不是"倒着念"，它是左、右、根的递归顺序；倒序碰巧在某些小树上像后根，但一般树上不对。

笔记本一句话： 第 4 题＋森林转二叉树先还原森林（先根开头、后根结尾定根），再按左孩子右兄弟挂，兄弟挂上一个兄弟的右边；后根左、右、根。

【图像】



森林转二叉树：左孩子右兄弟挂法，兄弟e挂在d的右孩子上，后根遍历b,f,e,d,c,a

【定位】 数据结构 · 第 5 章树与二叉树 · 树、森林与二叉树的转换

【易混对比】

- "森林的后根"和"二叉树的后根"是两回事：题干给的 b,a,d,f,e,c 是森林 F 自己的后根；转换后二叉树 T 的后根是 b,f,e,d,c,a——先转换再走，不能直接抄题干。
- "兄弟挂父亲的右孩子"和"兄弟挂上一个兄弟的右孩子"是两回事：e 是 d 的兄弟，要挂在 d 的右孩子上，不是挂在 c 的右孩子上；挂错一个，整串遍历顺序全变。
- "后根"和"倒着念先根"不是一回事：后根是左、右、根的递归顺序；先根倒过来念（f,e,d,c,b,a）碰巧在某些小树上像后根，一般树上不对。

先还原森林，再按左孩子右兄弟挂，兄弟挂上一个兄弟的右边。

【坑位】

兄弟链要挂在"上一个兄弟"的右孩子上，不是挂在父亲的右孩子上。

最容易踩的坑，是把 e 当成 c 的右孩子。e 和 d 都是 c 的孩子，d 是第一个孩子（挂 c 的左孩子），e 是 d 的下一个兄弟——按"左孩子、右兄弟"规则，e 要挂在 d 的右孩子上。按错的挂法（e 挂 c 右边），后根走出来是 b,d,f,e,c,a（选项 B）；按对的挂法（e 挂 d 右边），后根是 b,f,e,d,c,a（选项 C）。就这一个挂点之差，答案就从 B 变 C。

【套路】

- 第一步，还原森林：先根开头定根（a 开头 → 第一棵树根是 a），后根结尾定根（后根里 a 在第 2 位 → 第一棵树只有 a、b）。
- 第二步，左孩子右兄弟挂：第一个孩子挂左，下一个兄弟挂上一个兄弟的右。
- 第三步，后根走一遍：左、右、根，递归走。

还原 → 挂（兄弟挂兄右）→ 后根走。

【类比】

像家族企业改组：每个老板的第一个下属当左膀，第二个下属不当右膀、而是当第一个下属的副手（右孩子）。e 是 d 的弟弟，要当 d 的副手，不是直接当 c 的右膀。组织架构挂错一级，下面的汇报顺序全乱。

【口诀】 先根开头后根尾，左孩右兄挂对位。

【考频】 「左孩子右兄弟」在 2009–2022 年解析中出现 3 次 · 中频

第 5 题 · 二叉排序树生成序列（正确答案：B，4，5，1，2，3）

题干与选项：

下列给定的关键字输入序列中，不能生成如下二叉排序树的是：（）

- A. 4，5，2，1，3
- B. 4，5，1，2，3
- C. 4，2，5，3，1
- D. 4，2，1，3，5

> 注：本题题干依赖图片（目标二叉排序树图），图片未能直接查看，本题标注"部分核对"。图中树形由四个输入序列与答案反推：A、C、D 三个序列生成同一棵树——根是 4，左孩子是 2，右孩子是 5；2 的左孩子是 1，右孩子是 3；B 生成另一棵链形树，与答案 B 自洽。图中细节以原卷为准。

入手点： 把每个序列按"小了往左、大了往右、走到空位坐下"的规则实际插一遍，看哪一个插出来的树不一样。

题干专有名词从零解释：

- 先想给一串数找座位：新来的数从根开始比，比当前结点小就往左走，比它大就往右走，走到空椅子就坐下；椅子一旦有人坐，后来的人不能挤开他，只能往他下一层找。这种"左小右大"的树，在卷子上叫"二叉排序树"，意思是每个结点的左子树所有数都比它小、右子树所有数都比它大。
- "输入序列"就是数按什么顺序一个一个进来；"生成"就是按上面的规则插完之后得到的树的形状。
- 注意：插的时候只往下找空位，不调整已经坐好的结点——所以插入顺序不同，树的形状可能不同。

考查知识点：

- 先想"插入规则"：从根出发，小往左、大往右，走到空位坐下；第一个进来的数一定是根（本题四个序列都以 4 开头，所以根都是 4）。
- 再想"为什么不同序列能生成同一棵树"：只要每个数坐下时的"路径"一样，树的形状就一样。A、C、D 虽然顺序不同，但每个数最终都坐到了同一个位置：5 坐到 4 的右边，2 坐到 4 的左边，1 坐到 2 的左边，3 坐到 2 的右边。
- 再想"为什么有的序列生成不了"：座位一旦被占，后来的数就只能往下挂，再也回不到被占的位置。B 的问题就出在这里：1 先坐到了 4 的左孩子位置，2 来了之后比 1 大、比 4 小，只能坐到 1 的右边；3 来了比 2 大、比 4 小，只能坐到 2 的右边。结果 2 不是 4 的左孩子，而是一条链 4→1→2→3，形状全变了。
- 对照"中序遍历"：二叉排序树按左、根、右走出来一定是递增的（本题都是 1，2，3，4，5），所以中序区分不了这四个序列；区分它们的办法是实际插一遍看形状，或者看"谁是谁的孩子"。
- 边界情况：如果输入序列本身就是递增的（比如 1，2，3，4，5），每个新数都坐到前一个的右边，树退化成一条链，查找就和顺序表一样慢；如果输入序列是 4，2，1，3，5 这种，树就比较平衡。本题考的不是平衡，是"能不能生成"。
- 容易踩的坑：以为"只要最后的数集一样，树就一样"——错，顺序决定形状；以为"2 比 4 小就一定是 4 的左孩子"——错，要看 4 的左孩子位置是不是已经被 1 占了。

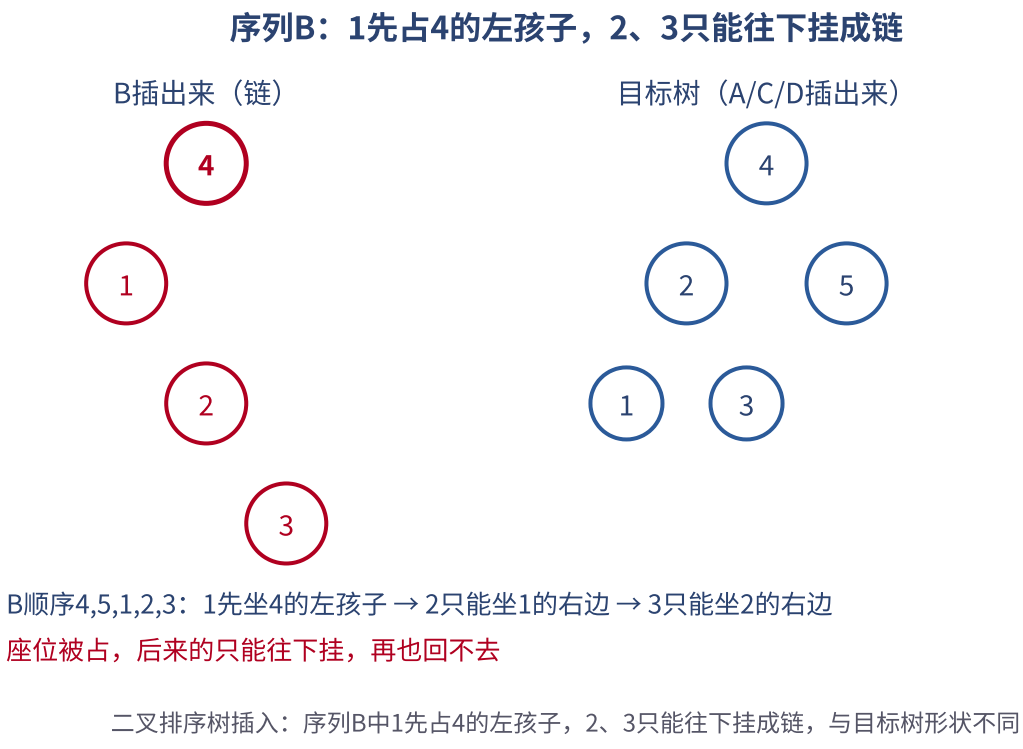
正确选项为什么对： 逐个插 B：4 是根；5 比 4 大，坐到 4 的右边；1 比 4 小，坐到 4 的左边；2 比 4 小往左走，到 1：比 1 大往右走，1 的右边是空的，坐下；3 比 4 小往左走，到 1：比 1 大往右走，到 2：比 2 大往右走，2 的右边是空的，坐下。结果是 4 的左孩子是 1，1 的右孩子是 2，2 的右孩子是 3——一条链，不是目标树（目标树里 4 的左孩子是 2，2 的左右孩子是 1、3）。所以 B 不能生成目标树。选 B。

错误选项为什么错＋举例：

- A（4，5，2，1，3）：插一遍：4 是根；5 坐 4 右边；2 坐 4 左边；1 比 4 小往左到 2，比 2 小往左坐下；3 比 4 小往左到 2，比 2 大往右坐下。结果：4 的左孩子 2、右孩子 5，2 的左孩子 1、右孩子 3——正是目标树，能生成。
- C（4，2，5，3，1）：插一遍：4 是根；2 坐 4 左边；5 坐 4 右边；3 比 4 小往左到 2，比 2 大往右坐下；1 比 4 小往左到 2，比 2 小往左坐下。结果和 A 一模一样，能生成。教训：顺序不同不一定形状不同，关键看每个数坐下时的路径。
- D（4，2，1，3，5）：插一遍：4 是根；2 坐 4 左边；1 坐 2 左边；3 坐 2 右边；5 坐 4 右边。结果同样是目标树，能生成。

笔记本一句话： 第 5 题＋二叉排序树插入题逐个插一遍，座位被占后来的只能往下挂；B 里 1 先占了 4 的左孩子，2 再也回不去。

【图像】



二叉排序树插入：序列B中1先占4的左孩子，2、3只能往下挂成链，与目标树形状不同

【定位】 数据结构 · 第 7 章查找 · 二叉排序树

【易混对比】

- "数集一样"和"形状一样"是两回事：四个序列的数集都是 1,2,3,4,5，但 B 插出来是一条链，A、C、D 插出来是目标树——顺序决定形状。
- "中序遍历"区分不了这四个序列：二叉排序树中序走出来都是递增的 1,2,3,4,5；要区分"能不能生成"，得实际插一遍看形状，或者看"谁是谁的孩子"。
- "2 比 4 小"和"2 是 4 的左孩子"是两回事：2 能不能坐 4 的左孩子，要看那个位置是不是已经被 1 占了；被占了就只能往下挂。

座位被占，后来的只能往下挂，再也回不去。

【坑位】

不要以为"只要最后的数集一样，树就一样"——插入顺序决定树的形状。

最容易踩的坑，是觉得 B 的 2"比 4 小就该是 4 的左孩子"。B 的顺序是 4,5,1,2,3：1 先坐到了 4 的左孩子位置；2 来了之后比 4 小往左走，到 1 发现位置被占，只能比 1 大往右坐到 1 的右边；3 来了比 2 大、比 4 小，只能坐到 2 的右边。结果 4→1→2→3 成了一条链，2 再也回不到 4 的左孩子位置。而 A、C、D 里 2 都是在位置空着时坐下去的，所以三棵树形状一样。

【套路】

- 第一步，定根：第一个进来的数一定是根（本题四个序列都以 4 开头，根都是 4）。
- 第二步，逐个插：小往左、大往右，走到空位坐下；位置被占就继续往下找。
- 第三步，比形状：插完看"谁是谁的孩子"，和目标树对不上就是不能生成。

逐个插一遍，看"谁是谁的孩子"。

【类比】

像抢会议室座位：4 先坐了主位。B 这一波里 1 先抢了 4 左边的椅子，2 再进来想坐 4 左边，发现已经有人了，只能坐到 1 的下手；3 进来更只能往下挤。最后 4 的左手边是 1 不是 2——先到先得，后到的再也挤不回好位置。

【口诀】 小左大右找空位，占了座位往下挂。

【考频】 「二叉排序树」在 2009–2022 年解析中出现 13 次 · 高频

第 6 题 · 修改后 DFS 输出序列（正确答案：B，逆拓扑有序序列）

题干与选项：

修改递归方式实现的图的深度优先搜索（DFS）算法，将输出（访问）顶点信息的语句移到退出递归前（即执行输出语句后立刻退出递归）。采用修改后的算法遍历有向无环图 G，若输出结果中包含 G 中的全部顶点，则输出的顶点序列是 G 的：（ ）

- A. 拓扑有序序列
- B. 逆拓扑有序序列
- C. 广度优先搜索序列
- D. 深度优先搜索序列

入手点： 先想清楚改动只改了"什么时候输出"：原来是一进结点就输出，改后是退出结点时才输出；再看这个改动让边的两端谁先谁后。

题干专有名词从零解释：

- 先想一张任务依赖图：箭头从先修课指向后修课，箭头不能绕一圈回到自己（没有环）。这种"边有方向、没有环"的图，在卷子上叫"有向无环图"，意思是 directed acyclic graph，简称 DAG。
- 再想"深度优先搜索"：从一个点出发，沿着一条路走到头，走不下去了再退回来换条路，好比走迷宫时一条道走到黑——这就是 DFS，意思是 depth-first search。
- 再想"拓扑有序序列"：把所有点排成一排，要求每条箭头都是从前面的点指向后面的点（先修课排在后修课前面）——这样的排列叫拓扑序；把这个排列整个倒过来，就叫"逆拓扑有序序列"（箭头都是从后面的点指向前面的点）。
- "广度优先搜索"（BFS）是另一种走法：一圈一圈往外扩，先把离起点最近的一圈走完，再走下一圈。

考查知识点：

- 先想"退出时输出"意味着什么：一个结点要等它的所有后继（箭头指向的点）都退出并输出之后，自己才退出、才输出。所以输出序列里，后继一定排在前驱前面。
- 再想"小例子"：一条链 u→v→w。DFS 进入 u，再进入 v，再进入 w；退出时先出 w，再出 v，再出 u，序列是 w，v，u。检查每条边：u→v，但 v 在 u 前面；v→w，但 w 在 v 前面——每条边都是"后继在前、前驱在后"，这正是逆拓扑序的定义。
- 再想"为什么是逆拓扑、不是拓扑"：拓扑要求前驱在前（u 在 v 前）；本题输出是前驱在后（v 在 u 前），方向正好反了，所以是逆拓扑，不是拓扑。
- 再想"为什么不是 DFS 序列"：通常说的 DFS 序列是进入结点时就输出的顺序（先根式）；改后是退出时输出，顺序变了，性质也变了——不能因为"还是 DFS 的递归框架"就以为序列还是 DFS 序列。
- 再想"为什么不是 BFS 序列"：BFS 是一圈一圈扩的顺序，和递归退出没有关系；本题的算法还是递归深入、退出才输出，和 BFS 的走法完全不同。
- 前提"输出结果中包含 G 中的全部顶点"的作用：保证每个点都被访问并输出了（图可能不连通时要从每个没访问过的点出发）；如果有的点没被访问，序列就不完整，也就谈不上是全图的逆拓扑序。
- 边界情况：这个结论只对没有环的图成立；有环的图没有拓扑序（环上的点谁先谁后说不清），逆拓扑序也就无从谈起。

正确选项为什么对： 改后的算法在退出递归时才输出。沿着任何一条边 u→v 走：v 一定比 u 先退出（因为 u 要等 v 退出后自己才退出），所以输出序列里 v 一定在 u 前面。所有边都满足"后继在前、前驱在后"，这正是逆拓扑有

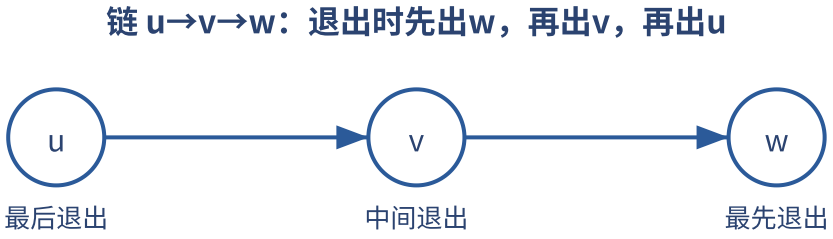
序序列的定义。选 B。

错误选项为什么错＋举例：

- A（拓扑有序序列）：拓扑要求每条边 u→v 都是 u 在前、v 在后；本题恰好反过来（v 在前、u 在后）。拿链 u→v→w 验一下：拓扑序是 u，v，w，本题输出是 w，v，u——是拓扑序倒过来的样子。教训：看到"退出时输出"先想到"顺序反了"，再对号入座。
- C（广度优先搜索序列）：BFS 是起点出发一圈一圈扩，比如起点 u 的邻居先出、邻居的邻居后出；本题是递归深入、退出才出，两者走法不同。拿链 u→v→w 验：BFS 从 u 出发是 u，v，w；本题是 w，v，u——完全不一样。
- D（深度优先搜索序列）：通常说的 DFS 序列是进入结点时就输出（u，v，w 这种先根式）；改后输出时机移到退出时，序列变成了 w，v，u，性质已经变了，不能再叫 DFS 序列。教训：输出时机一改，序列的性质就跟着改，不能只看"用的还是 DFS 框架"。

笔记本一句话： 第 6 题＋DFS 输出移到退出时，等于每个点等后继出完自己才出，序列是逆拓扑（后继在前）；拓扑是反方向，别选反。

【图像】



进入时输出（先根式）：u, v, w → 这是通常说的DFS序列
退出时输出：w, v, u → 每条边都是后继在前 = 逆拓扑序
拓扑序是 u, v, w（前驱在前）→ 方向正好反了，别选反

输出时机一改，序列性质就变：退出输出 = 逆拓扑

链u→v→w：退出时先出w再出v再出u，每条边都是后继在前、前驱在后，即逆拓扑序

【定位】 数据结构 · 第 6 章图 · 图的遍历与拓扑排序

【易混对比】

- "进入时输出"和"退出时输出"是两回事：进入时输出是先根式（u,v,w）；退出时输出是每个点等后继出完自己才出（w,v,u）——输出时机一改，序列的性质就跟着改。
- "拓扑序"和"逆拓扑序"是两回事：拓扑要求每条边前驱在前（u 在 v 前）；本题输出是后继在前（v 在 u 前），方向正好反了，别选反。
- "用的还是 DFS 框架"和"序列还是 DFS 序列"是两回事：递归框架没变，但输出时机移到退出时，序列已经不是通常说的 DFS 序列（先根式）了。

退出才输出，后继一定排在前驱前面——这就是逆拓扑。

【坑位】

看到"退出时输出"先想到"顺序反了"，再对号入座，别被"DFS"三个字带跑。

最容易踩的坑，是因为"用的还是 DFS 的递归框架"就选 D（DFS 序列）。通常说的 DFS 序列是进入结点时就输出的顺序；改后输出时机移到退出递归前，一个结点要等它的所有后继都退出并输出之后，自己才输出。拿链 u→v→w 验一下：进入顺序是 u,v,w，退出顺序是 w,v,u——序列已经反了，性质也变了，不能再叫 DFS 序列。

【套路】

- 第一步，看输出时机：退出时输出 → 每个点等后继出完自己才出。
- 第二步，拿一条边验方向：u→v，v 一定比 u 先退出，所以 v 在 u 前面。
- 第三步，对定义："后继在前、前驱在后"就是逆拓扑有序序列。

退出输出 → 后继在前 → 逆拓扑。

【类比】

像公司报销审批：单子 u 要经过 v、w 两级审批才能办结。w 先签完字退出，v 再签字退出，u 最后签字——签字完成的顺序是 w,v,u，正好和审批流向 u→v→w 反过来。谁先"办结退出"，谁就先出现在名单上。

【口诀】 退出才输出，后继排在前。

【考频】 「逆拓扑」在 2009–2022 年解析中出现 6 次 · 中频

第 7 题 · Kruskal 最小生成树（正确答案：A，{(b,f), (b,d), (a,e), (c,e), (b,e)}）

题干与选项：

已知无向图 G 如下所示，使用克鲁斯卡尔（Kruskal）算法求图 G 的最小生成树，加入到最小生成树中的边依次是（）

- A. {(b,f), (b,d), (a,e), (c,e), (b,e)}
- B. {(b,f), (b,d), (b,e), (a,e), (e,c)}
- C. {(a,e), (b,e), (c,e), (b,d), (b,f)}
- D. {(a,e), (c,e), (b,e), (b,f), (b,d)}

> 注：本题题干依赖图片（无向图 G），图片未能直接查看，本题标注"部分核对"。边加入顺序按答案表写，图中各边的权值未查看；图中细节以原卷为准。

入手点： 记住 Kruskal 就一句话：边按权值从小到大排队，一个一个试，不成环就收下，收到 n−1 条就停。

题干专有名词从零解释：

- 先想把几个村子连起来修路：每个村子是一个点，每条可能修的路是一条边，边上的数是修路的代价（权值）。要把所有村子连通、总代价最小，选出来的那组路就叫"最小生成树"，意思是连通所有点、边数最少（n 个点用 n−1 条边）、权值之和最小。
- 再想 Kruskal 的做法：把所有边按权值从小到大排队，从最便宜的开始试：试一条边时看它两端的点是不是已经连通了——已经连通就跳过（不然会成环），还没连通就收下；收到 n−1 条边就停。
- "成环"就是新边和已收下的边围成一个圈：圈里的路是多余的，收了它总代价只会变大，所以不能收。
- 判断"是不是已经连通"有个工具叫"并查集"：给每个点贴一个组标签，收下一条边就把两端的组标签合并；试新边时看两端标签是不是一样，一样就是已连通。

考查知识点：

- 先想"为什么按权值从小到大试是对的"：贪心——每次收当前最便宜且不成环的边，最后总代价最小；这是 Kruskal 算法正确的核心，考试只要求会执行，不要求证明。
- 再想"收到几条停"：n 个点连通最少要 n−1 条边；本题边里出现的点是 a、b、c、d、e、f，共 6 个点，所以收到 5 条边就停；5 条收满后剩下的边不管多便宜都不再试。
- 再想"成环怎么判断"：新边的两个端点如果已经通过已收下的边连通了，收它就会成环，必须跳过；如果还没连通，收下并把两端合并成一组。
- 对照另一种算法 Prim：Prim 是从一个点出发、一圈一圈往外长（每次收连接已收点集和未收点集的最便宜边）；Kruskal 是全局按边排队、不管点集从哪长。两种算法结果的总权值一样，但选边顺序可能不同——本题问的是 Kruskal 的顺序，不能用 Prim 的顺序答。
- 边界情况：如果图不连通（有的点孤立、怎么都连不上），Kruskal 收不满 n−1 条边，得不到生成树，只能得到生成森林；本题能选出 5 条边，说明图是连通的。
- 容易踩的坑：权值相等的边顺序可以互换（不影响总权值），但本题答案表给出了一个确定的顺序，答题时以答案表为准；还有，成环的边要跳过、不是收下，跳过后继续试下一条，不是停下来。

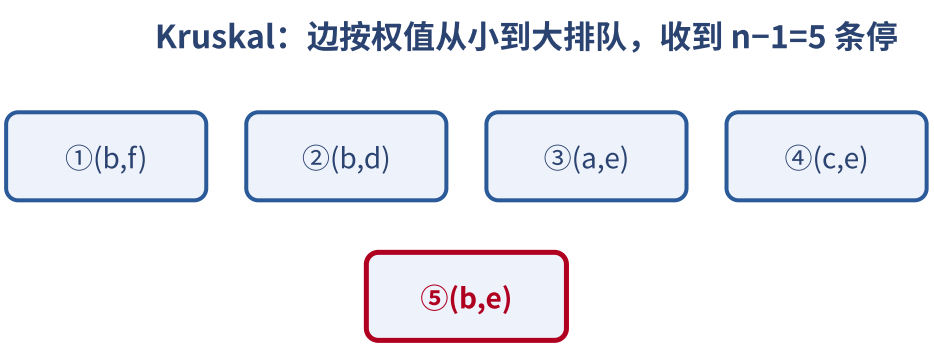
正确选项为什么对： 按答案表，Kruskal 的执行顺序是：先收 (b,f)，再收 (b,d)，再收 (a,e)，再收 (c,e)，最后收 (b,e)，共 5 条边，正好是 6 个点的 n−1 条，停。选项 A 的顺序与答案表一致。选 A。

错误选项为什么错＋举例：

- B（{(b,f), (b,d), (b,e), (a,e), (e,c)}）：前两条和 A 一样，但第三条就收了 (b,e)，把 (a,e) 挤到了第四条——按答案表的权值顺序，(a,e) 比 (b,e) 先被试到。教训：Kruskal 的顺序就是权值从小到大的顺序，一条边提前或推后，整个序列就不对；答题时按权值排队，一条一条对。
- C（{(a,e), (b,e), (c,e), (b,d), (b,f)}）：起手就是 (a,e)，把答案表里最先收的 (b,f)、(b,d) 排到了最后——权值顺序全反了。教训：起手边一定是权值最小的那条，先看最小边是谁，再往后排。
- D（{(a,e), (c,e), (b,e), (b,f), (b,d)}）：同样起手 (a,e)，且 (b,f)、(b,d) 被排到最后，与答案表顺序不符。教训同上：先定最小边，再按权值顺序往后写，不要凭感觉排。

笔记本一句话： 第 7 题＋Kruskal 按权值从小到大试，成环跳过，收到 n−1 条停；顺序就是权值顺序，别凭感觉排。

【图像】



Kruskal执行：边按权值从小到大排队，(b,f)→(b,d)→(a,e)→(c,e)→(b,e)依次收下，5条停

【定位】 数据结构 · 第 6 章图 · 最小生成树

【易混对比】

- "Kruskal"和"Prim"是两回事：Kruskal 是全局按边排队、不管从哪个点长；Prim 是从一个点出发一圈一圈往外长——本题问的是 Kruskal 的选边顺序，不能用 Prim 的顺序答。
- "成环跳过"和"停下来"是两回事：试到成环的边就跳过它、继续试下一条，不是停下来；停下来的条件只有一个——收到 n−1 条边。
- "权值相等的边"顺序可以互换（不影响总权值），但本题答案表给出了确定顺序，答题时以"权值从小到大"为准。

Kruskal 的顺序就是权值从小到大的顺序，别凭感觉排。

【坑位】

起手边一定是权值最小的那条，先定最小边，再按权值顺序往后写。

最容易踩的坑，是凭感觉排顺序（比如选项 C、D 起手就是 (a,e)，把答案表里最先收的 (b,f)、(b,d) 排到了最后）。Kruskal 就一句话：边按权值从小到大排队，一个一个试，不成环就收下。答题时先把最小边定死，再一条

一条往后对——一条边提前或推后，整个序列就不对。

【套路】

- 第一步，边按权值排队：从小到大。
- 第二步，一条一条试：两端还没连通就收下（并查集合并），已经连通就跳过（成环）。
- 第三步，收到 n−1 条停：本题 6 个点，收到 5 条就停，剩下的不再试。

排队 → 试 → 不成环收下 → n−1 条停。

【类比】

像修路队竞标：把所有可修的路段按报价从低到高排队，从最便宜的开始看——这段路如果已经和修好的路连通了（修了也白修，成环），就跳过；还没连通就修。修满 n−1 段、所有村子都连上就收工，不再看后面的报价。

【口诀】 边按权值排队走，不成环就收下。

【考频】 「Kruskal」在 2009–2022 年解析中出现 3 次 · 中频

第 8 题 · AOE 网关键路径（正确答案：B，从源点到汇点路径长度最长的路径）

题干与选项：

若用 AOE 网估算工程进度，则下列叙述中正确的是：（）

- A. 关键路径是从源点到汇点边数最多的一条路径
- B. 关键路径是从源点到汇点路径长度最长的路径
- C. 增加任一关键活动的时间不会延长工程的工期
- D. 缩短任一关键活动的时间将会缩短工程的工期

入手点： 先抓住"关键"的意思：关键路径就是决定总工期的那条路——它有多长，工期就有多长。

题干专有名词从零解释：

- 先想盖一栋楼：每道工序（比如打地基、砌墙）是一条边，边上的数是这道工序要花的时间；每道工序开始和结束的时刻是一个点（里程碑）。这种"点是事件、边是活动、边上有时间"的图，在卷子上叫"AOE 网"，意思是 activity on edge network。
- 再想"源点"和"汇点"：源点是开工那个点（没有箭头指进来），汇点是完工那个点（没有箭头指出来）；从源点到汇点的一条路，就是完成工程的一种工序先后顺序。
- 再想"路径长度"：一条路上所有边的权值（时间）加起来；比如一路上有 3 道工序各花 2 天，路径长度就是 6 天。
- 再想"关键活动"和"关键路径"：没有一点拖延余地的活动叫关键活动（拖一天工期就拖一天）；全由关键活动组成的路叫关键路径；工程的工期就等于关键路径的长度。

考查知识点：

- 先想"工期由谁定"：工程要全部完工，所有路都得走完，所以工期 = 所有源点到汇点路径中最长的那条——这就是关键路径。关键路径可能不止一条（两条路一样长、都是最长，那就都是关键路径）。
- 再想"关键活动的余量"：关键活动的最早开始时间和最迟开始时间相等，没有松弛时间（余量为 0）；非关键活动有余量，在余量内拖延不影响工期。
- 再想"边数和长度是两回事"：边数是数有几条边，长度是把每条边的时间加起来；边多的路不一定长（每条边都很短），边少的路可能很长（每条边都很长）。关键路径看的是长度，不是边数。
- 再想"动关键活动会怎样"：关键活动在最长路径上，它变长，最长路径就变长，工期跟着变长；它变短，工期不一定变短——因为可能还有另一条同样长的关键路径顶着，工期还是原来那个数。
- 边界情况：关键路径唯一时，缩短任一关键活动一定缩短工期；关键路径不唯一时，缩短一条关键路径上的活动，工期可能不变。考试里"缩短任一关键活动将会缩短工期"这种绝对说法，一律按错处理——因为它没加"唯一"这个前提。

正确选项为什么对： 关键路径的定义就是从源点到汇点、路径长度（各边时间之和）最长的那条路径；工期等于它的长度。选项 B 说的就是这个定义。选 B。

错误选项为什么错＋举例：

- A（边数最多的一条路径）：边数和长度是两回事。举例：路 P1 有 3 条边，每条 1 天，长度 3 天；路 P2 有 2 条边，每条 5 天，长度 10 天。P2 是关键路径（最长），但边数比 P1 少。教训：看到"边数最多"先想"边数≠长度"，再举一长一短的例子验一下。

- C（增加任一关键活动的时间不会延长工期）：关键活动在最长路径上，它变长，路径就变长，工期一定变长。举例：关键路径 10 天，其中某个关键活动从 3 天变 5 天，路径变成 12 天，工期变成 12 天。教训：关键活动没有余量，动它工期就动。
- D（缩短任一关键活动的时间将会缩短工期）：不一定，因为可能有另一条同样长的关键路径顶着。举例：两条并行的关键路径都是 10 天，缩短其中一条上的活动 2 天，那条路变成 8 天，但另一条还是 10 天，工期还是 10 天。教训："任一"这种绝对词要警惕——只要能举出一个反例（两条等长关键路径），说法就不成立。

笔记本一句话： 第 8 题＋关键路径看长度不看边数；关键活动变长工期必变长，变短不一定变短（另一条等长关键路径会顶着）。

【图像】



工期 = 最长路径的长度；关键路径可能不止一条

关键活动变长 → 工期一定变长（没有余量）

关键活动变短 → 不一定（另一条等长关键路径顶着）

"缩短任一关键活动将会缩短工期"：绝对词，举反例（两条等长）即错

关键活动=松弛时间为0的活动；非关键活动有余量

关键路径看长度不看边数：3条边各1天（长3天）vs 2条边各5天（长10天），后者是关键路径

【定位】 数据结构 · 第 6 章图 · 拓扑排序和关键路径

【易混对比】

- "边数"和"长度"是两回事：边数是数有几条边，长度是把每条边的时间加起来；3 条边各 1 天（长 3 天）不如 2 条边各 5 天（长 10 天）——关键路径看的是长度，不是边数。
- "关键活动变长"和"关键活动变短"是两回事：变长，工期一定变长（没有余量）；变短，工期不一定变短——可能还有另一条同样长的关键路径顶着。
- "任一"这种绝对词要警惕：只要能举出一个反例（两条等长关键路径），"缩短任一关键活动将会缩短工期"就不成立。

关键路径看长度不看边数；变长必拖期，变短不一定。

【坑位】

关键路径可能不止一条——"缩短任一关键活动将会缩短工期"这种绝对说法，一律按错处理。

最容易踩的坑，是觉得"关键活动缩短了，工期当然缩短"。举例：两条并行的关键路径都是 10 天，缩短其中一条上的活动 2 天，那条路变成 8 天，但另一条还是 10 天，工期还是 10 天。关键活动没有松弛时间不假，但"工期"由所有路径中最长的那条决定——只要还有另一条等长的关键路径顶着，工期就下不来。

【套路】

- 第一步，找最长：工期 = 所有源点到汇点路径中最长的那条。
- 第二步，数有几条最长：一条还是多条——决定"缩短"类说法的对错。
- 第三步，见"任一""将会"先找反例：能举出反例的绝对说法就是错的。

找最长 → 数几条 → 绝对词找反例。

【类比】

像接力赛定总时长：总时长由最慢的那一棒决定。如果有两棒同样慢（都是 10 秒），其中一棒提速到 8 秒，总时长还是 10 秒——另一棒顶在那儿。但任何一棒变慢了，总时长一定变长。关键路径就是"最慢的那一棒"。

【口诀】 关键看长度，变长必拖期，变短看顶替。

【考频】 「关键路径」在 2009–2022 年解析中出现 5 次 · 中频

第 9 题 · 大根堆（正确答案：C，I、II、IV）

题干与选项：

下列关于大根堆（至少含 2 个元素）的叙述中正确的是（ ）

- I. 可以将堆看成一棵完全二叉树；II. 可采用顺序存储方式保存堆；
- III. 可以将堆看成一棵二叉排序树；IV. 堆中的次大值一定在根的下一层。
- A. I、II、III
- B. II、III、IV
- C. I、II、IV
- D. I、II、III、IV

入手点： 四句话逐个验：I、II 是堆的"长相和存法"，III 是拿堆和二叉排序树比，IV 要沿着次大值往上走想一想。

题干专有名词从零解释：

- 先想金字塔式排队：最上面的人最大，下面每一层的人都不超过他上面的人——这种"父结点不小于孩子结点"的结构，在卷子上叫"大根堆"，意思是根最大、越往下越小（或相等）。
- 再想"完全二叉树"：除了最后一层，其他层都排满；最后一层从左往右排，中间不留空位。好比电影院前排坐满、最后一排从左往右坐。
- 再想"顺序存储"：用一排格子（数组）存，结点按完全二叉树的编号对号入座，父亲和孩子靠编号算出来，不用指针。
- 再想"二叉排序树"：左子树所有数都比根小、右子树所有数都比根大，按左、根、右走出来是递增的——这是另一套规矩，和堆不一样。
- 再想"次大值"：除最大值之外最大的那个值；根是最大值，次大值就是"第二名"。

考查知识点：

- 先想"堆的长相"：堆的逻辑结构就是完全二叉树（I 对）；完全二叉树编号连续、没有空缺，正好适合用数组存（II 对）——这也是堆排序用数组实现的原因。
- 再想"堆和二叉排序树的区别"：堆只要求父 ≥ 子，左右孩子之间没有大小关系；二叉排序树要求左 < 根 < 右。举例：根 10，左孩子 9，右孩子 1——是堆（10≥9，10≥1），但按左、根、右走是 9，10，1，不是递增的，所以不是二叉排序树（III 错）。
- 再想"次大值为什么一定在根的下一层"：从次大值往上走，每个祖先都不比它小（堆的规矩：父 ≥ 子）。如果次大值在第 3 层或更深，它上面至少有 2 个不比它小的结点（父亲和根），那它最多排第 3，矛盾；所以它只能在第 2 层，也就是根的下一层（IV 对）。注意：次大值不一定是根的左孩子还是右孩子，两边都有可能。
- 再想"堆的用途"：堆常用来实现优先队列（每次取最大的那个）；堆排序就是反复取根（最大值）再调整的过程；大根堆排出来是升序。
- 边界情况：题干说"至少含 2 个元素"，保证根的下一层确实有结点（不然"次大值在下一层"无从谈起）；如果堆里有相等的值（比如两个 10），次大值就是 10，同样在根的下一层，结论不变。

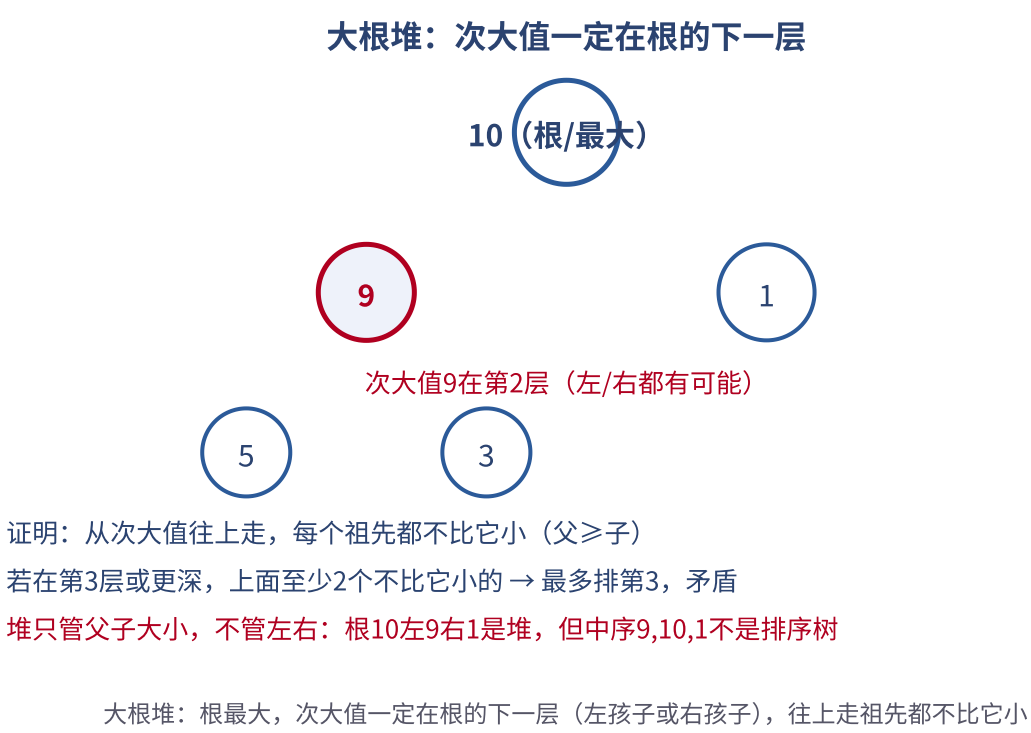
正确选项为什么对： I 对（堆就是完全二叉树）；II 对（完全二叉树适合数组存）；III 错（堆不管左右大小，只管父子大小）；IV 对（次大值往上走，祖先都不比它小，深于第 2 层就排不到第 2 名）。所以正确的是 I、II、IV。选 C。

错误选项为什么错＋举例：

- A（I、II、III）：含了 III。III 错在把堆当成了二叉排序树：堆只要求父 ≥ 子，左右孩子没关系；举例根 10、左 9、右 1 是堆却不是二叉排序树（中序 9，10，1 不递增）。教训：看到"堆看成二叉排序树"先想"左右要不要比"——堆不要，排序树要。
- B（II、III、IV）：同样含了 III（错），还漏了 I（I 是对的，堆的逻辑结构就是完全二叉树）。教训：多选题要逐个验，不能只看"含不含错的"，还要看"有没有漏对的"。
- D（I、II、III、IV）：全选，含了错的 III。教训："全选"在多选题里往往是陷阱——只要有一句错，全选就错；逐句验比猜"全选"可靠。

笔记本一句话： 第 9 题＋堆是完全二叉树、数组存、父 ≥ 子不管左右；次大值一定在根下一层（往上走祖先都不比它小）。

【图像】



大根堆：根最大，次大值一定在根的下一层（左孩子或右孩子），往上走祖先都不比它小

【定位】 数据结构 · 第 8 章排序 · 堆

【易混对比】

- "堆"和"二叉排序树"是两回事：堆只要求父 ≥ 子，左右孩子之间没有大小关系；二叉排序树要求左 < 根 < 右。根 10、左 9、右 1 是堆（10≥9，10≥1），但中序 9,10,1 不递增，所以不是二叉排序树。
- "堆的逻辑结构"和"堆的存储结构"是两回事：逻辑上是完全二叉树，存储上用数组（顺序存储）——完全二叉树编号连续无空缺，正好适合数组存。
- "最大值"和"次大值"位置不同：最大值一定是根；次大值一定在根的下一层（左孩子或右孩子都有可能）。

堆只管父子大小，不管左右大小；次大值一定在根下一层。

【坑位】

看到"堆看成二叉排序树"先想"左右要不要比"——堆不要，排序树要。

最容易踩的坑，是把 III（堆看成二叉排序树）当成对的。堆的规矩只有一条：父结点不小于孩子结点。左右孩子之间没有任何大小要求——左孩子 9、右孩子 1 可以，左孩子 1、右孩子 9 也可以。而二叉排序树要求左 < 根 <

右，左右必须比。拿反例验一下：根 10、左 9、右 1，中序走出来是 9,10,1，不是递增的，所以它不是二叉排序树，但它是堆。

【套路】

- 第一步，逐句验：Ⅰ（堆=完全二叉树？对）、Ⅱ（数组存？对）、Ⅲ（=二叉排序树？错）、Ⅳ（次大值位置？对）。
- 第二步，有疑问就画小例子：根 10、左 9、右 1，一句话验 Ⅲ。
- 第三步，次大值往上走：每个祖先都不比它小，深于第 2 层就排不到第 2 名。

逐句验，有疑问画小例子。

【类比】

像金字塔式排队：塔尖的人最大，每个人都不超过他上面的人。第二强的人（次大值）一定在塔尖的下一层——如果他在第 3 层，他上面至少有两个人（父亲和塔尖）都不比他弱，那他最多排第 3，矛盾。所以第二名只能紧挨着第一名站。

【口诀】 堆是完全树，数组存，父大子小不管左右。

【考频】 「大根堆」在 2009–2022 年解析中出现 19 次 · 高频

第 10 题 · 4 阶 B 树插入后根（正确答案：B，6，9）

题干与选项：

依次将关键字 5，6，9，13，8，2，12，15 插入初始为空的 4 阶 B 树后，根节点中包含的关键字是（）

- A. 8
- B. 6，9
- C. 8，13
- D. 9，12

入手点： 一个一个插，插到叶结点；哪个结点满了（4 个关键字）就分裂，中间关键字升上去；最后看根里剩谁。

题干专有名词从零解释：

- 先想多层抽屉柜：一个抽屉（结点）里能放多个关键字，关键字从小到大排，关键字之间是通往下一层抽屉的隔板。每个结点最多有 m 个孩子，就叫"m 阶 B 树"，意思是 m 阶。
- 再想"4 阶"的具体数字：每个结点最多 4 个孩子、最多 3 个关键字；除了根，每个结点至少 2 个孩子、至少 1 个关键字；根至少 1 个关键字（空树除外）。
- 再想"分裂"：一个结点有了 4 个关键字就满了，必须分裂——把中间那个关键字升到上一层，剩下的关键字分成左右两个结点。
- 再想"插入只插到叶结点"：新关键字一路往下比，找到最底层的叶结点再放进去；放满了就分裂，分裂可能一路向上传，但本题只传到根这一层。

考查知识点：

- 先想"B 树是干什么的"：磁盘读写一次读一个结点（一个块），一个结点能放多个关键字，树就很矮，找数据时读盘次数少——B 树是为磁盘设计的，二叉排序树一个结点只有一个关键字，树高，读盘次数多。
- 再想"4 阶的分裂规则"：结点有 4 个关键字时，取第 2 个关键字升到父结点，左边剩第 1 个，右边剩第 3、4 个；如果父结点也满了，继续分裂；如果根满了分裂，树长高一层。
- 再想"插入的完整过程"（本题 8 个数）：
- 插 5：根 [5]；插 6：根 [5，6]；插 9：根 [5，6，9]。
- 插 13：根变成 [5，6，9，13]，满了，分裂：第 2 个 6 升上去当根，左边 [5]，右边 [9，13]。树变成根 [6]，孩子 [5]、[9，13]。
- 插 8：8 比 6 大，往右孩子 [9，13] 走，8 比 9 小，放进去得 [8，9，13]。
- 插 2：2 比 6 小，往左孩子 [5] 走，放进去得 [2，5]。
- 插 12：12 比 6 大，往右孩子 [8，9，13] 走，12 在 9 和 13 之间，放进去得 [8，9，12，13]，满了，分裂：第 2 个 9 升到根，根变成 [6，9]，左边 [8]，右边 [12，13]。
- 插 15：15 比 9 大，往右孩子 [12，13] 走，放进去得 [12，13，15]。
- 最后：根 [6，9]，三个孩子分别是 [2，5]、[8]、[12，13，15]。
- 对照"二叉排序树"：二叉排序树一个结点一个关键字，插入不分裂；B 树一个结点多个关键字，满了要分裂——这是 B 树题和排序树题最大的不同。
- 边界情况：根分裂时树长高一层（本题第一次分裂就是根分裂，但根原来只有一个结点，分裂后 6 成了新根，树从 1 层变 2 层）；删除时还有合并、借关键字等操作，本题不考插入之外的操作。

- 容易踩的坑：分裂时升上去的是中间关键字，不是最大或最小的；插入要先找到叶结点，不能插到中间结点；4 阶的"4"指的是孩子数，不是关键字数（关键字数最多 3 个）。

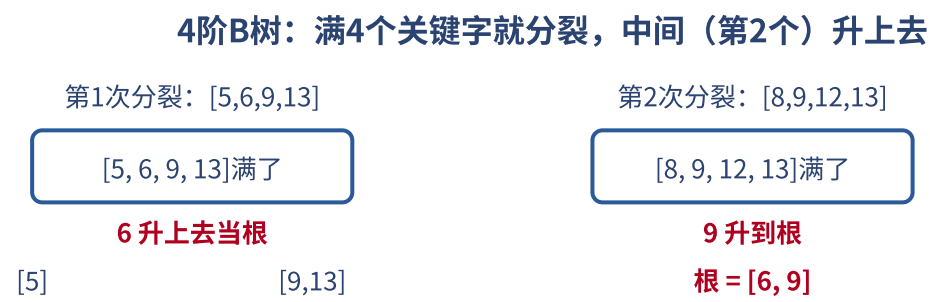
正确选项为什么对： 按上面的过程插完 8 个数，根里是 6 和 9。选 B。

错误选项为什么错＋举例：

- A（8）：8 在叶结点 [8] 里，不在根里。教训：分裂时升上去的是中间关键字（6、9），不是后来插进来的 8；答题时先把整棵树插完再看根，不要凭"最后插的数"猜。
- C（8，13）：8 和 13 从不在同一个结点里——8 在叶结点 [8]，13 在叶结点 [12，13，15]。教训：根里的关键字是分裂时升上去的（6、9），叶结点里的数再大也不会跑到根里。
- D（9，12）：9 确实在根里，但 12 在叶结点 [12，13，15] 里，不在根里。教训：9 是第二次分裂升上去的，12 是后来插进右孩子的，两者不在同一层。

笔记本一句话： 第 10 题＋4 阶 B 树一个结点最多 3 个关键字，满 4 个就分裂、中间（第 2 个）升上去；插完再看根，别凭感觉猜。

【图像】



4阶的"4"指孩子数：每结点最多4个孩子、最多3个关键字
分裂：4个关键字里第2个升上去，左剩第1个，右剩第3、4个
插入只插到叶结点；8个数插完：根[6,9]，孩子[2,5]、[8]、[12,13,15]

根里的数是分裂升上去的（6、9），不是最后插的（8、12）

B树为磁盘设计：一个结点多个关键字，树矮，读盘次数少
二叉排序树一个结点一个关键字，插入不分裂——这是最大不同

4阶B树插入：满4个关键字就分裂，中间（第2个）升上去；插完8个数根是6,9

【定位】 数据结构 · 第 7 章查找 · B 树

【易混对比】

- "4 阶的 4"指的是孩子数，不是关键字数：4 阶 B 树一个结点最多 4 个孩子、最多 3 个关键字；满 4 个关键字就分裂。
- "升上去的是中间关键字"，不是最大或最小的：4 个关键字里第 2 个升上去，左边剩第 1 个，右边剩第 3、4 个。
- "B 树"和"二叉排序树"是两回事：二叉排序树一个结点一个关键字，插入不分裂；B 树一个结点多个关键字，满了要分裂——这是 B 树题和排序树题最大的不同。

满 4 个就分裂，中间（第 2 个）升上去。

【坑位】

插入要先找到叶结点，不能插到中间结点；分裂时升的是中间关键字，别凭"最后插的数"猜根。

最容易踩的坑，是凭"最后插的数"猜根（比如选 8 或 12）。根里的关键字是分裂时升上去的：第一次分裂升 6，第二次分裂升 9，所以根是 6,9。8 在叶结点 [8] 里，12 在叶结点 [12,13,15] 里——叶结点里的数再大也不会跑到根里。答题时先把整棵树插完再看根，不要猜。

【套路】

- 第一步，一路往下找到叶结点：新关键字只插到叶结点。
- 第二步，满 4 个就分裂：第 2 个关键字升到父结点，左右各剩一边。
- 第三步，插完再看根：8 个数全插完，最后看根里剩谁。

插到叶 → 满 4 分裂 → 中间升 → 插完看根。

【类比】

像多层抽屉柜：一个抽屉（结点）最多放 3 个文件夹（关键字），放第 4 个就得分——把中间那个文件夹拿到上一层抽屉，剩下的分两摞。最后看顶层抽屉里剩哪几个文件夹，那就是根的关键字。

【口诀】 满四分裂中间升，插完再看根是谁。

【考频】 「B树」在 2009–2022 年解析中出现 2 次 · 低频

第 11 题 · 直接插入比简单选择高效原因（正确答案：A，仅 I）

题干与选项：

对大部分元素已有序的数组进行排序时，直接插入排序比简单选择排序效率更高，其原因是：（）

- I. 直接插入排序过程中元素之间的比较次数更少
- II. 直接插入排序过程中所需要的辅助空间更少
- III. 直接插入排序过程中元素的移动次数更少
- A. I
- B. III
- C. I，II
- D. I，II，III

入手点： 抓住前提"大部分元素已有序"：两种排序在这个前提下，谁省了什么，一项一项算。

题干专有名词从零解释：

- 先想整理一手扑克牌：左手已经排好的牌是"有序区"，右手没排的是"无序区"。直接插入排序就是每次从右手摸一张，往左手里找位置插进去——这就叫"直接插入排序"，意思是逐个插入。
- 再想简单选择排序：每次从没排的部分里找出最小的一张，和没排部分的第 1 张交换——这就叫"简单选择排序"，意思是每次选最小。
- 再想"比较次数"和"移动次数"：比较次数是"比了几次大小"，移动次数是"挪了几次位置"；辅助空间是"额外用了几个格子"。
- 再想"大部分已有序"：数组里绝大多数元素已经在正确位置上，只有少数几个乱了。

考查知识点：

- 先想"直接插入的比较次数"：左手已经排好了，新摸的牌只要和左手最右边那张比一次，位置对了就不用再比——所以基本有序时，比较次数大约是 n 次（每张牌比 1 次），时间复杂度最好是 O(n)。
- 再想"简单选择的比较次数"：不管数组有没有序，每趟都要把没排的部分从头扫到尾找最小——比较次数恒为 $n(n-1)/2$，和初始顺序一点关系都没有，时间复杂度恒为 $O(n^2)$。举例：n = 100 时，直接插入大约比 100 次，简单选择要比 4950 次，差了约 50 倍。
- 再想"辅助空间"：直接插入只用一个临时格子放当前要插的牌；简单选择只用一个格子记最小值的下标（交换时再用一个临时格子）——两者都是 O(1)，一样多，没有谁更少。
- 再想"移动次数"：简单选择每趟最多交换一次，总共至多 n−1 次交换，而且移动次数和初始顺序无关——本来就很少；直接插入在基本有序时移动也不多，但两者移动次数差距不大（完全有序时两者移动都是 0 次）。所以"移动次数更少"不是直接插入胜出的原因——胜出靠的是比较次数从 $n^2/2$ 降到 n 附近。
- 再想"稳定性"：直接插入是稳定的（相等的元素相对顺序不变），简单选择是不稳定的（交换可能打乱相等元素的顺序）——这是附加区别，本题不直接考，但容易在相邻题里混淆。
- 边界情况：如果数组完全逆序，直接插入的比较和移动都变成 $O(n^2)$，优势就没了；本题的前提是"大部分已有序"，优势才成立。

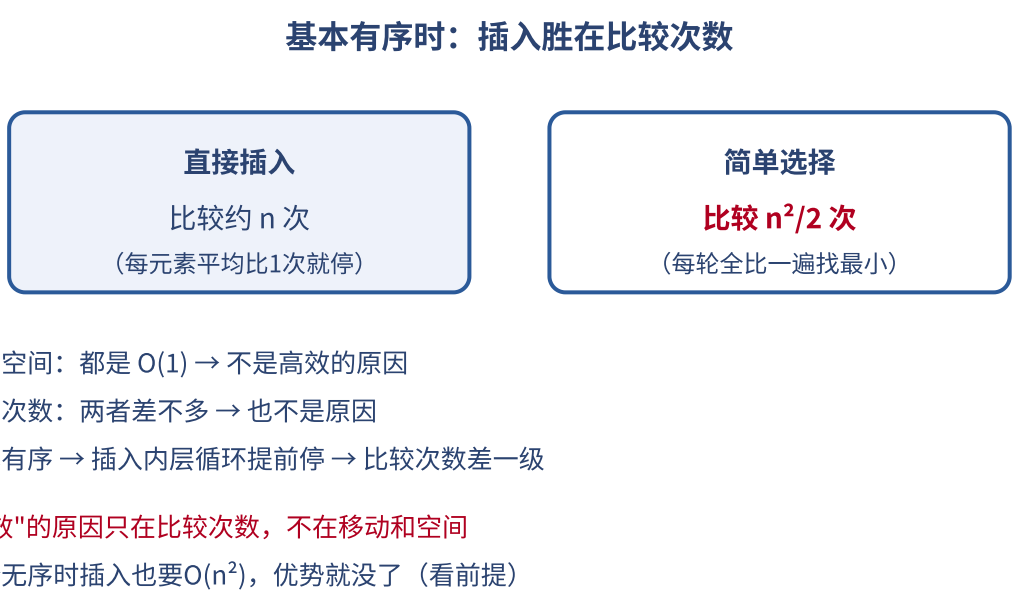
正确选项为什么对： 基本有序时，直接插入每张牌大约比 1 次，总共约 n 次；简单选择不管有没有序都要比 $n(n-1)/2$ 次。所以 I 对：比较次数更少是直接插入更高效的原因。选 A。

错误选项为什么错＋举例：

- B（III）：III 说移动次数更少。实际上简单选择的移动本来就很少（每趟最多换一次，总共至多 n−1 次交换，和顺序无关）；完全有序时两者移动都是 0 次，没有"更少"。教训：比较次数和移动次数是两回事——本题拉开差距的是比较次数，不是移动次数。
- C（I，II）：含了 II。II 说辅助空间更少，实际上两者都是 O(1)（各用一个临时格子），一样多。教训：辅助空间要先算量级——都是 O(1) 就没有"更少"。
- D（I，II，III）：含了错的 II 和 III。教训：多选题逐句验，I 对不代表全对；II、III 各有一个反例（空间一样多、移动差不多），全选就错。

笔记本一句话： 第 11 题＋基本有序时插入胜在比较次数（约 n 次对 $n^2/2$ 次）；辅助空间都是 O(1)，移动次数两者差不多，别把原因安错地方。

【图像】



【定位】 数据结构 · 第 8 章排序 · 插入排序与选择排序

【易混对比】

- "比较次数"和"移动次数"是两回事：基本有序时插入胜在比较次数（约 n 次对 $n^2/2$ 次）；移动次数两者差不多——别把"高效"的原因安到移动次数上。
- "辅助空间"两者一样：都是 O(1)，只用常数个临时变量；"高效"的原因不在空间上。
- "基本有序"和"完全无序"是两回事：基本有序时插入的内层循环很快停下；完全无序时插入也要 $O(n^2)$，优势就没了——本题的前提是"基本有序"。

基本有序时，插入胜在比较次数，不在移动次数和空间上。

【坑位】

别把"高效"的原因安错地方：辅助空间都是 O(1)，移动次数也差不多，差的是比较次数。

最容易踩的坑，是觉得"插入排序移动元素少所以快"。实际上基本有序时，直接插入每个元素平均只比较 1 次就找到位置（约 n 次），而简单选择不管有序无序，每轮都要把剩下所有元素比一遍找最小（$n^2/2$ 次）。移动次数上两者差不多（都要把元素挪位），辅助空间都是 O(1)。所以"高效"的原因只在比较次数，不在移动和空间。

【套路】

- 第一步，看前提：基本有序 → 插入的内层循环提前停。
- 第二步，算比较次数：插入约 n 次，选择 $n^2/2$ 次。
- 第三步，排除干扰项：空间都是 O(1)，移动差不多——原因只能是比较次数。

看前提 → 算比较 → 排除干扰。

【类比】

像整理基本排好的扑克牌：直接插入是拿起一张牌，往前比一两张就找到位置插进去（比较很少）；简单选择是每轮都要把剩下的牌全翻一遍找最小的（比较很多）。牌堆移动的次数差不多，差的是"翻牌看"的次数。

【口诀】 基本有序插入胜，比较次数差一级。

【考频】 「直接插入」在 2009–2022 年解析中出现 8 次 · 中频

第 12 题 · 一定与机器字长相同部件（正确答案：B，仅 I、III）

题干与选项：

下列给出的部件中，其位数（宽度）一定与机器字长相同的是（ ）。

- I. ALU
- II. 指令寄存器
- III. 通用寄存器
- IV. 浮点寄存器
- A. 仅 I、II
- B. 仅 I、III
- C. 仅 II、III
- D. 仅 II、III、IV

入手点： 先定住"机器字长"的意思（CPU 一次能并行处理几位），再逐个部件问：它的宽度是不是由这个数决定的。

题干专有名词从零解释：

- 先想 CPU 一次能搬几块砖：一次能并行处理的数据位数，在卷子上叫"机器字长"，意思是 CPU 一次能处理的数据宽度（比如 64 位机就是 64 位）。
- 再想做算术的车间：专门做加减乘除和逻辑运算的部件叫"ALU"，意思是算术逻辑单元。
- 再想 CPU 手边的暂存格：放操作数和中间结果的通用格子叫"通用寄存器"，意思是谁都能用的寄存器。
- 再想放当前指令的格子：正在执行的那条指令存在"指令寄存器"里，意思是 instruction register，简称 IR。
- 再想放小数的格子：专门放浮点数（带小数点的数）的寄存器叫"浮点寄存器"。

考查知识点：

- 先想"ALU 为什么一定相同"：ALU 一次算几位，就是 CPU 一次能处理几位——ALU 的宽度就是机器字长的定义本身。64 位机的 ALU 一次算 64 位，32 位机的 ALU 一次算 32 位（I 对）。
- 再想"通用寄存器为什么一定相同"：通用寄存器放的是一个"字"（操作数、地址、结果），一个字的宽度就是机器字长。64 位机通用寄存器 64 位，32 位机 32 位（III 对）。
- 再想"指令寄存器为什么不一定相同"：指令寄存器的宽度由一条指令有几位决定，叫"指令字长"；指令字长和机器字长是两个数，不一定相等。比如某机器字长 32 位，但指令有 16 位、32 位、48 位好几种，指令寄存器要按最长的指令设计，宽度就不等于 32 位（II 错）。
- 再想"浮点寄存器为什么不一定相同"：浮点寄存器的宽度由浮点数格式决定（单精度 32 位、双精度 64 位、扩展精度 80 位），和机器字长没关系。举例：32 位机也可以有 64 位浮点寄存器（放双精度数）；64 位机也可以有 80 位浮点寄存器（x87 扩展精度）（IV 错）。
- 对照三个容易混的"字长"：机器字长（CPU 一次处理几位）、存储字长（一个存储单元有几位）、指令字长（一条指令有几位）——三者没有必然相等关系，考试常拿来混淆。
- 边界情况："一定"两个字是关键——只要能举出一个不相等的例子，该部件就不算。II、IV 各有反例，所以出局；I、III 找不到反例，才算"一定"。

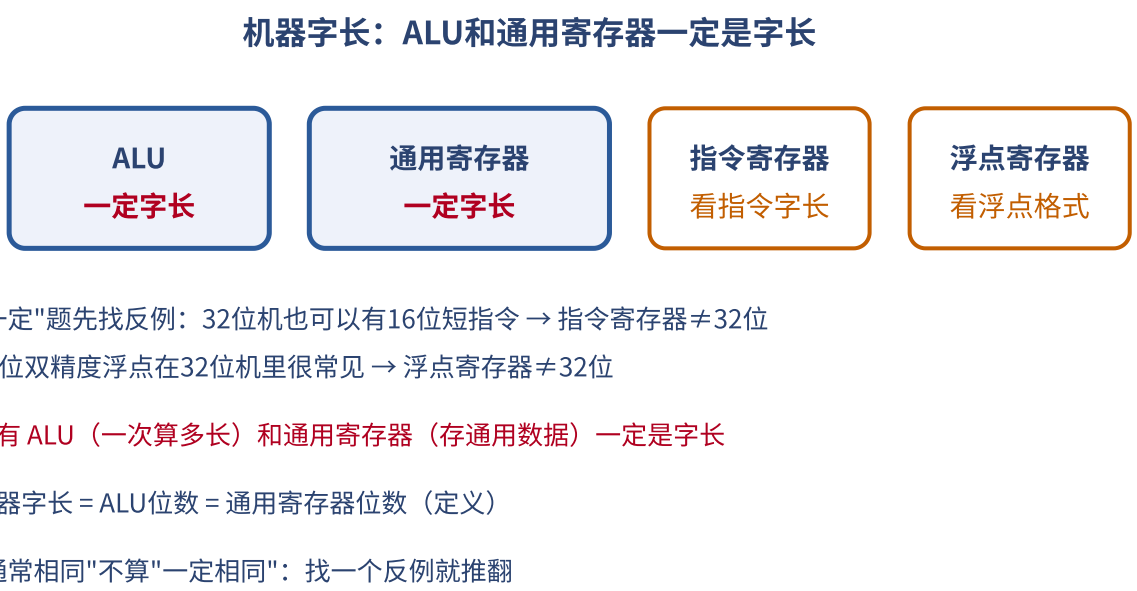
正确选项为什么对： ALU 的宽度就是机器字长本身，通用寄存器放一个字所以也是机器字长；指令寄存器由指令字长定，浮点寄存器由浮点格式定，两者都不一定等于机器字长。所以只有 I、III。选 B。

错误选项为什么错＋举例：

- A（仅 I、II）：含了 II（错：指令寄存器由指令字长定，比如字长 32 位、指令 48 位时 IR 就是 48 位），还漏了 III（通用寄存器一定是机器字长）。教训：多选题既要看"含不含错的"，也要看"有没有漏对的"。
- C（仅 II、III）：含了 II（错，同上），还漏了 I（ALU 一定是机器字长）。教训同上：I、III 是一对，都对，漏一个就不完整。
- D（仅 II、III、IV）：含了 II（错）和 IV（错：浮点寄存器由浮点格式定，32 位机也能有 64 位浮点寄存器），还漏了 I。教训："一定"题先找反例——II、IV 各有一个反例，先排除，再看剩下的。

笔记本一句话： 第 12 题＋ALU 和通用寄存器一定是机器字长；指令寄存器看指令字长，浮点寄存器看浮点格式——"一定"题先找反例。

【图像】



机器字长：ALU和通用寄存器一定是字长；指令寄存器看指令字长，浮点寄存器看浮点格式

【定位】 计算机组成原理 · 第 1 章计算机系统概述 · 机器字长

【易混对比】

- "ALU/通用寄存器"和"指令寄存器"是两回事：ALU 算多长的数据、通用寄存器存多长的数据，一定是机器字长；指令寄存器存的是指令，看指令字长，不一定等于机器字长。
- "通用寄存器"和"浮点寄存器"是两回事：通用寄存器存整数地址数据，是字长；浮点寄存器存浮点数，看浮点格式（比如 64 位双精度），不一定等于字长。
- "一定"和"通常"是两回事：题干问"一定相同"，找一个反例就能推翻——"通常相同"不算对。

ALU 和通用寄存器一定是字长；指令看指令字长，浮点看浮点格式。

【坑位】

"一定"题先找反例：能举出一个不等的情况，这个部件就不选。

最容易踩的坑，是觉得"寄存器都和字长一样"。指令寄存器存指令，指令字长和机器字长是两套概念——比如 32 位机也可以有 16 位的短指令，指令寄存器就不一定是 32 位。浮点寄存器存浮点数，64 位双精度浮点数在 32 位机里很常见。只有 ALU（一次算多长）和通用寄存器（存通用数据），才一定和机器字长相同。

【套路】

- 第一步，定标准：机器字长 = ALU 一次算的位数 = 通用寄存器位数。
- 第二步，逐个找反例：指令寄存器（指令字长可变）→ 有反例，不选；浮点寄存器（看浮点格式）→ 有反例，不选。
- 第三步，剩下的就是"一定"：ALU、通用寄存器。

定标准 → 找反例 → 剩下的是答案。

【类比】

像问"哪些工人的工牌一定和公司编号位数一样"：一线工人（ALU）和仓库管理员（通用寄存器）的工牌就是公司编号；但门禁卡（指令寄存器）按门禁系统定，食堂饭卡（浮点寄存器）按食堂系统定——这两的位数和公司编号没必然关系。

【口诀】 ALU通用必字长，指令浮点各看各。

【考频】 「机器字长」在 2009–2022 年解析中出现 1 次 · 低频

第 13 题 · 机器数 C800 0000H 的值（正确答案：A，−7×2^27）

题干与选项：

已知带符号整数用补码表示，float 型数据用 IEEE 754 标准表示，假定变量 x 的类型只能是 int 或 float。当 x 的机器数为 C800 0000H 时，x 的值可能是：（）

- A. −7×2^27
- B. −2^16
- C. 2^17
- D. 25×2^27

入手点： 同一串 0/1，按 int 读是一种值，按 float 读是另一种值——两种都算一遍，看哪个在选项里。

题干专有名词从零解释：

- 先想同一串字母按英文读和按拼音读意思不同：计算机里 32 个 0/1 到底表示哪个数，要看它是什么类型——这就叫"机器数"，意思是存在机器里的 0/1 串；同一个机器数，类型不同，值就不同。
- 再想"补码"：带符号整数（可正可负的整数）在机器里存的方法——正数直接存，负数存"取反加 1"的结果；最高位是符号位（0 正 1 负）。
- 再想 float 的存法（IEEE 754 标准）：32 位分成三段——第 1 位是符号位，第 2–9 位是阶码（指数部分，存的是指数 + 127，叫"偏置 127"），后 23 位是尾数（小数部分，前面默认还有个 1.，不存出来）。

考查知识点：

- 先想"按 int（补码）读"：C800 0000H 写成 0/1 是 1100 1000 0000…（后面全 0）。最高位 1，是负数。求真值：取反（0011 0111 1111…）再加 1，得 0011 1000 0000…，即 0x38000000。0x38000000 = 2^29 + 2^28 + 2^27 = 7×2^27，所以 int 的值是 −7×2^27。
- 再想"按 float 读"：符号位 1（负数）；阶码是接下来的 8 位 10010000，换成十进制是 144，减去偏置 127，得指数 17；尾数全 0，前面默认 1.，所以值是 −1.0×2^17 = −2^17。注意：−2^17 不在选项里，2^17（选项 C）是漏了符号位的版本。
- 再想"特殊值"：补码里 0x80000000 是最小的数 −2^31（取反加 1 还是它自己），0x7FFFFFFF 是最大的数 2^31−1；float 里阶码全 0 是 0 或很小的非规格化数，阶码全 1 是无穷大或 NaN（不是数）——本题阶码是 144，既不是全 0 也不是全 1，是正常数。
- 对照"无符号数"：如果把 C800 0000H 当成无符号数（没有正负），直接算就是 2^31 + 2^30 + 2^27 = 25×2^27（选项 D）——但题干说 x 是带符号整数或 float，不是无符号数，所以 D 不对。
- 边界情况：机器数 C800 0000H 的 int 值 −7×2^27 在 int 范围（−2^31 ~ 2^31−1）内，合法；float 值 −2^17 也合法；题目问"可能是"，两种解读各算一个，哪个在选项里就是答案。

正确选项为什么对： 按 int 的补码读：符号位 1 是负数，取反加 1 得 0x38000000 = 7×2^27，所以值是 −7×2^27。选 A。

错误选项为什么错＋举例：

- B（−2^16）：这是 float 读法里指数算错一位的结果——阶码 144 减 127 是 17，不是 16；如果误把阶码看成 143（少算 1），就会得 −2^16。教训：阶码 10010000 换成十进制要一位一位算（128 + 16 = 144），再减 127，别凭感觉减。

- C（2^17）：这是 float 读法漏了符号位的结果——符号位是 1，值应该是负的 −2^17；漏看最高位就变成正的 2^17。教训：float 的第一位永远是符号位，算值之前先看它是 0 还是 1。
- D（25×2^27）：这是把补码当无符号数直接读的结果（2^31 + 2^30 + 2^27）。教训：题干说了"带符号整数用补码表示"，负数要取反加 1 再添负号，不能直接当无符号数算。

笔记本一句话： 第 13 题＋同一机器数两种解读都算：int 补码负数取反加 1，float 阶码减 127、别漏符号位。

【图像】



【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 机器数（补码/FIEEE 754）

【易混对比】

- "按 int 读"和"按 float 读"是两回事：同一串 0/1，类型不同值就不同——两种解读都要算一遍，看哪个在选项里。
- "补码负数"和"无符号数"是两回事：带符号整数最高位 1 是负数，要取反加 1 再添负号；直接当无符号数算（25×2^27）是选项 D 的陷阱。
- "2^17"和"−2^17"是两回事：float 读法的符号位是 1，值是负的；漏看最高位就变成正的 2^17（选项 C 的陷阱）。

同一机器数两种解读都算：int 取反加 1，float 阶码减 127、别漏符号位。

【坑位】

float 的第一位永远是符号位，算值之前先看它是 0 还是 1。

最容易踩的坑，是 float 读法漏了符号位。C800 0000H 的最高位是 1：按 float 读，符号位 1 说明是负数，阶码 10010000 = 144，减 127 得指数 17，尾数全 0，所以值是 −2^17。漏看符号位就会选 C（2^17）。另一个坑是阶码算错：10010000 = 128 + 16 = 144，一位一位算再减 127，别凭感觉。

【套路】

- 第一步，按 int（补码）读：最高位 1 是负数，取反加 1 得真值，添负号。

- 第二步，按 float 读：符号位 → 阶码 8 位转十进制减 127 → 尾数。
- 第三步，两个结果对选项：哪个在选项里就是答案。

int、float 各算一遍，对号入座。

【类比】

像同一串字母按英文读和按拼音读意思不同：C800 0000H 这串 0/1，按"整数话"读是 −7×2^27，按"浮点话"读是 −2^17。读之前先问"这是什么类型"，类型不同，读音（值）就不同。

【口诀】 同串两读都算遍，补码取反加一，浮点阶码减一二七。

【考频】 「IEEE 754」在 2009–2022 年解析中出现 23 次 · 高频

第 14 题 · 小端对齐结构体字节地址（正确答案：D，2020FE06H）

题干与选项：

在按字节编址，采用小端方式的 32 位计算机中，按边界对齐方式为以下 C 语言结构型变量 a 分配存储空间。

struct record{ short x1; int x2;}a;

若 a 的首地址为 2020FE00H，a 的成员变量 x2 的机器数为 12340000H，则其中 34H 所在存储单元的地址是：

- A. 2020FE03H
- B. 2020FE04H
- C. 2020FE05H
- D. 2020FE06H

入手点： 两步走：先按对齐规则定 x2 从哪个地址开始，再按小端规则定 34H 是 x2 的第几个字节。

题干专有名词从零解释：

- 先想按字节编址：内存一个格子放一个字节（8 位），每个格子有个地址，地址加 1 就是下一个格子。
- 再想小端方式：一个多字节的数存进格子时，低位字节（最小的那部分）放在低地址（最前面的格子），高位字节放在高地址——好比"小头在前"，倒着放。反过来叫"大端方式"：高位字节在低地址，顺着放。
- 再想边界对齐：每个成员要放在自己大小倍数的地址上——short 是 2 字节，要放在偶数地址；int 是 4 字节，要放在 4 的倍数地址；中间对不上的格子空出来，叫"填充"（不放数据）。
- 再想 short 和 int 的大小：32 位机里 short 是 2 字节，int 是 4 字节。

考查知识点：

- 先想"对齐后 x2 在哪"：a 从 2020FE00H 开始。x1 是 short，占 2 字节：FE00、FE01。接下来 x2 是 int，要放在 4 的倍数地址：FE02、FE03 不是 4 的倍数，空出来当填充；FE04 是 4 的倍数，x2 从 FE04 开始，占 FE04、FE05、FE06、FE07。
- 再想"x2 的 4 个字节怎么排"：x2 的机器数是 12340000H，拆成 4 个字节：最高字节 12H，次高 34H，次低 00H，最低 00H。小端规则是低位字节在低地址：最低字节 00H 放 FE04，次低 00H 放 FE05，次高 34H 放 FE06，最高 12H 放 FE07。所以 34H 在 FE06。
- 对照"大端"：大端是高位字节在低地址，x2 从 FE04 开始的话，FE04 放 12H，FE05 放 34H——34H 就在 FE05（选项 C）。本题是小端，不能用大端算。
- 对照"不对齐"：如果不对齐，x2 紧跟 x1 从 FE02 开始，小端排：FE02 放 00H，FE03 放 00H，FE04 放 34H——34H 就在 FE04（选项 B）。本题要求边界对齐，不能跳过填充。
- 边界情况：结构体总大小也要按最大成员对齐：本题 x1（2 字节）+ 填充（2 字节）+ x2（4 字节）= 8 字节，正好是 4 的倍数；如果 x1 是 3 字节之类的，总大小还要补到 4 的倍数。
- 容易踩的坑：34H 是"次高字节"不是"第 2 个字节"——小端里"第几个格子"和"第几高"正好是反的，先定低地址放低位字节，再数。

正确选项为什么对： 对齐：x1 占 FE00、FE01，FE02、FE03 是填充，x2 从 FE04 开始占 4 格。小端：x2 的 4 个字节从低地址到高地址依次是 00H、00H、34H、12H，所以 34H 在 FE06。选 D。

错误选项为什么错＋举例：

- A（2020FE03H）：FE03 是填充格子，不放 x2 的任何字节。教训：对齐空出来的格子是"占位不放数据"，数地址时要跳过它们，不能把填充格子当成数据格子。

- B（2020FE04H）：FE04 放的是 x2 的最低字节 00H，不是 34H；如果不对齐（x2 从 FE02 开始），34H 才会在 FE04。教训：题干说了"边界对齐"，先算填充再定 x2 的起始地址，不能直接紧跟 x1 放。
- C（2020FE05H）：FE05 放的是 x2 的次低字节 00H；如果用大端（高位在前），34H 才会在 FE05。教训：先确认是小端还是大端——小端低位在前，34H 是次高字节，排在第 3 格（FE06），不是第 2 格。

笔记本一句话： 第 14 题＋对齐先定起始地址（int 从 4 的倍数起，中间是填充），小端低位字节在低地址，34H 是次高字节排第 3 格。

【图像】

对齐+小端：x2从FE04起，34H是次高字节在FE06



x1(short)占FE00、FE01；x2(int)要从4的倍数起 → FE02、FE03填充，x2从FE04起

x2=12340000H拆字节：最高12H、次高34H、次低00H、最低00H

小端：低地址放低位字节 → FE04=00H、FE05=00H、FE06=34H、FE07=12H

34H是次高字节，排第3格（FE06）——不是"第2个字节"

大端才会在FE05（高位在前）；不对齐x2从FE02起，34H才在FE04

填充格子占位不放数据，数地址时要跳过（FE03是填充，不是答案A）

验算：m11式自查——最小元素下标必须是0，起数不错才能对

对齐：x1占FE00-FE01，FE02-FE03填充，x2从FE04起；小端34H是次高字节，在FE06

【定位】 计算机组成原理 · 第 2 章数据的表示和运算 · 大小端与边界对齐

【易混对比】

- "小端"和"大端"是两回事：小端低位字节在低地址（倒着放），大端高位字节在低地址（顺着放）；本题小端，34H 是次高字节，排第 3 格（FE06）——大端才会在 FE05。
- "对齐"和"紧挨着放"是两回事：对齐时 int 要从 4 的倍数地址起，FE02、FE03 是填充；不对齐 x2 才从 FE02 开始——题干说了边界对齐，先算填充。
- "第几个格子"和"第几高"是两回事：小端里两者正好是反的——先定"低地址放低位字节"，再数 34H 是第几个格子。

对齐先定起始地址，小端低位字节在低地址。

【坑位】

34H 是"次高字节"不是"第 2 个字节"——小端里"第几个格子"和"第几高"正好是反的。

最容易踩的坑，是数错 34H 是第几个字节。x2 的机器数 12340000H 拆成 4 个字节：最高 12H、次高 34H、次低 00H、最低 00H。小端规则是低位字节在低地址：FE04 放最低 00H，FE05 放次低 00H，FE06 放次高 34H，FE07 放最高 12H。所以 34H 在 FE06（第 3 格），不是 FE05（那是按"第 2 高=第 2 格"的顺排思维）。

【套路】

- 第一步，对齐定起点：x1（short）占 FE00、FE01；x2（int）从 4 的倍数 FE04 起，中间 FE02、FE03 是填充。
- 第二步，拆字节定高低：12340000H → 最高 12H、次高 34H、次低 00H、最低 00H。
- 第三步，小端排：低地址放低位字节，FE04=00H、FE05=00H、FE06=34H、FE07=12H。

对齐定起点 → 拆字节 → 小端低位在前。

【类比】

像停车场按车位号停车：小端是"车头朝外倒着进"——车尾（低位字节）先进最小编号的车位。4 节车厢（12H、34H、00H、00H）倒着进 FE04 到 FE07，车尾 00H 停 FE04，第二节车厢 34H 停 FE06。要是顺着进（大端），34H 就停 FE05 了。

【口诀】 对齐定起点，小端低位在前。

【考频】 「小端」在 2009–2022 年解析中出现 7 次 · 中频

第 15 题 · TLB 与 Cache（正确答案：D，不正确）

题干与选项：

下列关于 TLB 和 Cache 的叙述中，错误的是（）。

- A. 命中率都与程序局部性有关
- B. 缺失后都需要去访问主存
- C. 缺失处理都可以由硬件实现
- D. 都由 DRAM 存储器组成

入手点： 题干问"错误的是"——先找哪句话本身站不住脚；TLB 和 Cache 的"材料"是最容易想错的地方。

题干专有名词从零解释：

- 先想查字典：程序要访问一个地址，先翻一张"速查表"——虚拟地址翻译成物理地址的速查表叫"TLB"，意思是快表；主存数据的速查表叫"Cache"，意思是高速缓存。两者都是"把常用的东西放在手边，省得每次都跑远路"。
- 再想"命中率"：翻速查表时翻到的比例——翻到叫"命中"，没翻到叫"缺失"。
- 再想"局部性"：程序有个习惯——刚用过的东西一会儿还用（时间局部性），旁边挨着的东西一会儿也用（空间局部性）。好比刚翻过的书页一会儿还翻，相邻的几页也快翻到。
- 再想两种存储材料：SRAM 像白板——快、贵、容量小，不用刷新；DRAM 像笔记本——慢、便宜、容量大，要定期刷新防漏电。

考查知识点：

- 先想"TLB 和 Cache 为什么都用 SRAM"：两者存在的意义就是"快"——CPU 等不起主存的速度，所以必须用快的材料；SRAM 快但贵、容量小，正好适合做小容量的速查表。主存要的是大和便宜，用 DRAM。
- 再想"命中率为什么和局部性有关"：速查表容量小，只能放一部分；程序有局部性（刚用过的、挨着的还会用），速查表里放的恰好就是这些，所以命中率高；如果程序东一榔头西一棒子（没有局部性），速查表就总翻不到，命中率就低。
- 再想"缺失后去哪"：TLB 缺失（翻译表里没有）→ 去查页表，页表在主存里；Cache 缺失（数据表里没有）→ 去主存取数据块。两者缺失后都要访问主存。
- 再想"缺失处理谁来做"：Cache 缺失由硬件处理（硬件自动换入数据块）；TLB 缺失可以由硬件做（硬件页表遍历），也可以由软件做（缺失异常进操作系统处理）——题干说"都可以由硬件实现"，意思是硬件实现是可行的，不是说"只能由硬件"。注意"可以"和"只能"的区别。
- 边界情况：TLB 缺失和 Cache 缺失可能同时发生（翻译和数据都没命中），处理顺序是先翻译（TLB）再取数据（Cache）；缺失处理本身也可能再缺失（比如页表本身不在 Cache 里），但这不改变"缺失后访问主存"的结论。

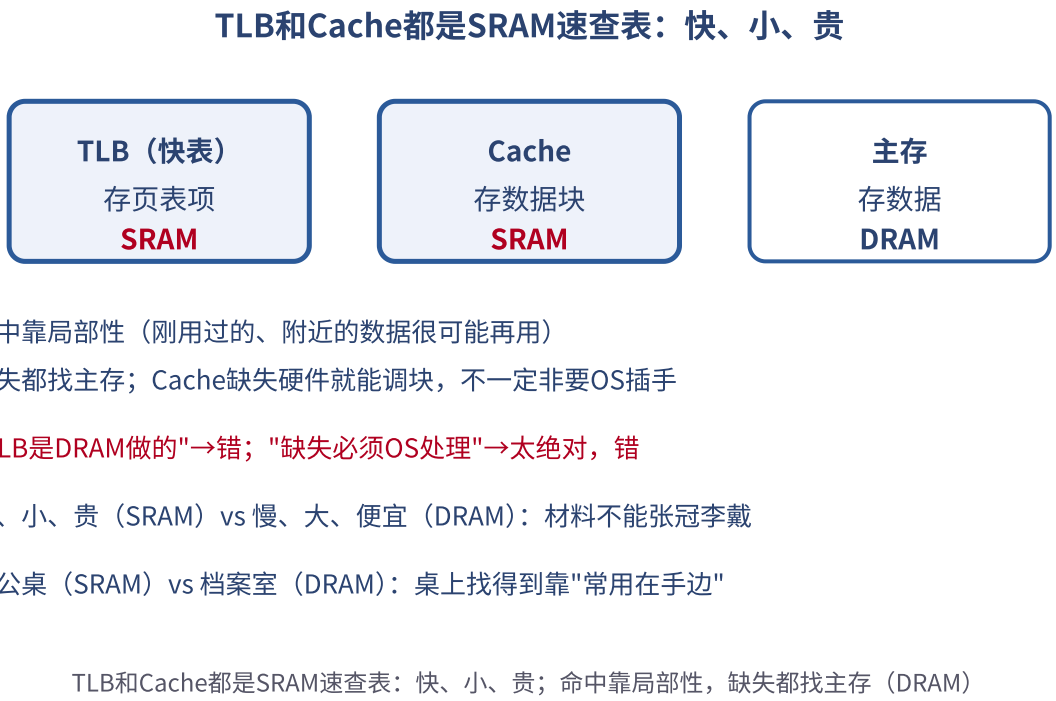
正确选项为什么对（即 D 为什么是错述）： D 说 TLB 和 Cache"都由 DRAM 组成"。错在材料：两者都要快，必须用 SRAM；DRAM 慢，是主存用的材料。如果 TLB 和 Cache 用 DRAM，速度和主存差不多，速查表就失去了意义。所以 D 是错述，是本题的答案。选 D。

其余选项为什么是对的＋举例：

- A（命中率都与程序局部性有关）：对。举例：循环里反复访问同一个数组——时间局部性让 TLB（翻译）和 Cache（数据）都反复命中；顺序访问数组——空间局部性让相邻地址的翻译和数据都被提前或顺带命中。反过来，随机跳着访问（没有局部性），两者命中率都掉下来。
- B（缺失后都需要去访问主存）：对。举例：TLB 缺失时，CPU 要去主存里的页表查翻译；Cache 缺失时，CPU 要去主存取数据块。两者缺失的下一站都是主存。
- C（缺失处理都可以由硬件实现）：对。举例：Cache 缺失一律由硬件换入数据块；TLB 缺失在很多机器上由硬件页表遍历完成（也有机器用软件异常处理，但"可以由硬件实现"这个说法本身成立）。教训："可以"不等于"只能"——只要硬件实现存在，说法就对。

笔记本一句话： 第 15 题＋TLB 和 Cache 都是 SRAM 速查表（快、小、贵），主存才是 DRAM；命中靠局部性，缺失都找主存，缺失处理硬件能做。

【图像】



TLB和Cache都是SRAM速查表：快、小、贵；命中靠局部性，缺失都找主存（DRAM）

【定位】 计算机组成原理 · 第 3 章存储系统 · TLB 与 Cache

【易混对比】

- "TLB/Cache"和"主存"是两回事：TLB 和 Cache 都是 SRAM 做的速查表（快、小、贵）；主存是 DRAM（慢、大、便宜）——"TLB 是 DRAM"之类的说法一律错。
- "命中靠什么"：靠程序的局部性（刚用过的、附近的数据很可能再用）；不是靠"表大"，是靠"规律"。
- "缺失找谁"：TLB 缺失和 Cache 缺失最后都找主存；缺失处理硬件能做（比如 Cache 缺失由硬件调块），不是"都必须操作系统插手"。

TLB 和 Cache 都是 SRAM 速查表；命中靠局部性，缺失都找主存。

【坑位】

看到"TLB/Cache 是 DRAM 做的"或"缺失必须操作系统处理"这类说法，直接判错。

最容易踩的坑，是把"快、小、贵"的 SRAM 和"慢、大、便宜"的 DRAM 张冠李戴。TLB（快表，存页表项）和 Cache（存数据块）都是用 SRAM 做的，追求速度；主存用 DRAM，追求容量。另一个坑是缺失处理：Cache 缺

失时硬件就能完成调块，不一定非要操作系统插手——"都必须 OS 处理"太绝对，是错的。

【套路】

- 第一步，定材料：TLB、Cache = SRAM（快小贵）；主存 = DRAM（慢大便宜）。
- 第二步，定原理：命中靠局部性，缺失找主存。
- 第三步，逐句验：凡说"DRAM 做""必须 OS 处理"的，一律错。

定材料 → 定原理 → 逐句验。

【类比】

像办公桌和档案室：TLB/Cache 是办公桌（小、贵、伸手就够到），主存是档案室（大、便宜、要走过去拿）。桌上找得到（命中）靠的是"你最近常用的文件就在手边"（局部性）；桌上没有（缺失）就去档案室拿。桌子本身是实木的（SRAM），档案室是铁皮柜（DRAM）——材料不能搞混。

【口诀】 TLB Cache都是表，快小贵靠局部性。

【考频】 「TLB」在 2009–2022 年解析中出现 6 次 · 中频

第 16 题 · 单地址指令直接寻址范围（正确答案：A，0~255）

题干与选项：

某计算机采用 16 位定长指令字格式，操作码位数和寻址方式位数固定，指令系统有 48 条指令，支持直接、间接、立即、相对 4 种寻址方式。单地址指令中，直接寻址方式的可寻址范围是（）。

- A. 0~255
- B. 0~1023
- C. −128~127
- D. −512~511

入手点： 16 位指令字是一张固定格数的表格：先算操作码占几格、寻址方式占几格，剩下的格子就是地址码，地址码有几格，直接寻址范围就是 0 到 2 的几次方减 1。

题干专有名词从零解释：

- 先想一张 16 格的表格：一条指令就是这 16 个 0/1，分成三段——第一段写"干什么"（操作码），第二段写"怎么找数"（寻址方式），第三段写"数在哪"（地址码）。这就叫"定长指令字格式"，意思是每条指令都是 16 位。
- 再想"操作码"：告诉 CPU 做什么操作（加、减、传送……）；48 条指令就是 48 种操作，操作码的位数要够区分这 48 种。
- 再想"寻址方式"：告诉 CPU 怎么根据地址码找到真正的操作数；4 种寻址方式就是 4 种找法。
- 再想"直接寻址"：地址码本身就是操作数的地址——指令里写"去 5 号柜子拿数"，有效地址就是 5。

考查知识点：

- 先想"操作码占几格"：48 种操作，5 格只够区分 32 种（2^5 = 32），不够；6 格能区分 64 种（2^6 = 64），够了。所以操作码至少 6 格。
- 再想"寻址方式占几格"：4 种找法，2 格能区分 4 种（2^2 = 4），所以寻址方式占 2 格。
- 再想"地址码剩几格"：16 − 6 − 2 = 8 格。直接寻址时，地址码就是有效地址，8 格能表示 2^8 = 256 个地址，从 0 到 255。
- 再想"其他三种寻址方式的区别"（寻址是弱项，多讲一步）：
- 直接寻址：地址码就是有效地址。例子：指令写"去 5 号柜子"，就去 5 号柜子。
- 间接寻址：地址码指的地方放的不是数，而是一张纸条，纸条上写的号才是真正的地址。例子：指令写"去 5 号柜子"，5 号柜子里有张纸条写着"8 号"，真正的数在 8 号柜子。
- 立即寻址：地址码本身就是操作数，不用去柜子找。例子：指令写"把 5 这个数拿来"，5 就是数本身。
- 相对寻址：地址码是一个位移量，真正的地址 = 当前指令地址（PC 的值）+ 位移量；位移量可正可负，所以相对寻址的范围是负数到正数。例子：当前指令在 100 号，位移量是 −3，真正的地址是 97 号。
- 对照"单地址指令"：指令里只有一个地址码（只有一个柜子号）；零地址指令没有地址码（比如停机），双地址指令有两个地址码（比如"把 A 号柜子的数加到 B 号柜子"）。
- 边界情况：直接寻址的范围从 0 开始，没有负数（地址不能是负的）；相对寻址的范围有负数（位移量可正可负），8 位有符号位移的范围是 −128~127。

正确选项为什么对： 操作码 6 格（48 条指令，5 格不够），寻址方式 2 格（4 种方式），地址码剩 16 − 6 − 2 = 8 格。直接寻址时地址码就是有效地址，8 格表示 0~255。选 A。

错误选项为什么错＋举例：

- B（0~1023）：这是 10 位地址码的范围（2^10 = 1024）。得这个答案是把操作码算成了 4 格（16 − 4 − 2 = 10），但 4 格只够 16 条指令，不够 48 条。教训：操作码位数先按"够区分指令条数"算（48 条要 6 格），再算地址码剩几格。
- C（−128~127）：这是 8 位有符号数的范围，是相对寻址的位移范围，不是直接寻址的地址范围。直接寻址的地址从 0 开始，没有负数。教训：看到负数范围先想"相对寻址的位移"，不是"直接寻址的地址"。
- D（−512~511）：这是 10 位有符号数的范围——既错了位数（地址码是 8 格不是 10 格），又错了寻址方式（有负数的是相对寻址，不是直接寻址）。教训：位数和寻址方式两个开关都要对，一个错答案就错。

笔记本一句话： 第 16 题＋16 位指令字先扣操作码（48 条要 6 格）和寻址方式（4 种要 2 格），剩 8 格地址码，直接寻址 0~255；负数范围是相对寻址的位移。

【图像】

16位指令字：先扣操作码和寻址方式，剩8格地址码

操作码 6格 48条：5格不够(32)，6格够(64)	寻址 2格 4种方式	地址码 8格 直接寻址 0~255

16 − 6 − 2 = 8 格 → 2^8 = 256 个地址 → 0~255

直接寻址：地址码就是有效地址，从0开始，没有负数

间接：地址码指的地方有张纸条，纸条上的号才是真地址

立即：地址码本身就是操作数；相对：地址=PC+位移（可正可负）

−128~127是相对寻址的位移范围，不是直接寻址

0~1023是10位地址码的范围（操作码算成4格就错了）

16位指令字：操作码6格（48条）+寻址方式2格（4种）+地址码8格，直接寻址0~255

【定位】 计算机组成原理 · 第 4 章指令系统 · 指令格式

【易混对比】

- "直接寻址"和"相对寻址"是两回事：直接寻址的地址码就是有效地址，从 0 开始没有负数；相对寻址的地址码是位移量（可正可负），8 位位移范围是 −128~127——看到负数范围先想相对寻址。
- "操作码位数"和"地址码位数"是两回事：操作码先按"够区分指令条数"算（48 条要 6 格，5 格只够 32 条）；地址码是 16 位扣完剩下的（16−6−2 = 8 格）。
- "单地址"和"零地址/双地址"是两回事：单地址指令只有一个地址码；零地址没有地址码（如停机），双地址有两个。

16 位先扣操作码和寻址方式，剩 8 格地址码，直接寻址 0~255。

【坑位】

操作码位数先按"够区分指令条数"算，再算地址码剩几格——顺序不能反。

最容易踩的坑，是把操作码算成 4 格（16−4−2 = 10 格地址码，得 0~1023）。4 格只够区分 16 条指令，48 条指令至少要 6 格（2^6 = 64）。另一个坑是看到 −128~127 就选——那是 8 位有符号数的范围，是相对寻址的位移范

围；直接寻址的地址从 0 开始，没有负数。

【套路】

- 第一步，算操作码：48 条指令，5 格不够（32 种），6 格够（64 种）→ 6 格。
- 第二步，算寻址方式：4 种方式 → 2 格。
- 第三步，算地址码：16 − 6 − 2 = 8 格 → 直接寻址 0~255。

操作码 → 寻址方式 → 地址码，一格一格扣。

【类比】

像一张 16 格的表格填简历：前面 6 格写"应聘岗位"（48 种岗位要 6 格才够写），接着 2 格写"联系方式类型"（4 种），剩下 8 格写"门牌号"。门牌号有 8 格，能写 0~255 号。想多写一位？前面两段已经占死了，没格了。

【口诀】 十六位先扣码，剩下八格零到二五五。

【考频】 「直接寻址」在 2009–2022 年解析中出现 9 次 · 中频

第 17 题 · 理想 CPI 为 1 处理器（正确答案：B，I、III）

题干与选项：

下列给出的处理器类型中，理想情况下 CPI 为 1 的是：

- I. 单周期 CPU；II. 多周期 CPU；III. 基本流水线 CPU；IV. 超标量流水线 CPU
- A. I，II
- B. I，III
- C. II，IV
- D. III，IV

入手点： CPI 就是"执行一条指令平均用几个时钟周期"——逐个问：这种 CPU 一条指令平均用几个周期。

题干专有名词从零解释：

- 先想"时钟周期"：CPU 的节拍，一拍做一小步；"CPI"就是执行一条指令平均用几拍，意思是 cycles per instruction。
- 再想"单周期 CPU"：一条指令在一拍里从头做到尾（取指、译码、执行、写回全在一拍里完成）。
- 再想"多周期 CPU"：一条指令分好几拍做，每拍做一步（第一拍取指，第二拍译码，依此类推）。
- 再想"基本流水线 CPU"：像洗衣房流水线——取指、译码、执行、写回是 4 个工位，不同的指令同时在不同工位上，理想情况下每拍完成一条指令。
- 再想"超标量流水线 CPU"：开了不止一条流水线，每拍能完成不止一条指令（比如两条线并行，每拍出 2 条指令）。

考查知识点：

- 先想"单周期为什么 CPI 是 1"：一条指令一拍做完，平均每条指令 1 拍，CPI = 1（I 对）。注意：单周期的一拍很长（要做完最慢的那条指令），所以 CPI 是 1 不代表速度快——速度还要看一拍有多长（时钟频率）。
- 再想"多周期为什么 CPI 大于 1"：一条指令分好几拍做，比如简单指令 4 拍、复杂指令 5 拍，平均下来每条指令不止 1 拍，CPI > 1（II 错）。
- 再想"基本流水线为什么理想 CPI 是 1"：洗衣房例子——4 件衣服同时在 4 个工位上，每拍都有一件衣服洗完出来，平均每拍完成 1 条指令，CPI = 1（III 对）。"理想情况"的意思是：没有冒险（没有等前面的结果、没有猜错分支）、没有中断、没有 Cache 缺失，流水线一直满载。
- 再想"超标量为什么 CPI 小于 1"：两条线并行，每拍出 2 条指令，平均每条指令 0.5 拍，CPI = 0.5，小于 1（IV 错）。超标量的 CPI 不是 1，是小于 1。
- 对照"CPI 和 IPC"：IPC 是每拍完成几条指令（instructions per cycle），IPC = 1 / CPI；单周期和基本流水线 IPC = 1，超标量 IPC > 1。
- 边界情况：现实里流水线 CPI 往往大于 1——数据冒险（等前一条的结果）、控制冒险（分支猜错要重来）、结构冒险（两个工位抢同一个部件）都会让流水线停顿（冒气泡），CPI 就上去了；本题问的是"理想情况下"，所以按没有冒险算。

正确选项为什么对： 单周期一条指令一拍做完，CPI = 1；基本流水线理想情况下每拍完成一条指令，CPI = 1。多周期一条指令好几拍，CPI > 1；超标量每拍多条指令，CPI < 1。所以是 I、III。选 B。

错误选项为什么错＋举例：

- A（I，II）：含了 II。多周期一条指令分好几拍做，比如 4 拍一条，CPI 是 4 左右，不是 1。教训：多周期和单周期正好相反——单周期一拍一条，多周期多拍一条。
- C（II，IV）：II 是多拍一条（CPI > 1），IV 是每拍多条（CPI < 1），两个都不是 1。教训：CPI = 1 的只有"一拍一条"那两种，多的少的都不算。
- D（III，IV）：含了 IV。超标量开了多条线，每拍出 2 条指令，CPI 是 0.5，不是 1。教训：超标量比基本流水线更进一步——基本流水线每拍 1 条，超标量每拍多条，CPI 小于 1。

笔记本一句话： 第 17 题＋CPI = 1 的是单周期（一拍一条）和基本流水线（理想每拍一条）；多周期大于 1，超标量小于 1。

【图像】



单周期：一拍从头做到尾 → CPI=1（但一拍很长，不代表快）

基本流水线：理想每拍出1条 → CPI=1（"理想"=无冒险满载）

超标量：两条线并行，每拍2条 → CPI=0.5 <1

现实流水线常因数据/控制/结构冒险停顿，CPI会上去

CPI对比：单周期一拍一条=1；多周期多拍一条>1；基本流水线理想每拍一条=1；超标量每拍多条<1

【定位】 计算机组成原理 · 第 5 章中央处理器 · CPI

【易混对比】

- "单周期"和"多周期"是两回事：单周期一条指令一拍做完（CPI = 1）；多周期一条指令分好几拍做（CPI > 1）——名字像，方向正好反了。
- "基本流水线"和"超标量"是两回事：基本流水线理想情况下每拍完成 1 条（CPI = 1）；超标量开了多条线，每拍完成多条（CPI < 1，比如 0.5）。
- "CPI = 1"和"速度快"是两回事：单周期的一拍很长（要做完最慢的那条指令），CPI 是 1 不代表速度快——速度还要看一拍有多长。

CPI = 1 的是单周期（一拍一条）和基本流水线（理想每拍一条）。

【坑位】

超标量比基本流水线更进一步：基本流水线每拍 1 条，超标量每拍多条，CPI 小于 1。

最容易踩的坑，是把超标量也当成 CPI = 1。超标量开了不止一条流水线，比如两条线并行，每拍出 2 条指令，平均每条指令 0.5 拍，CPI = 0.5，小于 1。记住这个数轴：多周期（>1）— 单周期/基本流水线（=1）— 超标量（<1），"一拍一条"才等于 1，多的少的都不算。

【套路】

- 第一步，问"一条指令平均几拍"：单周期 1 拍 → CPI = 1；多周期多拍 → >1。
- 第二步，问"一拍出几条"：基本流水线 1 条 → =1；超标量多条 → <1。
- 第三步，看"理想"二字：理想 = 没有冒险、没有中断、流水线满载；现实里流水线常因冒险停顿，CPI 会上去。

一拍一条是 1，多拍一条大于 1，一拍多条小于 1。

【类比】

像洗衣房：单周期是一台洗衣机一次只洗一件、从头洗到尾（一拍一件）；多周期是一件衣服分好几拍洗（一拍洗不完）；基本流水线是 4 个工位同时开工，每拍都出来一件干净衣服（一拍一件）；超标量是开了两排洗衣房，每拍出来两件（一拍两件）。问"平均一件衣服占几拍"，只有中间两种是 1 拍。

【口诀】 单周期流水线，理想都是一；多周期大于一，超标量小于一。

【考频】 「CPI」在 2009–2022 年解析中出现 9 次 · 中频

第 18 题 · 自陷叙述（正确答案：A，不正确）

题干与选项：

下列关于"自陷"（Trap，也称陷阱）的叙述中错误的是：

- A. 自陷是通过陷阱指令预先设定的一类外部中断事件
- B. 自陷可用于实现程序调试时的断点设置和单步跟踪
- C. 自陷发生后 CPU 将转去执行操作系统内核相应程序
- D. 自陷处理完成后返回到陷阱指令的下一条指令执行

入手点： 题干问"错误的是"——先分清自陷是"内部"还是"外部"：自陷是 CPU 执行陷阱指令自己引起的，是内部事件。

题干专有名词从零解释：

- 先想两种"打断"：一种是外面设备喊停（比如键盘来了数据、定时器到点了），叫"外部中断"，意思是中断源在 CPU 外面；一种是 CPU 自己执行时出事（比如除零、执行了一条特殊指令），叫"内部中断"，意思是中断源在 CPU 里面，也叫"异常"。
- 再想"自陷"：程序里预先放了一条特殊指令（陷阱指令），CPU 执行到它就主动停下来、转去执行一段处理程序——好比程序自己举手说"我要进内核办点事"。自陷是内部中断的一种。
- 再想"断点"和"单步跟踪"：调试时在某一行设个断点，程序跑到那行就停下来让你看；单步跟踪是一行一行走，每走一行停一次。

考查知识点：

- 先想"中断的分类"：外部中断分可屏蔽（可以关掉不响应）和不可屏蔽（关不掉）；内部中断分三类——故障（执行指令时出错，比如缺页、除零，处理完返回到出错的指令本身重执行）、自陷（陷阱指令主动引起，处理完返回到陷阱指令的下一条）、终止（严重错误无法恢复，比如硬件故障，不返回）。
- 再想"自陷为什么是内部中断"：自陷的起因是 CPU 执行陷阱指令这个动作本身，不需要任何外部设备发信号；"通过陷阱指令预先设定"这半句是对的，错的是"外部中断事件"这五个字——自陷是内部中断，不是外部中断。
- 再想"自陷的用途"：系统调用就是用陷阱指令实现的（用户程序想用内核服务，就执行陷阱指令进内核）；调试的断点（int 3）和单步跟踪也是自陷。
- 再想"自陷处理完回哪"：自陷是"自愿"的，陷阱指令本身就是要进内核办件事，办完继续往下走，所以返回到陷阱指令的下一条；对照故障（比如缺页）是"非自愿"的，处理完要回到故障指令本身重执行一次（因为那条指令还没执行完）。
- 边界情况：陷阱指令和"访管指令"是同一个东西的不同叫法；外部中断里的"陷阱"说法不存在——看到"陷阱"先想内部中断。

正确选项为什么对（即 A 为什么是错述）： A 说自陷是"外部中断事件"。错在内外之分：自陷由 CPU 执行陷阱指令主动引起，中断源在 CPU 内部，是内部中断（异常），不是外部中断。"通过陷阱指令预先设定"是对的，但"外部"两个字错了，整句就是错述。所以 A 是答案。选 A。

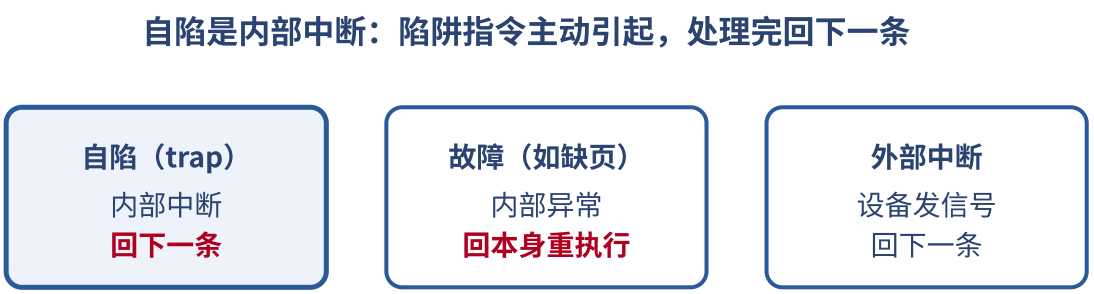
其余选项为什么是对的＋举例：

- B（自陷可用于断点设置和单步跟踪）：对。举例：调试器在某一行设断点，实际是把那行的指令换成陷阱指令，程序跑到就自陷停下来；单步跟踪是每执行一条指令就自陷一次，停下来让你看寄存器和内存。

- C（自陷发生后 CPU 转去执行操作系统内核相应程序）：对。举例：用户程序要读文件，执行陷阱指令进内核，内核的系统调用处理程序接手；断点自陷后，内核的调试处理程序接手。转去的都是内核程序，不是用户程序。
- D（自陷处理完成后返回到陷阱指令的下一条指令）：对。举例：系统调用的陷阱指令在第 100 行，处理完回到第 101 行继续执行；因为陷阱指令本身已经"执行完了"（它的任务就是进内核），不需要重执行。对照缺页故障：故障指令没执行完，处理完要回到故障指令本身重执行。

笔记本一句话： 第 18 题＋自陷是内部中断（陷阱指令主动引起），不是外部中断；处理完回下一条（故障才回本身重执行）。

【图像】



自陷：陷阱指令主动引起 → 起因在CPU内部 → 内部中断
使命完成（服务做完）→ 回下一条；故障是"想干没干成"→ 回本身重干

"自陷是外部中断"→错；判内外看"谁先动的手"

陷阱指令：专门用来主动引起自陷、请求OS服务的指令
普通指令不会主动引起中断

自陷是内部中断：陷阱指令主动引起，处理完回下一条；故障才回本身重执行

【定位】 计算机组成原理 · 第 7 章输入输出系统 · 中断与异常

【易混对比】

- "自陷"和"外部中断"是两回事：自陷是 CPU 内部、由陷阱指令主动引起的（内部中断）；外部中断是外面设备发的信号——"自陷是外部中断"的说法直接错。
- "自陷"和"故障"是两回事：自陷处理完回下一条指令（使命完成）；故障（比如缺页）是"想干没干成"，处理完要回本身重执行。
- "陷阱指令"和"普通指令"是两回事：陷阱指令是专门用来主动引起自陷、请求操作系统服务的；普通指令不会主动引起中断。

自陷是内部中断，陷阱指令主动引起；处理完回下一条。

【坑位】

别把任何带"中断"字眼的都当成外部中断——看起因在内还是在外。

最容易踩的坑，是看到"中断"两个字就以为是外部的。自陷的英文是 trap，是程序主动执行陷阱指令引起的——起因在 CPU 内部，所以是内部中断（异常）。外部中断的起因在外面（键盘、网卡、定时器发信号）。判内外就看一句话：是谁先动的手——自己（陷阱指令）动手的是内部，外面设备动手的是外部。

【类比】

像公司里两种"找领导"：自陷是员工主动敲门汇报（内部、主动）——汇报完回去干下一件事；外部中断是客户打电话进来（外部、被动）——接完电话继续干手头的事。而故障像做到一半发现缺材料——补上材料后，这道工序得重做一遍（回本身）。

【口诀】 自陷是内中断，陷阱主动引；处理完走下一条。

【考频】 「自陷」在 2009–2022 年解析中出现 10 次 · 高频

第 19 题 · QPI 总线带宽（正确答案：C，19.2 GB/s）

题干与选项：

QPI 总线是一种点对点全双工同步串行总线，总线上的设备可同时接收和发送信息，每个方向可同时传输 20 位信息（16 位数据＋4 位校验位），每个 QPI 数据包有 80 位信息，分 2 个时钟周期传送，每个时钟周期传递 2 次，因此 QPI 总线带宽为每秒传送次数×2B×2。若 QPI 时钟频率为 2.4GHz，则总线带宽为：

- A. 4.8
- B. 9.6
- C. 19.2
- D. 38.4（单位 GB/s）

入手点： 题干已经给了公式"每秒传送次数×2B×2"——只要算出"每秒传送次数"，代入就行；别被 20 位、80 位这些数带跑。

题干专有名词从零解释：

- 先想送货车队：总线就是设备之间运数据的路；"带宽"就是这条路每秒能运多少数据（单位 GB/s，G 是 10 的 9 次方，B 是字节）。
- 再想"点对点"：两个设备之间一条专线直连，不和其他设备共用；"全双工"：同一时刻能同时收和发（像双向车道），不是收发轮流。
- 再想"串行"：数据一位一位排队走（像单车道），不是多位并排走。
- 再想"时钟频率 2.4GHz"：每秒 2.4G 个时钟周期；"每个时钟周期传递 2 次"：一个周期里传两次（上下沿各一次，像一拍走两步）。

考查知识点：

- 先想"每秒传送次数"：2.4GHz 是每秒 2.4G 个周期，每周期传 2 次，所以每秒传送 2.4G × 2 = 4.8G 次。
- 再想"每次传多少数据"：每个方向每次传 20 位，其中 16 位是数据、4 位是校验位；算带宽只算数据部分：16 位 = 2 字节（2B）。校验位是"防错的附赠"，不算进数据带宽。
- 再想"为什么乘 2"：全双工是双向车道，收和发同时进行，所以一个方向的带宽要 ×2 才是总带宽。
- 合起来：4.8G 次/秒 × 2B × 2（两个方向）= 19.2 GB/s。
- 对照"半双工"：半双工同一时刻只能一个方向（像单车道轮流走），总带宽就是一个方向的，不乘 2；本题是全双工，必须乘 2。
- 边界情况：单位别搞混——GB/s 是每秒多少字节，Gb/s 是每秒多少位，两者差 8 倍；本题单位是 GB/s，所以 16 位要先换成 2B 再算。
- 容易踩的坑：20 位不能直接当 2.5B 算带宽（校验位不算数据）；80 位数据包、分 2 个周期这些数是干扰信息（公式里已经把它们折算进去了，不用再算一遍）。

正确选项为什么对： 每秒传送次数 = 2.4G × 2 = 4.8G 次；每次数据 2B；全双工两个方向 ×2。带宽 = 4.8 × 2 × 2 = 19.2 GB/s。选 C。

错误选项为什么错＋举例：

- A（4.8）：只算了"每秒传送次数"的数值 4.8G，没乘每次的字节数和方向数。教训：公式是"次数×字节×方向"，三个数都要乘，少一个结果就小。

- B（9.6）：只算了一个方向（4.8 × 2 = 9.6），忘了全双工要 ×2。教训：看到"全双工"先把"双向"这个 2 记下来，最后别漏乘。
- D（38.4）：在正确结果上又多乘了一个 2（比如把"每周期传 2 次"算了两遍）。教训：公式里的每个 2 只能乘一次——"每周期 2 次"已经算进 4.8G 里了，"全双工 2 个方向"是另一个 2，两者各乘一次，不能重复。

笔记本一句话： 第 19 题＋QPI 带宽 = 时钟×每周期次数×数据字节×方向数；校验位不算数据，全双工方向要 ×2。

【图像】

QPI带宽 = 每秒传送次数 × 数据字节 × 方向数

4.8G × 2B × 2 = 19.2 GB/s

① 每秒传送次数 = 2.4GHz × 2（每周期传2次）= 4.8G 次

② 每次数据 = 16位 = 2B（4位校验位是附赠，不算）

③ 全双工双向 → ×2（半双工不乘）

公式里每个2只能乘一次："每周期2次"已进4.8G，"双向"是另一个2

4.8（只算次数，少乘两项）→ 9.6（只算单向，漏×2）→ 38.4（重复×2）

80位数据包、分2个周期：干扰信息，公式已折算，不用再算

单位：GB/s是字节，Gb/s是位，差8倍 → 16位先换2B再算

点对点=专线直连；串行=一位一位走；同步=靠时钟

QPI带宽=每秒传送次数×数据字节×方向数：4.8G×2B×2=19.2GB/s；校验位不算，全双工×2

【定位】 计算机组成原理 · 第 6 章总线 · 总线带宽

【易混对比】

- "20 位"和"2B"是两回事：每个方向每次传 20 位，其中 16 位数据、4 位校验位；算带宽只算数据部分（16 位 = 2B），校验位是防错的附赠，不算。
- "全双工"和"半双工"是两回事：全双工双向同时传，带宽要 ×2；半双工同一时刻只能一个方向，不乘 2。
- "GB/s"和"Gb/s"是两回事：B 是字节，b 是位，差 8 倍；本题单位是 GB/s，16 位要先换成 2B 再算。

带宽 = 时钟 × 每周期次数 × 数据字节 × 方向数；校验位不算。

【坑位】

公式里的每个 2 只能乘一次："每周期 2 次"和"全双工 2 个方向"是两个不同的 2。

最容易踩的坑，是重复乘 2 得 38.4。拆开看：每秒传送次数 = 2.4G × 2（每周期传 2 次）= 4.8G 次——这个 2 已经算进去了；每次数据 2B；全双工两个方向 ×2——这是另一个 2。带宽 = 4.8 × 2 × 2 = 19.2 GB/s。80 位数据包、分 2 个周期这些数是干扰信息，公式里已经折算过了，不用再算一遍。

【套路】

- 第一步，算每秒传送次数：时钟频率 × 每周期次数（2.4G × 2 = 4.8G）。
- 第二步，算每次数据字节：只算数据位（16 位 = 2B），校验位扔掉。

- 第三步，看双工：全双工 ×2，半双工不乘。

次数 → 字节（去校验）→ 双工，一步一乘。

【类比】

像快递车队算运力：每秒发车 4.8G 趟（次数），每趟拉 2 吨货（数据字节，押车的保安不算货），双向车道同时跑（×2）。运力 = 趟数 × 每趟货 × 方向数。保安（校验位）不拉货，不能算进运力；单车道（半双工）就不用乘方向数。

【口诀】 时钟乘次数乘字节，全双工再乘二。

【考频】 「QPI」在 2009–2022 年解析中出现 2 次 · 低频

第 20 题 · 外部中断事件（正确答案：C，II、III）

题干与选项：

下列事件中属于外部中断事件的是：

- I. 访存时缺页；II. 定时器延时；III. 网络数据包到达
- A. I、II
- B. I、III
- C. II、III
- D. I、II、III

入手点： 判断标准就一个：中断源在 CPU 外面还是里面——外面设备的叫外部中断，CPU 自己执行时出事的叫内部中断。

题干专有名词从零解释：

- 先想两种"打断"：外面设备喊停（键盘来了数据、定时器到点了、网卡收到了包），叫"外部中断"，意思是中断源在 CPU 外面；CPU 自己执行时出事（除零、访存时页不在），叫"内部中断"，意思是中断源在 CPU 里面，也叫"异常"。
- 再想"缺页"：程序要访问的页不在内存里（在磁盘上），CPU 访问时发现没这个页，只好停下来先把页换进来——这叫"缺页"，意思是缺了一页。
- 再想"定时器"：一个到点就喊停的时钟设备；"网络数据包到达"：网卡收到了别人发来的数据包，喊 CPU 来取。

考查知识点：

- 先想"缺页为什么是内部中断"：缺页发生在 CPU 执行访存指令的过程中，是执行这条指令时内部发现的问题，不需要任何外部设备发信号——所以是内部中断（具体是"故障"类，处理完返回到故障指令本身重执行）。
- 再想"定时器为什么是外部中断"：定时器是 CPU 外面的设备（时钟芯片），到点发信号打断 CPU——中断源在外面，所以是外部中断。
- 再想"网络数据包为什么是外部中断"：网卡是 CPU 外面的设备，收到数据包发信号打断 CPU——中断源在外面，所以是外部中断。
- 对照"内部中断的常见例子"：除零、溢出、非法指令、缺页、陷阱指令（自陷）、断点——都是 CPU 执行时内部出的事。
- 对照"外部中断的常见例子"：键盘输入、鼠标点击、定时器到点、网卡收包、磁盘完成读写——都是外面设备发信号。
- 边界情况：缺页虽然要去磁盘取页（磁盘是外部设备），但中断的起因是 CPU 访存时内部发现页不在，所以算内部中断——分类看"起因"，不是看"处理时找谁"。

正确选项为什么对： I 缺页是 CPU 访存时内部出的事，是内部中断；II 定时器是外部设备到点发信号，是外部中断；III 网卡是外部设备收包发信号，是外部中断。所以外部中断事件是 II、III。选 C。

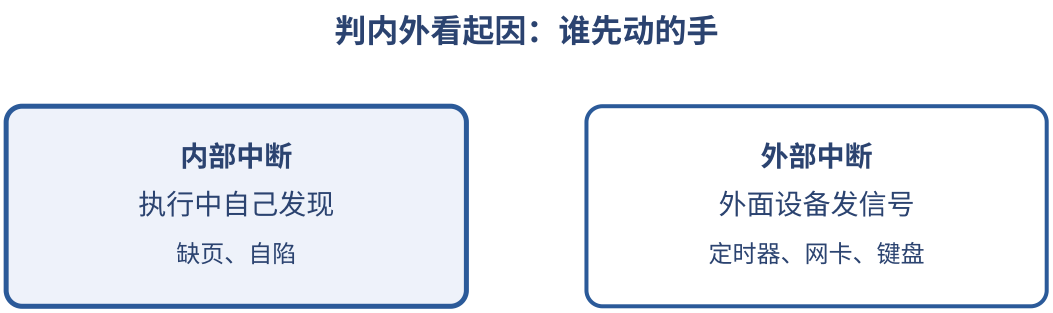
错误选项为什么错＋举例：

- A（I、II）：含了 I（缺页是内部中断，不是外部），还漏了 III（网卡收包是外部中断）。教训：缺页一出现先想"访存指令执行时内部发现"，别因为"要去磁盘取页"就当成外部。

- B（I、III）：含了 I（缺页是内部），还漏了 II（定时器到点是外部）。教训：定时器是典型外部设备，看到"定时器"先想到外部中断。
- D（I、II、III）：全选，含了错的 I。教训：只要有一项错，全选就错；逐项按"内外"验，不要图省事全选。

笔记本一句话： 第 20 题＋内外看起因：缺页是访存时内部出的事（内部），定时器、网卡是外面设备发信号（外部）。

【图像】



内外中断看起因：缺页是访存时内部出的事（内部）；定时器、网卡是外面设备发信号（外部）

【定位】 计算机组成原理 · 第 7 章输入输出系统 · 中断源分类

【易混对比】

- "内部中断"和"外部中断"看起因：起因在 CPU 内部执行过程中（访存缺页、陷阱指令）是内部；起因在外面设备发信号（定时器到点、网卡收到包）是外部。
- "缺页"和"缺纸"不是一回事：缺页是访存时地址转换失败（内部）；打印机缺纸是外设在指令执行完后报告（外部）——名字都带"缺"，内外相反。
- "自陷"和"外部中断"不是一回事：自陷是陷阱指令主动引起（内部）；不要看到"中断"两个字就当成外部。

判内外就看一句话：是谁先动的手。

【坑位】

缺页是内部中断——它是访存过程中地址转换失败，不是外面设备发信号。

最容易踩的坑，是把"缺页"当成外部中断。缺页发生在指令执行过程中：CPU 访存，查页表发现页不在内存——这事是 CPU 自己在干活时发现的，起因在内部，所以是内部中断（异常）。而定时器、网卡是外面设备主动发信号"报告"CPU，起因在外部，才是外部中断。记住：访存时出的事归内部，设备发信号归外部。

【套路】

- 第一步，问起因：执行过程中自己发现的 → 内部；外面设备发信号 → 外部。
- 第二步，对号：缺页、自陷 → 内部；定时器、网卡、键盘 → 外部。

- 第三步，验名字陷阱："缺页"和"缺纸"、"自陷"和"中断"，名字像但内外可能反。

执行中发现归内部，设备发信号归外部。

【类比】

像两种"被打断"：你在厨房做菜，做到一半发现盐没了（缺页）——是你自己在干活时发现的（内部）；手机响了（外部中断）——是外面有人找你。缺盐这事怪不了外面，手机响也怪不了你。判内外，就看是谁先动的手。

【口诀】 内外看起因，访存内部，设备外部。

【考频】 「外部中断」在 2009–2022 年解析中出现 11 次 · 高频

第 21 题 · 外部中断叙述（正确答案：B，不正确）

题干与选项：

外部中断包括不可屏蔽中断（NMI）和可屏蔽中断，下列关于外部中断的叙述中错误的是：

- A. CPU 处于关中断状态时也能响应 NMI 请求
- B. 一旦可屏蔽中断请求信号有效，CPU 将立即响应
- C. 不可屏蔽中断的优先级比可屏蔽中断的优先级高
- D. 可通过中断屏蔽字改变可屏蔽中断的处理优先级

入手点： 题干问"错误的是"——盯住 B 里的"立即"两个字：可屏蔽中断不是信号一来就响应，要等几个条件。

题干专有名词从零解释：

- 先想两种外面来的打断：一种是"关不掉的"（比如电源要断了、内存出错了），叫"不可屏蔽中断"，意思是 NMI，non-maskable interrupt，CPU 关中断也得响应；一种是"能关掉的"（比如键盘、网卡），叫"可屏蔽中断"，意思是 CPU 关中断时可以先不响应。
- 再想"关中断"和"开中断"：CPU 有个开关（中断允许位）——开着叫"开中断状态"，可以响应可屏蔽中断；关着叫"关中断状态"，可屏蔽中断先挂着不响应。
- 再想"中断屏蔽字"：一张表，每个中断源占一位，写 1 就是"屏蔽这个源"（不让它打断），写 0 就是"放行"；改这张表就能改哪些中断能打断、哪些不能。
- 再想"指令边界"：一条指令执行完、下一条指令开始前的那个时刻——CPU 只在这个时刻响应中断，不会在一条指令执行到一半时停下来。

考查知识点：

- 先想"可屏蔽中断响应的三个条件"：第一，CPU 处在开中断状态（开关开着）；第二，当前指令执行完了（到了指令边界）；第三，没有更高级的中断请求、也没有 DMA 在抢总线。三个条件都满足，CPU 才响应。所以"信号一有效就立即响应"是错的——正在执行的指令要先执行完，关中断时更要先挂着。
- 再想"NMI 为什么关中断也能响应"：NMI 的意思是"不可屏蔽"，屏蔽开关管不住它；电源掉电、内存严重错误这种十万火急的事，必须立刻处理，不能等。关中断状态下，可屏蔽中断挂着，NMI 照样响应。
- 再想"优先级"：NMI 比任何可屏蔽中断都高级——同时来好几个请求时，CPU 先处理 NMI。
- 再想"中断屏蔽字怎么改优先级"：屏蔽字可以把某个可屏蔽中断源屏蔽掉（不让它打断），也可以放行；处理中断时改屏蔽字，就能决定"当前处理能不能被别的中断打断"（比如处理低级中断时屏蔽同级和低级、放行高级，就实现了"高级能打断低级"）。
- 边界情况：DMA 请求（直接访存）比中断优先级更高——DMA 要抢总线时，中断响应也要往后让；还有"中断响应过程"本身：CPU 响应后先关中断（防止嵌套乱套）、保存断点和程序状态字、转服务程序，服务程序里可以再开中断实现嵌套。

正确选项为什么对（即 B 为什么是错述）： B 说"一旦可屏蔽中断请求信号有效，CPU 将立即响应"。错在"立即"：可屏蔽中断要等当前指令执行完（指令边界），还要处在开中断状态，还要没有更高级请求或 DMA 抢总线。信号有效只是"挂号"，不是"立刻接诊"。所以 B 是错述，是本题的答案。选 B。

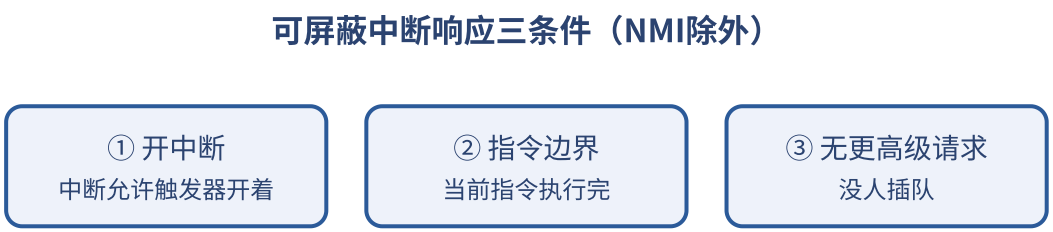
其余选项为什么是对的＋举例：

- A（CPU 处于关中断状态时也能响应 NMI 请求）：对。举例：电源掉电时 NMI 发信号，CPU 即使在关中断状态（比如正在处理另一个中断）也必须立刻响应掉电处理——NMI 的意思是"关不掉"，屏蔽开关对它无效。

- C（不可屏蔽中断的优先级比可屏蔽中断的优先级高）：对。举例：NMI 和键盘中断同时到，CPU 先处理 NMI（掉电、内存错这种十万火急的事），再处理键盘——NMI 永远排在可屏蔽中断前面。
- D（可通过中断屏蔽字改变可屏蔽中断的处理优先级）：对。举例：正在处理打印机中断时，把屏蔽字改成"屏蔽打印机、放行网卡"，网卡中断就能打断打印机处理；反过来把网卡屏蔽了，网卡就打断不了——改屏蔽字就是改"谁能打断谁"，也就是改处理优先级。

笔记本一句话： 第 21 题＋可屏蔽中断不是"立即"响应，要等开中断＋指令边界＋无更高级请求；NMI 关不掉、优先级最高；屏蔽字能改"谁能打断谁"。

【图像】



三者全满足才响应 → "立即响应"是错的
NMI（不可屏蔽）：屏蔽字关不掉，优先级最高，来了就处理

可屏蔽 vs NMI：响应规则是反的，不能混

屏蔽字：软件可改的"谁能打断谁"开关；改屏蔽字能改变实际顺序

优先级：硬件定的先后顺序
普通电话等三条件；老板专线（NMI）静音也拦不住

可屏蔽中断响应三条件：开中断+指令边界+无更高级请求；NMI关不掉优先级最高

【定位】 计算机组成原理 · 第 7 章输入输出系统 · 中断响应

【易混对比】

- "可屏蔽中断"和"不可屏蔽中断（NMI）"是两回事：可屏蔽能被屏蔽字关掉，要等条件；NMI 关不掉、优先级最高、有请求就处理。
- "立即响应"和"等条件响应"是两回事：可屏蔽中断不是一来就响应，要等开中断、等到指令边界、且没有更高级的请求——"立即"二字是错的。
- "屏蔽字"和"优先级"是两回事：屏蔽字是软件可改的"谁能打断谁"的开关；优先级是硬件定的先后顺序——改屏蔽字能改变实际响应顺序。

可屏蔽中断不是"立即"响应，要等开中断、等指令边界、等无更高级请求。

【坑位】

看到"立即响应"四个字先打问号：可屏蔽中断的响应有三个前置条件。

最容易踩的坑，是把可屏蔽中断当成"一来就处理"。实际上 CPU 要等三个条件都满足：中断允许触发器开着（开中断）、当前指令执行到边界（一条指令执行完）、没有更高级的中断请求插队。缺一个都不响应。而 NMI（不可屏蔽中断）正好相反：屏蔽字关不掉它，优先级最高，来了就处理——两者的响应规则是反的，不能混。

【套路】

- 第一步，分可屏蔽还是 NMI：NMI 关不掉、最高优；可屏蔽看条件。
- 第二步，数三条件：开中断、指令边界、无更高级请求——三者全满足才响应。
- 第三步，见"立即"判错：可屏蔽中断没有"立即"一说。

分内外 → 数三条件 → 见"立即"判错。

【类比】

像开会时两种打断：普通电话（可屏蔽中断）要等三个条件——手机没静音（开中断）、这句话说完（指令边界）、没有更重要的电话插队；老板的专线（NMI）静音也拦不住，一响就接。"立即接电话"只对专线成立，普通电话不行。

【口诀】 可屏蔽非立即，等开中断等边界。

【考频】 「可屏蔽中断」在 2009–2022 年解析中出现 7 次 · 中频

第 22 题 · 周期挪用 DMA（正确答案：C，不正确）

题干与选项：

若设备采用周期挪用 DMA 方式进行输入输出，每次 DMA 传送的数据块大小为 512 字节，相应的 I/O 接口中有一个 32 位数据缓冲寄存器，对于数据输入过程，下列叙述中错误的是：

- A. 每准备好 32 位数据，DMA 控制器就发出一次总线请求
- B. 相对于 CPU，DMA 控制器的总线使用权的优先级更高
- C. 在整个数据块的传送过程中，CPU 不可以访问主存储器
- D. 数据块传送结束时，会产生"DMA 传送结束"的中断请求

入手点： 题干问"错误的是"——抓住"周期挪用"四个字：每次只挪用一个周期，挪完就还，CPU 在空闲周期里照常用主存。

题干专有名词从零解释：

- 先想不用 CPU 动手的搬运工：外设和内存之间直接搬数据，不用 CPU 一个字节一个字节搬——这就叫"DMA"，意思是直接存储器存取；管搬运的部件叫"DMA 控制器"。
- 再想"总线"：CPU、内存、外设共用的一条数据通道，同一时刻只能一方用；"总线请求"就是"我要用总线"的申请。
- 再想"周期挪用"：DMA 每次只借一个存取周期（一个总线节拍）搬 32 位，搬完马上把总线还给 CPU；CPU 在 DMA 不用的节拍里照常访问主存——好比搬运工每次只借一分钟，借完就还。
- 再想"数据缓冲寄存器"：I/O 接口里临时放数据的格子，本题是 32 位（4 字节），攒满 32 位就通知 DMA 来搬。

考查知识点：

- 先想"周期挪用怎么工作"：外设每准备好 32 位数据（缓冲寄存器满了），DMA 控制器就发一次总线请求；拿到总线后用一个周期把 32 位搬进主存，然后还总线；512 字节的数据块要搬 128 次（512 ÷ 4 = 128），每次各挪一个周期。
- 再想"CPU 能不能访存"：周期挪用只挪一个周期，CPU 在 DMA 不用的周期里完全可以访问主存——CPU 只是偶尔被"挪用"一个周期，不是全程靠边站。整个数据块传完前，CPU 照常用主存（只是偶尔让一个周期）。
- 对照"CPU 停止法"：那是另一种 DMA 方式——整个数据块传送期间，CPU 完全不能访问主存（总线全被 DMA 占了）。选项 C 说的"整个过程都不可以访问"，说的正是 CPU 停止法，不是周期挪用。
- 对照"交替访存"：第三种方式——DMA 和 CPU 轮流用总线（比如各用一半时间），谁也不用等谁申请，适合 CPU 和 DMA 速度匹配的情况。
- 再想"优先级"：DMA 拿到总线请求时，CPU 要让路——DMA 的总线优先级高于 CPU（因为外设数据不等人，晚了就丢了）。
- 再想"传完怎么办"：512 字节搬完，DMA 控制器发一个"DMA 传送结束"的中断，通知 CPU"搬完了，可以来收尾了"（比如处理数据、启动下一次传送）。
- 边界情况：周期挪用适合外设速度不太快的情况（每次只挪一个周期，CPU 损失小）；外设速度很快时，周期挪用会频繁打断 CPU，这时更适合 CPU 停止法或交替访存。

正确选项为什么对（即 C 为什么是错述）： C 说"在整个数据块的传送过程中，CPU 不可以访问主存储器"。错在"整个过程都不可以"：周期挪用每次只挪一个周期，CPU 在 DMA 不用的周期里可以访问主存。"整个过程都不可以"说的是 CPU 停止法，不是周期挪用。所以 C 是错述，是本题的答案。选 C。

其余选项为什么是对的＋举例：

- A（每准备好 32 位数据，DMA 控制器就发出一次总线请求）：对。举例：缓冲寄存器是 32 位，攒满 32 位（4 字节）就发一次请求，搬完再攒下一批；512 字节要发 128 次请求，每次搬 4 字节。
- B（相对于 CPU，DMA 控制器的总线使用权的优先级更高）：对。举例：CPU 正在用总线，DMA 一发请求，CPU 就要让出一个周期给 DMA——因为外设的数据不等人，晚了就丢了，所以 DMA 优先。
- D（数据块传送结束时，会产生"DMA 传送结束"的中断请求）：对。举例：512 字节搬完，DMA 控制器发中断告诉 CPU"搬完了"；CPU 收到中断后做收尾（比如通知进程数据到了）。没有这个中断，CPU 就不知道什么时候搬完。

笔记本一句话： 第 22 题＋周期挪用每次只借一个周期，CPU 空闲周期照常访存；"全程不能访存"说的是 CPU 停止法，别张冠李戴。

【图像】



"DMA时CPU全程不能访存" → 这是停止法的特征，不是周期挪用

周期挪用=见缝插针：CPU不用的周期才借，偶尔被"偷"一拍

"全程不能访存"安到周期挪用头上 → 张冠李戴，错

DMA快在"不麻烦CPU"：硬件直接搬数据，中断传送每个数据都要CPU插手

借会议室：开大会（停止法）vs 上厕所功夫借5分钟（挪用）

三种DMA：CPU停止法全程不访存；周期挪用只借一周期；交替分时固定分时

【定位】 计算机组成原理 · 第 7 章输入输出系统 · DMA 方式

【易混对比】

- "周期挪用"和"CPU 停止法"是两回事：周期挪用每次只借一个周期，CPU 空闲的周期照常访存；"全程不能访存"说的是 CPU 停止法——张冠李戴是本题的陷阱。
- "借一个周期"和"借一整段时间"是两回事：周期挪用是见缝插针（CPU 不用的周期才借）；CPU 停止法是整个 DMA 传完之前 CPU 都不访存。
- "DMA"和"中断传送"是两回事：DMA 由硬件直接搬数据、不经过 CPU；中断传送每个数据都要 CPU 插手——DMA 快就快在"不麻烦 CPU"。

周期挪用每次只借一个周期，CPU 空闲周期照常访存。

【坑位】

"全程不能访存"说的是 CPU 停止法，不是周期挪用——别张冠李戴。

最容易踩的坑，是把三种 DMA 方式的特征记混。周期挪用（cycle stealing）的精髓是"偷"：I/O 设备只在需要时借用一个总线周期，CPU 在自己的空闲周期（比如不访存的周期）照常工作，只是偶尔被"偷"走一拍，感觉像停了一拍。而"DMA 传送过程中 CPU 不能访存"是 CPU 停止法的特征——整个传送期间 CPU 让出总线。题干把停止法的特征安到周期挪用头上，所以是错的。

【套路】

- 第一步，定三种方式：CPU 停止法（全程让）、周期挪用（借一拍）、交替分时（固定分）。
- 第二步，看题干说的是谁的特征："全程不能访存"→ 停止法。
- 第三步，对号：题干问周期挪用，特征对不上 → 错。

定三种 → 看特征是谁的 → 对号。

【类比】

像借会议室：CPU 停止法是"I/O 开大会，CPU 一整天都不许进"；周期挪用是"I/O 只在 CPU 上厕所的功夫进去用 5 分钟，CPU 回来照常用"。说"周期挪用时 CPU 全程不能进会议室"，是把"开大会"的规矩安到了"借 5 分钟"头上。

【口诀】 挪用只借一周期，CPU空闲照常访。

【考频】 「周期挪用」在 2009–2022 年解析中出现 16 次 · 高频

第 23 题 · 多进程共享文件（正确答案：B，系统打开文件表中仅有一个表项包含 F 的属性）

题干与选项：

若多个进程共享同一个文件 F，则下列叙述中，正确的是（）。

- A. 各进程只能用"读"方式打开文件 F
- B. 在系统打开文件表中仅有一个表项包含 F 的属性
- C. 各进程的用户打开文件表中关于 F 的表项内容相同
- D. 进程关闭 F 时，系统删除 F 在系统打开文件表中的表项

入手点： 分清两本账：系统那本账按文件记（一个文件一行），每个进程自己那本账按进程记（各记各的）。

题干专有名词从零解释：

- 先想图书馆总登记本：全馆一本，记录每本书的属性（书名、位置、被借了几本）——这就叫"系统打开文件表"，意思是全系统共用的一张表，一个文件只占一行，还记着"有几个进程在用它"（引用计数）。
- 再想每个读者的借书卡：每人一张，记录自己借了哪本书、读到第几页、是借阅还是复印——这就叫"用户打开文件表"，意思是每个进程自己的一张表。
- 再想"打开方式"：读、写、追加等；"读写指针"：读到文件第几个字节了（每个进程自己的进度）。

考查知识点：

- 先想"系统表为什么只有一个表项"：系统表是按文件记的——文件 F 的属性（大小、位置、权限）只有一份，不管几个进程打开它，系统表里都只有一行记 F 的属性；"有几个进程在用"记成一个数字（引用计数），3 个进程打开就记 3。
- 再想"用户表为什么各记各的"：每个进程打开 F 时，自己表里加一行，记自己的打开方式和读写指针——A 进程读到第 100 字节，B 进程读到第 50 字节，互不干扰；A 以读方式打开，B 以写方式打开（权限允许的话），也可以不一样。
- 再想"打开的流程"：进程打开 F 时，先查系统表——F 已经在系统表里了（别的进程打开过），引用计数加 1；F 不在系统表里，才新加一行；然后在自己用户表里加一行，得到一个文件描述符（用户表那一行的编号）。
- 再想"关闭的流程"：进程关闭 F 时，先删自己用户表里那一行，再把系统表里 F 的引用计数减 1；计数减到 0（没人用了）才删系统表那一行；还有别的进程在用，就不能删。
- 边界情况：同一个进程两次打开同一文件，用户表里有两行（两个描述符，两个读写指针）；不同进程打开同一文件，用户表各有一行，互不影响。

正确选项为什么对： 系统打开文件表是按文件记的，一个文件只占一行（属性一份，引用计数记人数）。多个进程共享文件 F，系统表里仍然只有一行记 F。选 B。

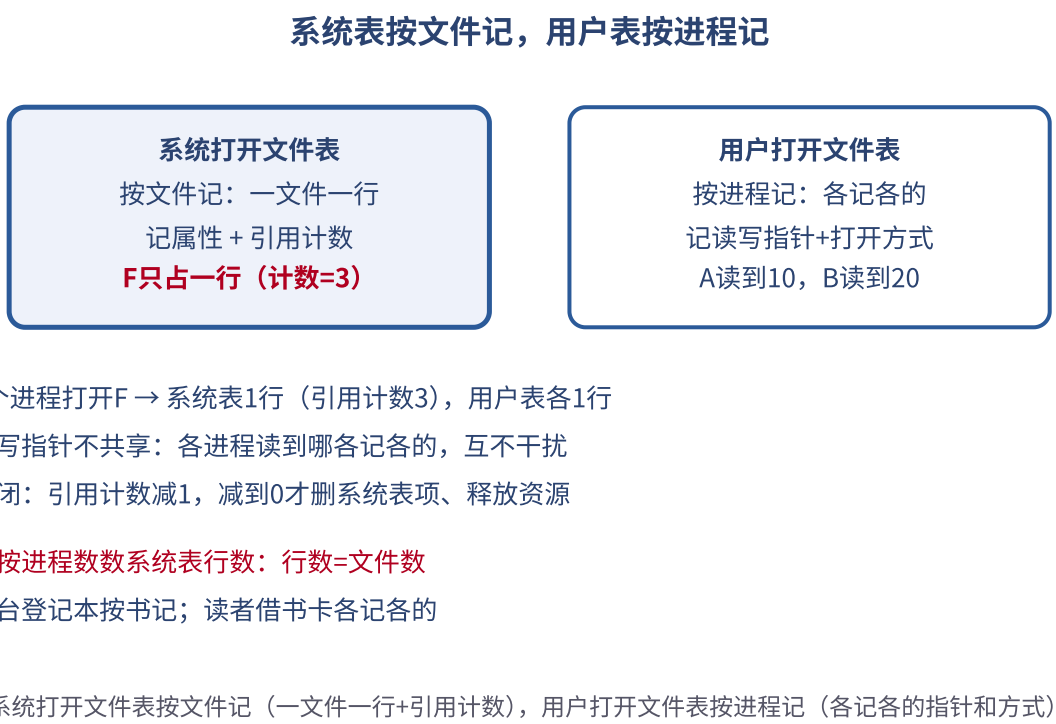
错误选项为什么错＋举例：

- A（各进程只能用"读"方式打开文件 F）：错在"只能读"。举例：A 进程以读方式打开 F 看内容，B 进程以写方式打开 F 改内容（只要权限允许），两者可以同时存在；共享不等于只读。
- C（各进程的用户打开文件表中关于 F 的表项内容相同）：错在"内容相同"。举例：A 进程读到第 100 字节，B 进程读到第 50 字节，两人的读写指针不一样；A 以读方式打开，B 以写方式打开，打开方式也不一样——用户表是各记各的，内容不一样。

- D（进程关闭 F 时，系统删除 F 在系统打开文件表中的表项）：错在"关闭就删"。举例：A、B、C 三个进程都打开了 F，引用计数是 3；A 关闭时计数减到 2，B、C 还在用，系统表那一行不能删；等 C 也关闭，计数减到 0，才删。教训：关闭是"计数减 1"，不是"直接删行"。

笔记本一句话： 第 23 题＋系统表按文件记（一文件一行＋引用计数），用户表按进程记（各记各的指针和方式）；关闭是计数减 1，减到 0 才删。

【图像】



系统打开文件表按文件记（一文件一行+引用计数），用户打开文件表按进程记（各记各的指针和方式）

【定位】 操作系统 · 第 4 章文件管理 · 文件的打开与关闭

【易混对比】

- "系统打开文件表"和"用户打开文件表"是两回事：系统表按文件记（一个文件一行，记属性和引用计数）；用户表按进程记（每个进程一张，记读写指针和打开方式）——"一个表项包含 F 的属性"说的是系统表。
- "打开"和"关闭"是两回事：打开是建表项、引用计数加 1；关闭是引用计数减 1，减到 0 才真正删表项、释放资源。
- "共享"和"独占"是两回事：多进程共享同一文件，系统表里只有一行（引用计数是几），各进程的用户表里各记各的读写位置——读写指针不共享。

系统表按文件记（一文件一行），用户表按进程记（各记各的）。

【坑位】

多进程共享同一文件，系统表里只有一行——别按进程数数行数。

最容易踩的坑，是觉得"三个进程打开 F，系统表里就有三行"。系统打开文件表是按文件记的：F 只占一行，行里有个引用计数记成 3，表示有 3 个进程在用它。而每个进程自己的用户打开文件表里各有一行，记着各自的读写指针（A 读到第 10 个字节，B 读到第 20 个字节，互不干扰）和打开方式。关闭时引用计数减 1，减到 0 才删系统表项。

【套路】

- 第一步，分两个表：系统表（按文件）记属性和计数，用户表（按进程）记指针和方式。
- 第二步，数行数：系统表行数 = 文件数，不是进程数。
- 第三步，算关闭：关闭一次计数减 1，减到 0 才删。

分表 → 按文件数行 → 关闭减计数。

【类比】

像图书馆借书登记：总台的登记本按书记（《F》这本书一行，记着被 3 个人借走了）；每个读者的借书卡上各记一笔（张三借到第 10 页，李四借到第 20 页）。还书时总台把计数减 1，3 个人都还了才把这一行销掉。不会因为 3 个人借同一本书，总台就记 3 行。

【口诀】 系统表按文件记，用户表按进程记。

【考频】 「系统打开文件表」在 2009–2022 年解析中出现 2 次 · 低频

第 24 题 · 支持长度可变、随机访问的分配方式（正确答案：A，索引分配）

题干与选项：

下列选项中，支持文件长度可变、随机访问的磁盘存储空间分配方式是（）。

- A. 索引分配
- B. 链接分配
- C. 连续分配
- D. 动态分区分配

入手点： 两个要求同时满足才行：长度能变（文件长大了不用搬家）＋能随机访问（想读第几块直接找到，不用从头找）——逐个方式验这两条。

题干专有名词从零解释：

- 先想书的目录页：每章在第几页，目录上写得清清楚楚，想看第 5 章翻目录就找到了——这就叫"索引分配"，意思是每个文件配一张索引表（索引块），表里写着文件每一块在磁盘的哪个位置。
- 再想寻宝游戏的纸条链：每张纸条写着下一张纸条在哪，要找第 5 张必须从第 1 张一张一张找下去——这就叫"链接分配"，意思是每个盘块尾巴指着下一个盘块。
- 再想一排连着的座位：文件占磁盘上一片连续的位置——这就叫"连续分配"，意思是文件的所有块挨在一起。
- 再想"动态分区分配"：这是内存的分配方式（给进程分内存），不是磁盘文件的分配方式——看到它先想"跑错场了"。

考查知识点：

- 先想"索引分配为什么两条都满足"：长度可变——文件长大了，往索引表里加一行（新块的位置）就行，不用搬家；随机访问——想读第 i 块，查索引表第 i 行，直接得到位置，不用从头找。
- 再想"链接分配为什么只满足一条"：长度可变——文件长大了，在链尾巴接一块就行，不用搬家；随机访问不行——想读第 i 块，必须从第 1 块顺着链找到第 i 块，越往后越慢。举例：读第 100 块要顺着链找 100 次。
- 再想"连续分配为什么只满足一条"：随机访问行——首地址 + 偏移直接算出第 i 块的位置；长度可变不行——文件长大了，后面可能已经有别的文件了，没地方续，只能整块搬家到更大的空地方。
- 再想"索引分配的代价"：每个文件要多占一个索引块（放表本身）；文件很大时，一张表放不下，要用多级索引（表里再指表）或混合索引（直接块 + 一级 + 二级 + 三级，Unix 就是这么做的）。
- 对照"链接分配的改进"：把链集中放到一张表里（FAT 表），随机访问就变快了——但本题选项说的是"链接分配"本身，不是 FAT 改进版，所以随机访问还是不行。
- 边界情况：文件很小（只有一两块）时，索引分配的索引块开销相对大，但功能上仍然满足两条要求；连续分配在文件创建时就要定大小，长大了就麻烦。

正确选项为什么对： 索引分配：长度可变（加索引行就行），随机访问（查表直达）。两条都满足。选 A。

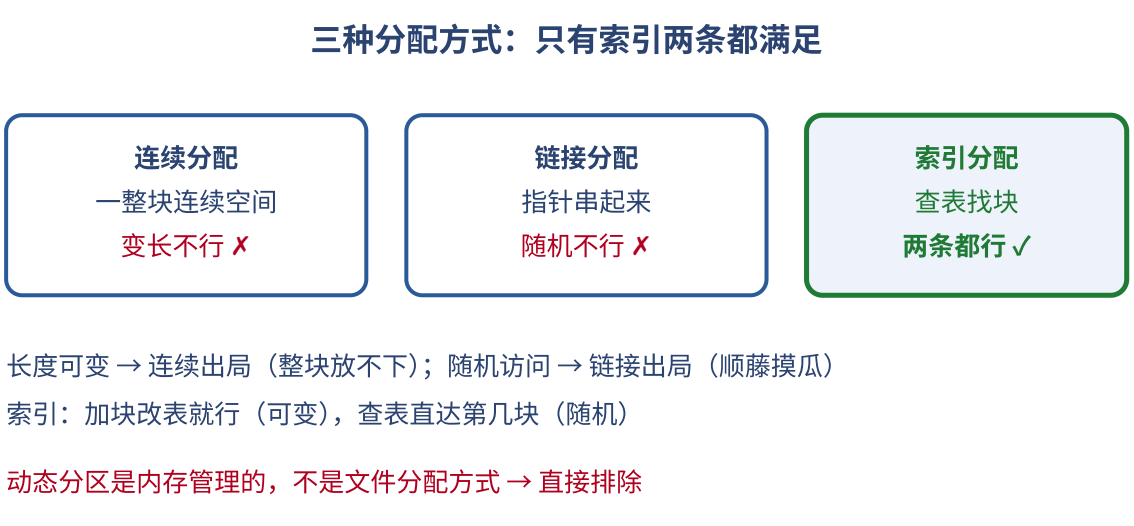
错误选项为什么错＋举例：

- B（链接分配）：长度可变是对的，但随机访问不行。举例：文件有 100 块，想读第 100 块，必须从第 1 块顺着链找 100 次——不能直接跳到第 100 块。教训：两个要求要同时验，只满足一个就不算。
- C（连续分配）：随机访问是对的（首地址 + 偏移直接算），但长度可变不行。举例：文件原来占 10 块连续位置，长大到 12 块，后面 2 块已经被别的文件占了，只能整块搬家。教训同上：只满足一个不算。

- D（动态分区分配）：这是内存分配方式（给进程分内存空间），不是磁盘文件的分配方式，跑错场了。教训：看到"动态分区"先想"内存"，不是"磁盘文件"。

笔记本一句话： 第 24 题＋索引两条都满足（加行可变、查表直达）；链接随机不行，连续变长不行，动态分区是内存的。

【图像】



三种分配方式：连续（变长不行）、链接（随机不行）、索引（两条都满足：加行可变、查表直达）

【定位】 操作系统 · 第 4 章文件管理 · 文件的物理结构

【易混对比】

- "连续分配"和"索引分配"是两回事：连续要求一整块连续空间，文件变长时可能放不下（变长不行）；索引是查表找块，加块就行（变长行）。
- "链接分配"和"索引分配"是两回事：链接要顺着指针一个一个找（随机访问不行）；索引查表直达第几块（随机行）。
- "动态分区"根本不是文件分配方式：它是内存管理的（给进程分内存），不是给文件分磁盘块的——看到就排除。

索引两条都满足：加块可变、查表直达。

【坑位】

动态分区是内存管理的，不是文件分配方式——看到"动态分区"先排除。

最容易踩的坑，是把内存管理的"动态分区"混进文件分配方式里。文件在磁盘上的三种分配方式：连续（快但变长难）、链接（省空间但只能顺序访问）、索引（查索引表直达任意块）。题目要"长度可变"又要"随机访问"：连续变长不行，链接随机不行，只有索引两条都满足。动态分区是给进程分配内存的，和文件在磁盘上怎么放没关系。

【套路】

- 第一步，列三种：连续、链接、索引，各记一条短板。

- 第二步，对题干两条要求：长度可变 → 连续出局；随机访问 → 链接出局。
- 第三步，剩下就是答案：索引。另见"动态分区"直接排除（内存的）。

列三种 → 对两条要求 → 排除出局。

【类比】

像停车：连续分配是"必须停一整排连号车位"（车变长了没连号位就停不下）；链接分配是"车位号只写在上一辆车的纸条上"，要找第 5 辆得一辆一辆问过去；索引分配是"门口有张总表，写着每辆车在几号位"，加车直接加一行表，想找第几辆查表直达。

【口诀】 索引两条都满足，加行可变查表直达。

【考频】 「索引分配」在 2009–2022 年解析中出现 4 次 · 中频

第 25 题 · OS 完成的中断操作（正确答案：D，II、III、IV）

题干与选项：

下列与中断相关的操作中，由操作系统完成的是（）。

- I. 保存被中断程序的断点；II. 提供中断服务；III. 初始化中断向量表；IV. 保存中断屏蔽字
- A. 仅 I、II
- B. 仅 I、II、IV
- C. 仅 III、IV
- D. 仅 II、III、IV

入手点： 分工口诀：硬件管"断点和入口"（快、固定），操作系统管"服务和现场"（灵活、可变）——逐项问"这是快的固定动作还是灵活的软件活"。

题干专有名词从零解释：

- 先想书签：程序被打断时执行到哪一行，这个位置叫"断点"，意思是回来时接着往下执行的地址。
- 再想电话簿：中断号对应处理程序地址的表叫"中断向量表"，意思是"几号中断找哪个处理程序"；初始化就是系统启动时把这本电话簿填好。
- 再想"中断服务"：真正处理中断的程序（比如键盘中断来了读键盘、时钟中断来了记时间）——这些程序是操作系统写的代码。
- 再想"免打扰名单"：中断屏蔽字就是一张表，写着哪些中断源现在不许打断；保存它就是处理中断时先记下原来的名单，处理完再恢复。

考查知识点：

- 先想"中断响应（硬件做的）"：CPU 收到中断信号后，硬件自动做三件事——关中断（防止嵌套乱套）、保存断点和程序状态字（书签）、查中断向量表找到处理程序入口并跳过去。这三件事又快又固定，必须由硬件做（软件来不及）。所以 I（保存断点）是硬件做的，不是操作系统做的。
- 再想"中断处理（操作系统做的）"：跳到处理程序后，操作系统做——保存现场（寄存器、屏蔽字等，IV）、执行真正的中断服务（II）、恢复现场、开中断、返回被打断的程序。
- 再想"初始化中断向量表（操作系统做的）"：系统启动时，操作系统把每个中断号对应的处理程序地址填进向量表（III）——这是启动时的一次性软件工作，不是硬件动作。
- 对照"硬件和软件的分工"：硬件管"断点和入口"（保存断点、找入口），操作系统管"服务和现场"（提供服务、保存恢复现场和屏蔽字、初始化表）。记住这个口诀，中断分工题就不会错。
- 边界情况：中断响应时硬件保存的只是断点和程序状态字，寄存器等"现场"由操作系统在处理程序里保存——"断点"和"现场"是两个东西，别混在一起。

正确选项为什么对： II 提供中断服务是操作系统代码，III 初始化中断向量表是操作系统启动时填表，IV 保存中断屏蔽字是操作系统处理时保存现场；I 保存断点是硬件在中断响应时自动做的。所以操作系统完成的是 II、III、IV。选 D。

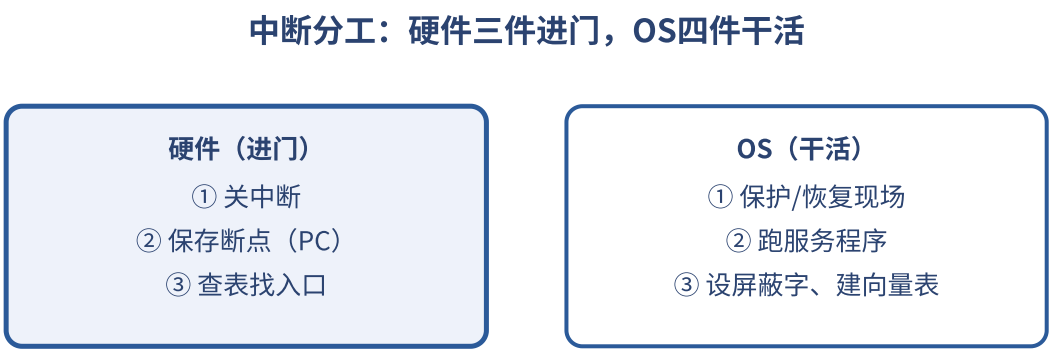
错误选项为什么错＋举例：

- A（仅 I、II）：含了 I（保存断点是硬件做的，不是操作系统做的），还漏了 III、IV。教训：看到"保存断点"先想"硬件自动保存"，不是操作系统。

- B（仅 I、II、IV）：含了 I（硬件的），还漏了 III（初始化向量表是操作系统启动时做的）。教训：I、III 一个是硬件一个是软件，别把硬件的算到操作系统头上，也别漏了软件的。
- C（仅 III、IV）：漏了 II（提供中断服务是最主要的操作系统工作）。教训：中断服务程序本身就是操作系统代码，这一项最不该漏。

笔记本一句话： 第 25 题＋中断分工：硬件管断点和入口，操作系统管服务、现场、屏蔽字和初始化向量表。

【图像】



向量表是OS开机初始化时建的；硬件只"查表"，不"建表"
保护现场=存寄存器（OS）；保存断点=存PC（硬件）——两回事

"硬件初始化向量表"→错；"缺失必须OS处理"→太绝对，错
门卫（硬件）：关门、登记、查名录；物业（OS）：招待、收拾、定规矩

中断分工：硬件管断点保存和入口查找，OS管服务程序、现场保护、屏蔽字和向量表

【定位】 操作系统 · 第 1 章计算机系统概述 · 中断处理

【易混对比】

- "硬件做的"和"OS 做的"是两回事：硬件只管"断点存哪、中断从哪进"（关中断、保存断点、查向量表找入口）；OS 管"进去之后的事"（跑服务程序、保护/恢复现场、设屏蔽字、初始化向量表）。
- "中断向量表"是谁建的：表是 OS 初始化时建的（把每个中断号和服务程序入口填进去）；硬件只负责"查表"，不负责"建表"。
- "保护现场"是谁的活：OS 的——中断来了，OS 先把 CPU 的寄存器存起来（保护现场），处理完再恢复；硬件只保存断点（PC）。

硬件管断点和入口，OS 管服务、现场、屏蔽字和向量表。

【坑位】

中断向量表是 OS 建的，硬件只查表——"硬件初始化向量表"的说法是错的。

最容易踩的坑，是把"查表"和"建表"混为一谈。中断来了，硬件做三件事：关中断（不让嵌套乱套）、保存断点（PC 压栈）、查向量表找到服务程序入口——然后跳进去，剩下的全是 OS 的活：保护现场（存寄存器）、执行中断服务程序、设屏蔽字、恢复现场、开中断。而向量表本身是 OS 在开机初始化时建好的，硬件从来不建表。

【套路】

- 第一步，分两段：进门（硬件）→ 干活（OS）。

- 第二步，硬件三件：关中断、存断点、查表找入口。
- 第三步，OS 四件：保护现场、跑服务、设屏蔽字、建向量表（初始化时）。

硬件三件进门，OS 四件干活。

【类比】

像小区门卫和物业：有访客（中断）来，门卫（硬件）只做三件事——关门不让乱进（关中断）、登记访客从哪来（存断点）、查业主名录找到门牌号（查表找入口）。进门之后招待客人、收拾房间、定规矩（服务、现场、屏蔽字），全是物业（OS）的活。业主名录（向量表）也是物业提前建好的，门卫只查不建。

【口诀】 硬件管断点入口，OS管服务现场。

【考频】 「中断处理」在 2009–2022 年解析中出现 27 次 · 高频

第 26 题 · 多级反馈队列设计要素（正确答案：D，I、II、III 和 IV）

题干与选项：

下列与进程调度有关的因素中，在设计多级反馈队列调度算法时需要考虑的是（）。

- I. 就绪队列的数量；II. 就绪队列的优先级；III. 各就绪队列的调度算法；IV. 进程在就绪队列间的迁移条件
- A. 仅 I、II
- B. 仅 III、IV
- C. 仅 II、III、IV
- D. I、II、III 和 IV

入手点： 把多级反馈队列想成"多条排队通道"：几条通道、哪条先、每条内部怎么排、什么时候换通道——四个问题一个都不能少。

题干专有名词从零解释：

- 先想多条排队通道：就绪的进程排成好几条队（就绪队列），每条队有个优先级（哪条先被 CPU 挑）——这就叫"多级队列"。
- 再想"反馈"和"迁移"：进程在一条队里没做完，可以被降到下一条队（优先级降低）；在低队里等太久了，可以被升回高队（防止饿死）——这种"能上下换队"就叫"反馈"，迁移条件就是"什么时候降、什么时候升"的规矩。
- 再想"时间片"：每条队给进程的时间额度——高优先级队的时间片短（让短进程快走），低优先级队的时间片长。

考查知识点：

- 先想"为什么要 I（队列的数量）"：几条通道是设计的第一步——3 条还是 5 条，直接影响调度效果；没有数量，后面都无从谈起。
- 再想"为什么要 II（队列的优先级）"：多条队总得有个先后——高优先级队里的进程先被 CPU 挑，低队里的等着；没有优先级，多条队就不知道先挑谁。
- 再想"为什么要 III（各队列的调度算法）"：每条队内部也要有个排法——高队常用时间片轮转（短进程快走），低队可以用先来先服务；不同队可以用不同算法，这是多级反馈队列灵活的地方。
- 再想"为什么要 IV（迁移条件）"：没有迁移条件，进程就固定在一条队里不动，那就退化成"多级队列"（固定分层），不是"多级反馈队列"了。迁移条件包括：用完时间片没做完 → 降级；低队等太久 → 升级（老化，防饥饿）。
- 对照"多级队列"和"多级反馈队列"：多级队列是进程分层后固定不动（比如系统进程一层、用户进程一层）；多级反馈队列是进程能上下换层——IV 正是区分两者的关键。
- 再想"多级反馈队列的好处"：不用预先知道进程要跑多久（短进程在高队很快跑完，长进程一路降级到底层慢慢跑）；I/O 繁忙的进程（经常主动让出 CPU）受益（总在高队）；老化机制防止长进程饿死。
- 边界情况：四项里缺任何一项，设计都不完整——缺数量不知道几条队，缺优先级不知道先挑谁，缺算法不知道队内怎么排，缺迁移条件就不是反馈队列了。所以四项全要。

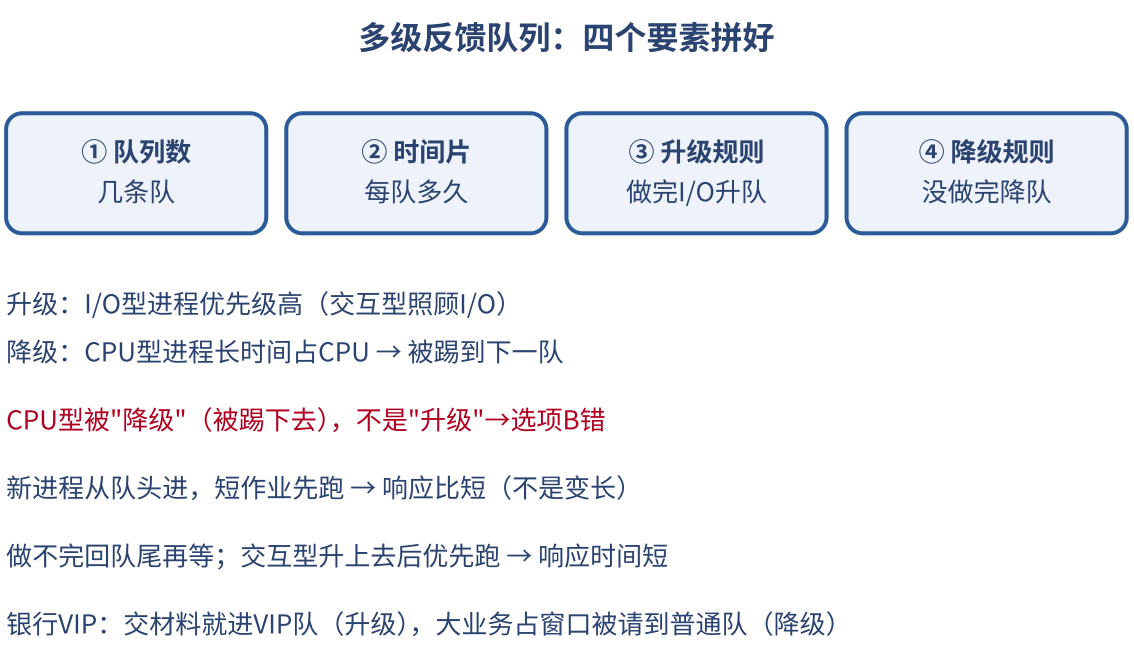
正确选项为什么对： 设计多级反馈队列要定：几条队（I）、哪条先（II）、每条队内怎么排（III）、什么时候换队（IV）——四项全是设计要素。选 D。

错误选项为什么错＋举例：

- A（仅 I、II）：漏了 III 和 IV。举例：定了 3 条队、定了优先级，但没定每条队内是轮转还是先来先服务，也没定用完时间片降不降级——设计不完整，没法运行。
- B（仅 III、IV）：漏了 I 和 II。举例：定了队内算法和迁移条件，但没定有几条队、哪条先——连队伍长什么样都不知道，算法和迁移条件无从附着。
- C（仅 II、III、IV）：漏了 I。举例：定了优先级、算法、迁移条件，但没定队列数量——几条队都没定，优先级排谁都不知道。教训：四项是一个整体，漏一个设计就不完整。

笔记本一句话： 第 26 题＋多级反馈队列设计四要素全要：几条队、哪条先、队内算法、换队条件（缺一个就不完整）。

【图像】



多级反馈队列四要素：几条队、哪条先、队内算法、换队条件，缺一个就不完整

【定位】 操作系统 · 第 2 章进程管理 · 处理机调度

【易混对比】

- "多级队列"和"多级反馈队列"是两回事：多级队列是进程固定在某条队；多级反馈队列是进程可以在队之间换（反馈）——"反馈"二字是关键。
- "设计要素"和"优点"是两回事：题干问的是"设计时要考虑的要素"（缺一个就不完整），不是"它有什么好处"——别把"响应快""兼顾长短作业"这些优点当成设计要素。
- 四个要素缺一不可：几条队列、哪条先调度、队内用什么算法、什么条件下换队——少了"换队条件"，就退化成普通多级队列了。

四要素全要：几条队、哪条先、队内算法、换队条件。

【坑位】

- 坑在"升级/降级"方向：CPU型进程长时间占CPU，是被降级（踢到下一队），不是升级——选项B把方向说反了，看到"升级"先问"谁被升级"。

- 坑在"要素 vs 优点"：题干问"设计时要考虑的"，缺一个设计就不完整；"响应时间短""兼顾长短作业"是它的优点，不是设计要素——别拿优点充数。

【套路】

- 第一步，数要素：I 几条队列、II 优先级顺序、III 队内算法、IV 换队条件——四个全在。
- 第二步，想缺一：缺"换队条件"就不是"反馈"了，缺"队内算法"就不知道队里怎么排——每个都有用。
- 第三步，全选：四个都是设计要素，选 D。

数四个 → 想缺一 → 全选。

【类比】

像设计一个多窗口银行：要定几个窗口（几条队）、VIP 窗口先办还是普通窗口先办（哪条先）、每个窗口里按取号还是按业务分（队内算法）、什么情况下客户可以换窗口（换队条件）。四个不定全，银行就开不起来——"换窗口"这个机制，正是"反馈"二字的来历。

【口诀】 几条队哪条先，队内算法换队条件。

【考频】 「多级反馈队列」在 2009–2022 年解析中出现 1 次 · 低频

第 27 题 · 银行家算法安全序列（正确答案：B，P2、P1、P3）

题干与选项：

某系统中有 A、B 两类资源各 6 个，t 时刻资源分配及需求情况如下表所示。

| 进程 | A 已分配数量 | B 已分配数量 | A 需求总量 | B 需求总量 |

| P1 | 2 | 3 | 4 | 4 |

| P2 | 2 | 1 | 3 | 1 |

| P3 | 1 | 2 | 3 | 4 |

t 时刻安全性检测结果是（）。

- A. 存在安全序列 P1、P2、P3
- B. 存在安全序列 P2、P1、P3
- C. 存在安全序列 P2、P3、P1
- D. 不存在安全序列

入手点： 三步走：先算"还需多少"（需求总量 − 已分配），再算"手里剩多少"（总数 − 已分配之和），然后找"手里够让谁先还完"，还完一个手里变多，再找下一个。

题干专有名词从零解释：

- 先想银行放贷：银行手里有 6 块 A 资源、6 块 B 资源，三个客户（进程）各借了一部分，还想再借一部分。银行放贷前先算账：手里剩的够不够让某个客户先还完？够就先满足他，他还完钱银行手里更多，再满足下一个——这就叫"银行家算法"，意思是放贷前的安全检查。
- 再想"还需"：还需 = 需求总量 − 已分配——客户总共想要多少，减去已经借到手的，就是还差多少。
- 再想"安全序列"：一个客户先后顺序——按这个顺序一个一个满足，每个客户轮到时手里的资源都够他还完，最后所有客户都还完——这样的顺序就叫安全序列；有安全序列就是"安全状态"（不会死锁）。

考查知识点：

- 先算"还需"：P1 还需 A：4 − 2 = 2，B：4 − 3 = 1，即（2，1）；P2 还需 A：3 − 2 = 1，B：1 − 1 = 0，即（1，0）；P3 还需 A：3 − 1 = 2，B：4 − 2 = 2，即（2，2）。
- 再算"手里剩"：A 已分配 2 + 2 + 1 = 5，剩 6 − 5 = 1；B 已分配 3 + 1 + 2 = 6，剩 6 − 6 = 0。手里剩（1，0）。
- 再找安全序列：手里（1，0），谁的还需 ⩽ 手里？P1（2，1）不行（A 要 2 个，手里只有 1 个）；P2（1，0）行（1 ⩽ 1，0 ⩽ 0）——P2 先。P2 还完释放（2，1），手里变成（1 + 2，0 + 1）=（3，1）。再看：P1（2，1）⩽（3，1）行——P1 第二，释放（2，3），手里变成（5，4）。P3（2，2）⩽（5，4）行——P3 第三。所以安全序列是 P2、P1、P3。
- 对照"不安全"和"死锁"：不安全只是"可能死锁"（没有安全顺序），不是"一定死锁"；安全则"一定不死锁"。银行家算法是死锁避免（事前检查），和死锁预防（破坏死锁条件）、死锁检测与解除（事后发现再处理）是三条不同路线。
- 边界情况：还需和手里要"两类资源同时够"才算够——A 够 B 不够也不行（P3 第二时就是 B 不够）；释放的资源要加回手里再算下一个，不能忘了加。

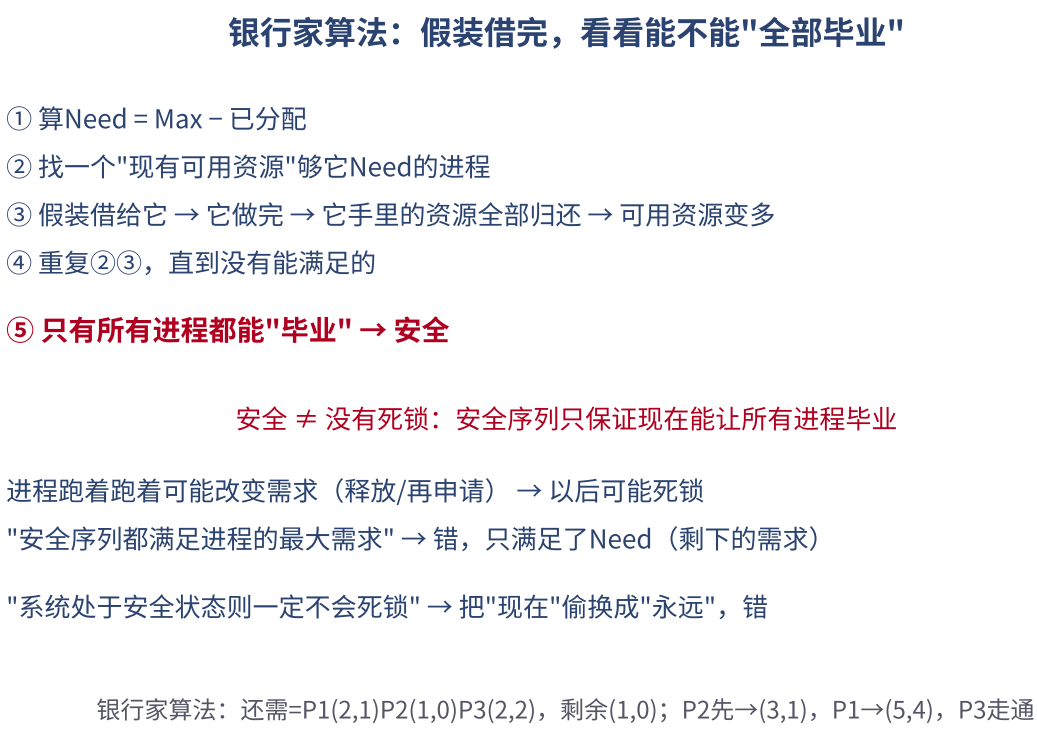
正确选项为什么对： 还需：P1（2，1）、P2（1，0）、P3（2，2）；手里剩（1，0）。P2 先（还需 ⩽ 手里），释放后手里（3，1）；P1 第二（还需 ⩽ 手里），释放后手里（5，4）；P3 第三（还需 ⩽ 手里）。序列 P2、P1、P3 全走通，是安全序列。选 B。

错误选项为什么错＋举例：

- A（P1、P2、P3）：P1 不能先——P1 还需 A 2 个，手里只有 1 个，A 不够。教训：找安全序列先看"手里够谁"，不是按 P1、P2、P3 的编号顺序排。
- C（P2、P3、P1）：P2 先是对的，但 P3 不能第二——P2 还完手里是（3，1），P3 还需 B 2 个，手里只有 1 个，B 不够。教训：每走一步都要重新算手里剩多少，两类资源同时够才算够。
- D（不存在安全序列）：错，因为 P2、P1、P3 就是安全序列。教训："不存在"要先穷举——只要找到一个安全序列，"不存在"就不成立。

笔记本一句话： 第 27 题＋银行家算法先算还需（总量－已分配）和剩余（总数－已分配和），找"还需 ⩽ 剩余"的先走，走完释放再加回手里，两类同时够才算够。

【图像】



【定位】 操作系统 · 第 2 章进程管理 · 死锁避免（银行家算法）

【易混对比】

- "还需"和"已分配"是两回事：还需 = 需求总量 − 已分配（还差多少）；已分配是已经拿在手里的——找安全序列看的是"还需 ⩽ 剩余"，不是"已分配"。
- "剩余"和"总数"是两回事：剩余 = 总数 − 已分配之和（银行手里还剩多少）；总数是 6 个，不会变。
- "不安全"和"死锁"是两回事：不安全只是"可能死锁"（没有安全顺序），不是"一定死锁"；安全则"一定不死锁"。

先算还需和剩余，找"还需 ⩽ 剩余"的先走，两类资源同时够才算够。

【坑位】

还需和剩余要"两类资源同时够"才算够——A 够 B 不够也不行。

最容易踩的坑，是只看一类资源够就放行。本题 P2 走完手里是（3,1），P3 还需（2,2）：A 够（2 ⩽ 3），但 B 不够（2 > 1）——P3 不能走。必须两类同时够（P1 的（2,1）⩽（3,1）才行）。另一个坑是忘了"释放加回"：P2 走完

要把它的已分配（2,1）加回手里，手里变成（3,1），再算下一个。

【套路】

- 第一步，算还需：需求总量 − 已分配 → P1(2,1)、P2(1,0)、P3(2,2)。
- 第二步，算剩余：总数 − 已分配之和 → (1,0)。
- 第三步，找"还需 ≤ 剩余"的先走：P2(1,0) ≤ (1,0) → P2 先；释放加回 → (3,1)；P1(2,1) ≤ (3,1) → P1；释放加回 → (5,4)；P3(2,2) ≤ (5,4) → P3。
- 第四步，全走通就是安全序列：P2、P1、P3。

算还需 → 算剩余 → 找够格的走 → 释放加回 → 走通即安全。

【类比】

像银行放贷前算账：手里剩 1 块 A、0 块 B。P2 还差 1 块 A、0 块 B——手里够，先借给他，他还完钱（连本带利 2 块 A、1 块 B）银行手里变成 3 块 A、1 块 B；再看 P1 还差 2 块 A、1 块 B——够，借给他，还完手里 5 块 A、4 块 B；最后 P3 还差 2 块 A、2 块 B——够。三个都还得上，银行就安全。

【口诀】 先算还需和剩余，还需小于剩余先走。

【考频】 「银行家算法」在 2009–2022 年解析中出现 8 次 · 中频

第 28 题 · 请求分页系统有效访存时间（正确答案：D，I、II、III 和 IV）

题干与选项：

下列因素中，影响请求分页系统有效（平均）访存时间的是（ ）。

- I. 缺页率；II. 磁盘读写时间；III. 内存访问时间；IV. 执行缺页处理程序的 CPU 时间
- A. 仅 II、III
- B. 仅 I、IV
- C. 仅 I、III、IV
- D. I、II、III 和 IV

入手点： 有效访存时间就是"平均每次访问内存花多久"——大部分时候一次命中，偶尔缺页要去磁盘取，两种情况加权平均，四个因素都在公式里。

题干专有名词从零解释：

- 先想按需取页：程序运行时，页（内存的一小块）不在内存里才去磁盘取，用到了才取，没用到的不取——这就叫"请求分页"，意思是"请求时才分页调入"。
- 再想"缺页"：要访问的页不在内存里，就叫缺页；"缺页率"就是访问中缺页的比例（比如访问 100 次缺页 2 次，缺页率就是 2%）。
- 再想"有效（平均）访存时间"：平均每次访问内存花的时间，简称 EAT——命中时花内存时间，缺页时花缺页处理时间，两者按比例加权平均。

考查知识点：

- 先想公式：EAT = （1 − 缺页率） × 内存访问时间 + 缺页率 × 缺页处理时间。命中（概率 1 − p）只花一次内存访问；缺页（概率 p）要花缺页处理时间。
- 再想"缺页处理时间由什么组成"：缺页中断处理（CPU 执行缺页处理程序的时间，IV）+ 去磁盘读入所缺的页（磁盘读写时间，II）+ 如果被换出的页被改过，还要先写回磁盘（也是磁盘读写时间）+ 最后再访问一次内存（内存访问时间，III）。
- 所以四个因素全在公式里：缺页率（I）是权重，磁盘读写时间（II）是缺页处理的大头，内存访问时间（III）是命中时的花费，CPU 处理时间（IV）是缺页处理的一部分。
- 对照"缺页率的影响"：缺页率越高，EAT 越接近缺页处理时间（很慢）；缺页率越低，EAT 越接近内存访问时间（很快）。所以降低缺页率（比如换更好的页面置换算法、给进程分更多内存块）是优化 EAT 的主要手段。
- 边界情况：如果缺页率是 0（所有页都在内存），EAT 就是内存访问时间；如果缺页率是 1（每次都缺页），EAT 就是缺页处理时间。实际系统缺页率很小（比如 1% 以下），但缺页处理时间是内存时间的上千倍，所以 EAT 对缺页率很敏感。
- 容易踩的坑：缺页处理时间里既有磁盘时间又有 CPU 时间，不能只算一个；内存访问时间既出现在命中项里，也出现在缺页处理后的重访问里——公式里两处都有它。

正确选项为什么对： 公式 EAT = （1 − p） × 内存时间 + p × 缺页处理时间，缺页处理时间 = CPU 处理时间 + 磁盘读写时间 + 再访内存时间。四个因素：缺页率（权重）、磁盘读写时间、内存访问时间、CPU 处理时间，全在公式里。选 D。

错误选项为什么错＋举例：

- A（仅 II、III）：漏了 I 和 IV。举例：缺页率从 1% 涨到 10%，EAT 会涨很多（缺页处理时间是内存时间的上千倍）——缺页率是权重，不能漏；CPU 执行缺页处理程序也要时间，不能漏。
- B（仅 I、IV）：漏了 II 和 III。举例：磁盘读写是缺页处理的大头（毫秒级），内存访问是命中时的花费（纳秒级）——两者都是公式里的量，漏了公式就不完整。
- C（仅 I、III、IV）：漏了 II。举例：缺页处理时间里，去磁盘读页的时间占大头（磁盘比内存慢上千倍）；漏了磁盘读写时间，缺页处理时间就只剩零头了。教训：四个因素逐个对公式，缺一个都不完整。

笔记本一句话： 第 28 题＋EAT =（1−p）×内存时间 + p×缺页处理时间；缺页处理 = CPU 处理 + 磁盘读写 + 再访内存，四个因素全在公式里。

【图像】

EAT = TLB命中路径 + TLB未命中路径

EAT = a(c+d) + (1−a)(c+2d)

TLB命中（概率a）	**TLB未命中（概率1−a）**
查TLB(c) + 访内存(d)	查TLB(c)+访页表(d)+访内存(d)
= c + d	= c + 2d

两种情况都包含查TLB → 公式是 a(c+d)+(1−a)(c+2d)，不是 ac+(1−a)2d

查表项在内存里，缺页调页才是例外（本题不考虑）

只算命中路径（ac+...）→ 漏了未命中那段路

期望时间：走路+开车两种情况加权平均，路都得走完整

EAT=(1−p)×内存时间+p×缺页处理时间；缺页处理=CPU处理+磁盘读写+再访内存

【定位】 操作系统 · 第 3 章内存管理 · 请求分页管理方式

【易混对比】

- "缺页"和"缺页中断"不是一回事：缺页是事件（要找的页不在内存）；缺页中断是处理这个事件的机制——EAT 公式里用的是缺页率 p。
- "缺页处理时间"和"内存访问时间"是两回事：缺页处理 = 中断 CPU 处理 + 磁盘读写页面 + 再访问一次内存——比单纯访内存慢得多，不能只算磁盘。
- "（1−p）"和"p"是两回事：不缺页的概率是 (1−p)，走内存时间；缺页的概率是 p，走缺页处理时间——加权平均，别漏了 (1−p) 那一项。

EAT =（1−p）× 内存时间 + p × 缺页处理时间；缺页处理三段全算。

【坑位】

缺页处理时间 = CPU 处理 + 磁盘读写 + 再访内存——"再访一次内存"最容易漏。

最容易踩的坑，是缺页处理只算"磁盘读写"。完整的缺页处理有三段：CPU 陷入中断做处理（存现场、找空页框），磁盘把页面读进内存（最慢的一段），再重新访问一次内存（把数据真正取出来）。EAT 是加权平均：(1−p)

的概率一次访存命中，p 的概率走完缺页三段。本题四个因素（缺页率、内存时间、缺页中断处理、磁盘读写）全在公式里，所以全选。

【套路】

- 第一步，写公式：EAT = (1−p)×t内存 + p×t缺页处理。
- 第二步，展开缺页处理：t缺页处理 = tCPU处理 + t磁盘 + t再访内存。
- 第三步，对因素：缺页率 p、内存时间、CPU 处理、磁盘读写——四个全在公式里，全选。

写公式 → 展开三段 → 对因素。

【类比】

像去档案室拿文件：(1−p) 的概率文件就在手边（一次拿到）；p 的概率得跑一趟档案室——先登记（CPU 处理），再等管理员找文件（磁盘读写），最后拿回来再看一眼（再访内存）。平均耗时 = 手边概率×手边时间 + 跑腿概率×跑腿全程。跑腿回来"再看一眼"那一下不能省。

【口诀】 EAT加权算，缺页三段加。

【考频】 「有效访存时间」在 2009–2022 年解析中出现 29 次 · 高频

第 29 题 · 父子进程（正确答案：B，不正确）

题干与选项：

下列关于父进程与子进程的叙述中，错误的是（）。

- A. 父进程与子进程可以并发执行
- B. 父进程与子进程共享虚拟地址空间
- C. 父进程与子进程有不同的进程控制块
- D. 父进程与子进程不能同时使用同一临界资源

入手点： 题干问"错误的是"——盯住 B 里的"共享虚拟地址空间"：共享地址空间的是线程，不是进程；父子进程各有各的地址空间。

题干专有名词从零解释：

- 先想生孩子：一个进程（父进程）通过 fork 生出另一个进程（子进程），子进程一开始和父进程长得一样（代码、数据都复制了一份）。
- 再想"进程控制块"：每个进程的"身份证"，记着进程号、状态、寄存器、资源清单——简称 PCB，一个进程一张，父子各有一张。
- 再想"虚拟地址空间"：每个进程以为自己独占的内存地图（从 0 开始的一大片地址），实际由操作系统映射到物理内存上——每个进程各有一张地图，互不干扰。
- 再想"临界资源"：一次只能一个人用的东西（比如打印机、某个共享变量），用的时候要互斥（排队一个一个用）。
- 再想"线程"：同一个进程里的多个干活小分队——线程们共用一个进程的地址空间，各自有自己的栈和寄存器。

考查知识点：

- 先想"父子进程的地址空间"：fork 之后，子进程得到父进程地址空间的一份复制品（代码、数据、堆栈都复制），但复制之后就是两份独立的地图——父进程改自己的变量，子进程看不到；子进程改自己的变量，父进程也看不到。实际实现用"写时复制"（开始时共享物理页，谁写了才真正复制），但虚拟地址空间从逻辑上讲各自独立。
- 再想"共享地址空间的是谁"：同一进程内的多个线程共享该进程的地址空间——线程 A 改了全局变量，线程 B 立刻看得到。所以"共享虚拟地址空间"说的是线程，不是父子进程。
- 再想"父子能不能并发"：能——多道程序下，父子进程可以交替执行（单核）或并行执行（多核），互不阻塞。
- 再想"PCB"：每个进程必有自己的 PCB（身份证），父子进程号不同、PCB 不同——没有 PCB，操作系统就管不了这个进程。
- 再想"临界资源"：互斥是铁律——不管父子还是陌生进程，临界资源一次只能一个进程用；父子也不能同时用，否则数据就乱了。
- 边界情况：父子进程可以共享打开的文件（子进程继承父进程的文件描述符，指向同一个文件表项），但这不等于共享地址空间——文件共享和地址空间共享是两回事。

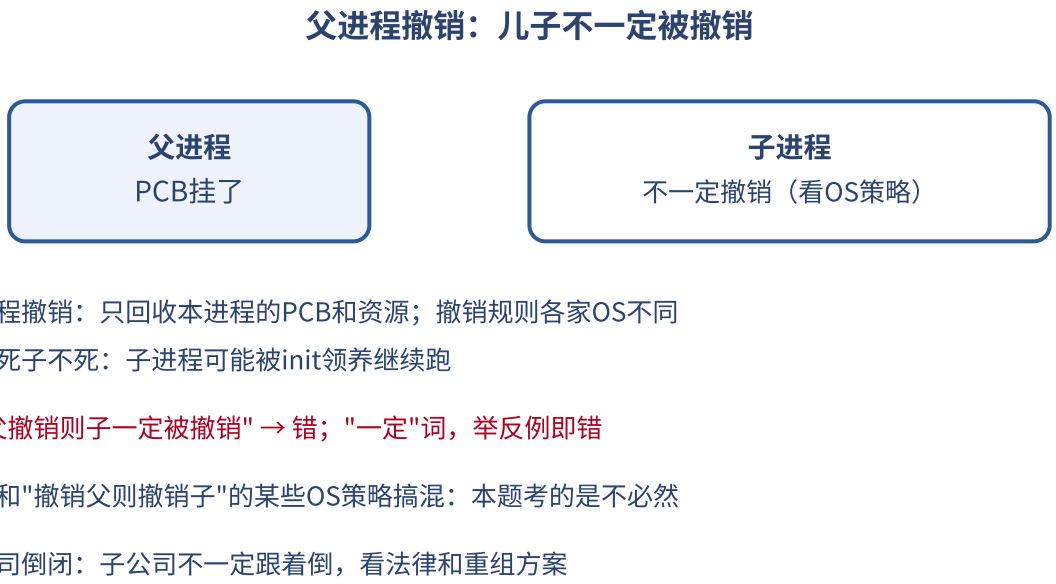
正确选项为什么对（即 B 为什么是错述）： B 说父进程与子进程"共享虚拟地址空间"。错在"共享"：父子进程各有独立的虚拟地址空间（fork 是复制，不是共享）；共享地址空间的是同一进程内的线程。所以 B 是错述，是本题的答案。选 B。

其余选项为什么是对的＋举例：

- A（父进程与子进程可以并发执行）：对。举例：父进程 fork 出子进程后，父子俩可以同时往下跑——单核上交替执行，多核上并行执行，谁也不用等谁先跑完。
- C（父进程与子进程有不同的进程控制块）：对。举例：父进程号是 100，子进程号是 101，各自有一张 PCB（身份证），记着各自的状态和资源——没有各自的 PCB，操作系统分不清谁是谁。
- D（父进程与子进程不能同时使用同一临界资源）：对。举例：父子俩都要用打印机，一次只能一个用——一个在用，另一个必须等着（互斥）；父子关系不打破互斥规则。

笔记本一句话： 第 29 题＋父子进程各有独立地址空间和 PCB，能并发；共享地址空间的是线程，不是父子进程；临界资源父子也不能同时用。

【图像】



父子进程各有独立地址空间和PCB，能并发；共享地址空间的是线程，不是父子进程

【定位】 操作系统 · 第 2 章进程管理 · 进程

【易混对比】

- "父子进程"和"线程"是两回事：父子进程各有独立的地址空间和 PCB；共享地址空间的是线程——"父子进程共享地址空间"的说法是错的。
- "并发"和"并行"是两回事：父子进程能并发（交替推进），有多个 CPU 时也能并行；但并发不等于共享地址空间。
- "临界资源"父子也不能同时用：临界资源一次只允许一个进程用，父子进程也要互斥——"父子可以同时用临界资源"是错的。

父子进程各有独立地址空间和 PCB；共享地址空间的是线程。

【坑位】

fork 出来的子进程是"复制"不是"共享"——地址空间各自独立，写操作互不影响。

最容易踩的坑，是觉得"子进程是父进程生的，所以共用一块内存"。fork 的语义是复制：子进程得到父进程地址空间的一份拷贝（写时复制优化下平时共用物理页，一写就分开），各自有独立的 PCB。父子能并发执行，但内存空间是两套。而"共享地址空间、共用一块内存"说的是线程——同一进程的多个线程才共享地址空间。把线程的特征安到父子进程头上，是本题的陷阱。

【套路】

- 第一步，定父子关系：fork 复制 → 独立地址空间 + 独立 PCB。
- 第二步，定线程关系：同一进程的线程 → 共享地址空间。
- 第三步，逐句验：凡说"父子共享地址空间""父子可同时用临界资源"的，一律错。

父子复制独立，线程共享；临界资源都要互斥。

【类比】

像分家：父亲（父进程）分家给儿子（子进程），房子是"复制"一套——户型一样，但两套房各住各的，儿子装修不影响父亲。而"合租"（线程）才是住同一套房、共用客厅。说"分家了还住同一套房"，是把合租的特征安到了分家头上。

【口诀】 父子各有独立空间，共享空间的是线程。

【考频】 「父子进程」在 2009–2022 年解析中出现 2 次 · 低频

第 30 题 · 设备独立性（正确答案：D，不正确）

题干与选项：

对于具备设备独立性的系统，下列叙述中，错误的是（）。

- A. 可以使用文件名访问物理设备
- B. 用户程序使用逻辑设备名访问物理设备
- C. 需要建立逻辑设备与物理设备之间的映射关系
- D. 更换物理设备后必须修改访问该设备的应用程序

入手点： 题干问"错误的是"——抓住"设备独立性"的意思：程序不用认具体设备，换了设备程序不用动；说"必须改程序"的正好反了。

题干专有名词从零解释：

- 先想统称和具体那台：程序里写的"打印机"是个统称，叫"逻辑设备名"；三楼那台具体的 HP 打印机叫"物理设备"。程序只认统称，不认具体型号——这就叫"设备独立性"，意思是程序独立于具体设备。
- 再想对照表：统称对应具体哪台，要有一张对照表，叫"逻辑设备与物理设备的映射关系"（逻辑设备表）——换设备时改这张表就行。
- 再想"文件名访问设备"：把设备当成文件看（比如 /dev/printer），用打开文件、读写文件的方式访问设备。

考查知识点：

- 先想"设备独立性的意义"：程序只写逻辑设备名（"打印机"），不写具体型号；换了一台新打印机，只要改映射表（"打印机"对应新那台），程序一行不用改。这就是设备独立性带来的好处——程序可移植、换设备不伤筋动骨。
- 再想"映射关系为什么必须有"：程序写的是统称，操作系统要知道统称对应哪台具体设备，必须建一张映射表；没有这张表，统称就找不到具体设备。
- 再想"文件名访问设备"：操作系统把设备统一成文件的样子（设备文件），程序用文件名打开设备、用读写文件的方式用设备——这是设备独立性的一种实现形式。
- 对照"没有设备独立性"：程序里写死具体设备（"三楼 HP LaserJet 1020"），换一台设备就得改程序、重新编译——这就是设备独立性要解决的问题。
- 边界情况：换设备时，驱动程序可能要换（新设备的操作代码不一样），映射表要改，但应用程序不用改——"改驱动 + 改映射表"和"改应用程序"是两回事，题干 D 说的是改应用程序，所以错。

正确选项为什么对（即 D 为什么是错述）： D 说"更换物理设备后必须修改访问该设备的应用程序"。错在"必须改程序"：设备独立性的意义恰恰是换设备不用改程序——改映射表（统称对应新设备）就行，应用程序一行不用动。所以 D 是错述，是本题的答案。选 D。

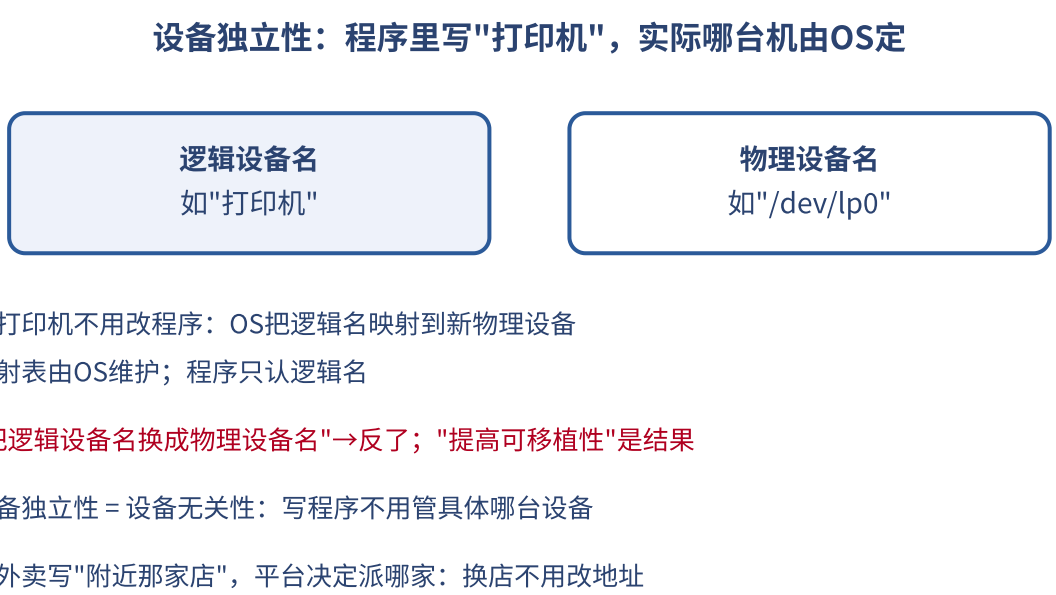
其余选项为什么是对的＋举例：

- A（可以使用文件名访问物理设备）：对。举例：把打印机当成文件 /dev/printer，程序打开这个"文件"、往里写数据就是打印——设备用文件的样子统一访问。
- B（用户程序使用逻辑设备名访问物理设备）：对。举例：程序里写"打印机"（统称），不写"三楼那台 HP 1020"（具体型号）——程序只认统称，具体哪台由操作系统定。

- C（需要建立逻辑设备与物理设备之间的映射关系）：对。举例：映射表里写着"打印机 → 三楼 HP 1020"；换了一台新打印机，映射表改成"打印机 → 四楼 HP 1030"，程序不用动——没有这张表，统称找不到具体设备。

笔记本一句话： 第 30 题＋设备独立性 = 程序只认统称（逻辑设备名），换设备改映射表就行，应用程序不用动；说"必须改程序"正好反了。

【图像】



设备独立性：程序只认逻辑设备名，换设备改映射表，应用程序不用动

【定位】 操作系统 · 第 5 章输入输出管理 · I/O 软件层次

【易混对比】

- "逻辑设备名"和"物理设备名"是两回事：程序里写的是逻辑名（如"打印机"这个统称）；物理名是具体哪台设备——设备独立性就是程序只认统称。
- "换设备改映射表"和"换设备改程序"是两回事：独立性好的系统，换设备只改逻辑到物理的映射表，应用程序一行不用动；说"必须改程序"正好反了。
- "设备独立性"和"设备无关性"是同一个意思的不同叫法，别当成两个概念。

程序只认统称（逻辑设备名），换设备改映射表就行，应用程序不用动。

【坑位】

说"换设备必须修改应用程序"的，正好把设备独立性说反了——直接判错。

最容易踩的坑，是把设备独立性的效果记反。设备独立性的意思就是：应用程序和具体的物理设备解耦——程序里只写"把数据送到打印机"（逻辑设备名），至于到底是哪台打印机，由系统的映射表决定。换了一台新打印机，改一下映射表就行，应用程序不用重新编译、不用改一行代码。说"必须修改应用程序才能换设备"，是把"独立"说成了"绑定"，正好反了。

【套路】

- 第一步，定定义：程序用逻辑设备名，不直接碰物理设备。

- 第二步，想换设备：改映射表，程序不动。
- 第三步，见"必须改程序"判错：和定义反着说的，一律错。

认统称 → 换设备改表 → 程序不动。

【类比】

像点外卖只认"黄焖鸡"这个品名：你下单写的是"黄焖鸡"（逻辑设备名），平台把单派给哪家店（物理设备）你不用管。换了一家新店，平台改一下派单映射就行，你的下单方式不用变。说"换店就得重学怎么下单"，是把"独立"说成了"绑定"。

【口诀】 程序只认统称，换设备改映射表。

【考频】 「设备独立性」在 2009–2022 年解析中出现 3 次 · 中频

第 31 题 · 文件数上限（正确答案：B，2^32）

题干与选项：

某文件系统的目录项由文件名和索引结点号构成。若每个目录项长度为 64 字节，其中 4 字节存放索引结点号，60 字节存放文件名。文件名由小写英文字母构成，则该文件系统能创建的文件数量的上限为（）。

- A. 2^26
- B. 2^32
- C. 2^60
- D. 2^64

入手点： 两个"上限"取小的那个：文件名能起多少种、索引结点号能编多少个——哪个少，文件数就卡在哪个。

题干专有名词从零解释：

- 先想车牌号：每个文件要有一个"索引结点号"，好比每辆车要有一个车牌号；号是 4 字节（32 位）编的，能编多少个号，文件数就最多多少个。
- 再想"目录项"：目录里的一行，记着文件名和这个文件的索引结点号——找文件时先翻目录，找到名字对应的号，再凭号找到文件本身。
- 再想"索引结点"：文件真正的"档案"（属性、大小、数据块位置），号就是档案的编号；一个号对应一个文件。

考查知识点：

- 先算"文件名能起多少种"：文件名 60 字节，每字节一个小写字母（26 种可能），总共 26^60 种可能的名字——这是个天文数字，基本用不完。
- 再算"索引结点号能编多少个"：号是 4 字节 = 32 位，2^32 个不同的号（从 0 到 2^32 − 1），所以最多 2^32 个文件。
- 再想"瓶颈"：文件名空间（26^60）大得用不完，号空间（2^32）小得多——文件数卡在号上，上限是 2^32。好比车主名字能起无数种，但车牌号只有 2^32 个，车再多，车牌不够就上不了牌。
- 对照"字节和位"：4 字节 = 32 位（1 字节 = 8 位），所以号空间是 2^32，不是 2^4；60 字节文件名也不是 60 位（60 字节 = 480 位，但每字节只用 26 种字母，所以是 26^60，不是 2^480）。
- 边界情况：索引结点号 0 通常保留（表示空），实际可用比 2^32 少 1 个，但题目问的是"上限"，按 2^32 算；目录项长度 64 字节本身不决定文件数上限（它只决定目录占多少空间）。

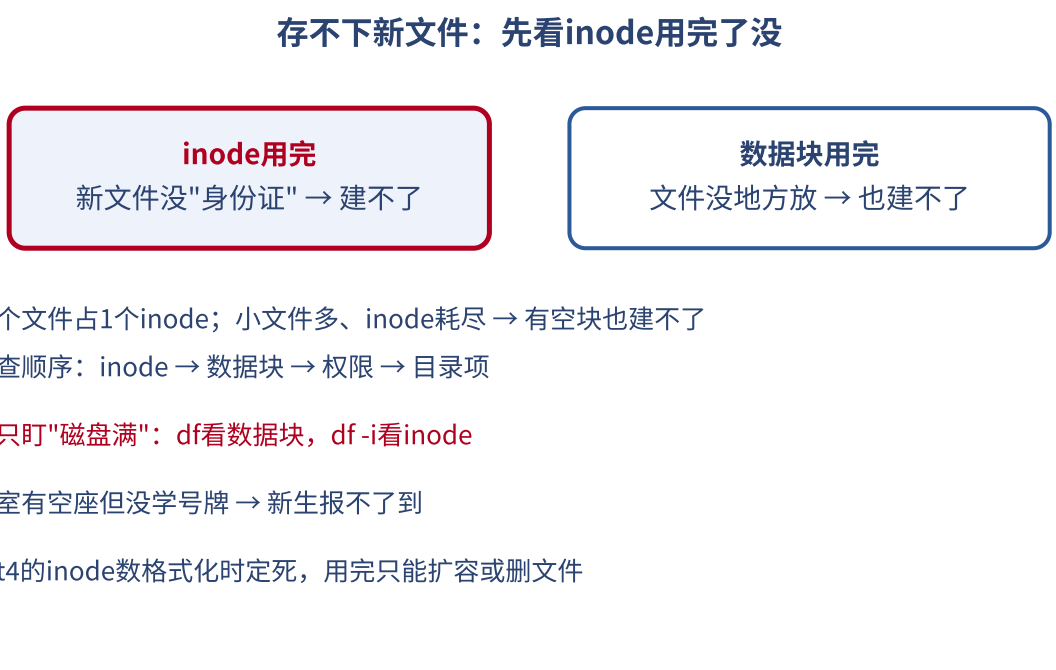
正确选项为什么对： 文件数上限由索引结点号空间定：4 字节 = 32 位，能编 2^32 个号，所以最多 2^32 个文件。选 B。

错误选项为什么错＋举例：

- A（2^26）：把"26 种字母"当成了上限。26 只是每字节的字母种类数，不是位数也不是文件数——文件名是 60 字节的组合，组合数是 26^60，不是 2^26。教训：字母种类数（26）和位数（32）是两回事，别混。
- C（2^60）：把"60 字节文件名"当成了 60 位。60 字节是长度，不是位数；而且文件名空间（26^60）再大也不是瓶颈，瓶颈是号空间。教训：先找瓶颈（号），不是哪个数大就用哪个。
- D（2^64）：把"64 字节目录项"当成了 64 位。目录项长度 64 字节只决定目录占多少空间，不决定文件数；文件数由号的位数（32 位）定。教训：目录项长度、文件名长度、号长度是三个数，文件数只看号长度。

笔记本一句话： 第 31 题＋文件数上限看索引结点号（4 字节 = 32 位 → 2^32），文件名空间再大也不是瓶颈；字节、位数、字母种类别混。

【图像】



【定位】 操作系统 · 第 4 章文件管理 · 文件系统

【易混对比】

- "索引结点号"和"文件名"是两回事：文件数上限看索引结点号（4 字节 = 32 位 → 2^32 个）；文件名空间再大也不是瓶颈。
- "字节"和"位"是两回事：4 字节 = 32 位，能编 2^32 个号；把"4 字节"当成"4 位"就错得离谱。
- "文件数上限"和"文件大小上限"是两回事：文件数看索引结点号有多少个；单个文件能多大看索引结构（直接/间接索引）——两个"上限"看的东西不同。

文件数上限看索引结点号：4 字节 = 32 位 → 2^32。

【坑位】

字节、位数、字母种类别混：4 字节是 32 位，不是 4 位也不是 4 种字母。

最容易踩的坑，是把"4 字节"和"能编多少个号"算错。索引结点号占 4 字节 = 32 位，32 位 0/1 能编 2^32 个不同的号，一个文件占一个号，所以文件数上限是 2^32。另一个坑是看文件名：文件名能起多长、能用多少种字母，和"能有多少个文件"是两回事——文件名空间再大，索引号不够也白搭。

【套路】

- 第一步，找"号"：文件数上限看索引结点号，不看文件名。
- 第二步，算位数：4 字节 = 32 位 → 2^32 个号。
- 第三步，排除干扰：文件名长度、字母种类都是干扰项。

找号 → 算位 → 排除干扰。

【类比】

像车牌号定汽车上限：一个城市能有多少辆车，看车牌号能编多少个（4 字节=32 位 → 2^32 个），不是看车主的名字能起多长。车牌号用完了，再长的名字也上不了牌。

【口诀】 文件数看索引号，四字节三十二位。

【考频】 「索引结点」在 2009–2022 年解析中出现 9 次 · 中频

第 32 题 · 临界区准则（正确答案：C，I、II、III）

题干与选项：

下列准则中，实现临界区互斥机制必须遵循的是（）。

- I. 两个进程不能同时进入临界区；II. 允许进程访问空闲的临界资源；
- III. 进程等待进入临界区的时间是有限的；IV. 不能进入临界区的执行态进程立即放弃 CPU
- A. 仅 I、IV
- B. 仅 II、III
- C. 仅 I、II、III
- D. 仅 I、III、IV

入手点： 把临界区想成"一次只能一个人进的房间"：不能两人同时进、空着就让人进、排队不能无限等——这三条是必须的；进不去时"立即放弃 CPU"不是必须的（原地打转等也是允许的）。

题干专有名词从零解释：

- 先想厕所单间：一次只能一个人进，进了把门锁上，出来开锁——这就叫"临界区"，意思是访问临界资源的那段代码；临界资源就是一次只能一个人用的东西（打印机、共享变量）。
- 再想"互斥"：不能两个人同时在里面——这是临界区最核心的规矩。
- 再想"空闲让进"：房间空着，就要让人进，不能空着还拦人。
- 再想"有限等待"：排队的人不能无限等下去，总能轮到他（不能饿死）。
- 再想"忙等"和"让权等待"：进不去时，原地打转等叫"忙等"（还占着 CPU）；把 CPU 让出来、等通知再来叫"让权等待"（不占 CPU）。

考查知识点：

- 先想"三条必须准则"：I 互斥（不能同时进）——这是临界区的定义，没有它数据就乱了；II 空闲让进（空着就让进）——房间空着还拦人是浪费；III 有限等待（等的时间有限）——排队总能轮到，不能饿死。
- 再想"IV 为什么不是必须准则"：IV 说"进不去的进程立即放弃 CPU"（让权等待）。让权等待是好的做法（省 CPU），但不是必须的——忙等（原地打转）也是允许的互斥实现方式（比如自旋锁就是忙等）。"必须遵循"和"建议这么做"是两回事：忙等虽然费 CPU，但不违反互斥，所以 IV 不能算必须准则。
- 对照"自旋锁"：进不去就原地循环检查（忙等），不放弃 CPU——这是合法实现，说明 IV 不是必须的。
- 边界情况：三条准则缺一不可——缺互斥数据乱，缺空闲让进浪费资源，缺有限等待会饿死；IV 可有可无（忙等和让权等待都是合法实现）。

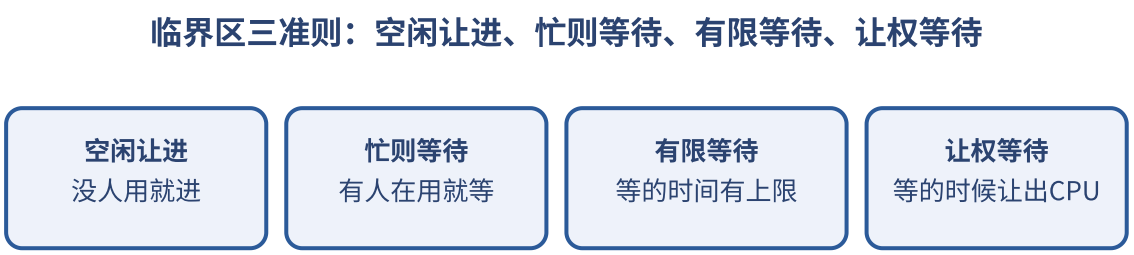
正确选项为什么对： 必须遵循的是 I（互斥）、II（空闲让进）、III（有限等待）；IV（让权等待）只是建议做法，忙等也合法，不是必须的。所以是 I、II、III。选 C。

错误选项为什么错＋举例：

- A（仅 I、IV）：含了 IV（让权等待不是必须的，忙等也合法），还漏了 II、III（空闲让进、有限等待都是必须的）。教训：IV 是"建议"，不是"必须"——看到"立即放弃 CPU"先想"忙等也行"。
- B（仅 II、III）：漏了 I（互斥是最核心的准则，没有互斥临界区就不存在）。教训：I 是定义本身，再怎么漏也不能漏 I。
- D（仅 I、III、IV）：含了 IV（不是必须的），还漏了 II（空闲让进是必须的）。教训：II 和 IV 一个是必须（空着就让进），一个是建议（让权等待），别把建议当必须、把必须漏了。

笔记本一句话： 第 32 题＋临界区三必须：互斥、空闲让进、有限等待；让权等待（立即放弃 CPU）只是建议，忙等也合法。

【图像】



题干列的三个（互斥进入、忙则等待、有限等待）就是这套准则
判断：看它列的是不是准则本身 → 是 → 正确

"临界区"的判定只看这几条，不看它访问了几个变量

互斥：同一时间最多一个进程在临界区里

单间厕所：没人就进（空闲让进），有人就排队（忙则等待），不能插队到天荒地老（有限等待）

临界区三准则：互斥、空闲让进、有限等待必须；让权等待只是建议，忙等也合法

【定位】 操作系统 · 第 2 章进程管理 · 进程同步

【易混对比】

- "必须"和"建议"是两回事：互斥、空闲让进、有限等待是三必须；让权等待（不等就立即放弃 CPU）只是建议，不是必须。
- "忙等"和"非法"是两回事：忙等（转圈圈等）浪费 CPU 但合法；让权等待是更优的做法，但"不等就必须让出 CPU"不是硬性要求。
- "有限等待"和"无限等待"是两回事：有限等待要求每个想进的进程最终都能进去（不饿死）；无限等待（饿死）是违反准则的。

临界区三必须：互斥、空闲让进、有限等待；让权等待只是建议。

【坑位】

忙等浪费 CPU 但合法——"必须让权等待"的说法太绝对，是错的。

最容易踩的坑，是把"好的做法"当成"必须的做法"。让权等待（等不到锁就主动放弃 CPU 去干别的）确实更省 CPU，是推荐的做法；但准则只要求三条：互斥（一次一个进）、空闲让进（没人用时想进就进）、有限等待（不饿死）。忙等（转圈圈死等）虽然浪费 CPU，但只要满足三条就是合法的同步机制。所以"让权等待是必须的"这个说法太绝对，是错的。

【类比】

像排队上厕所的三条铁律：一次进一个（互斥）、没人时想进就进（空闲让进）、不能让某个人永远排不上（有限等待）。至于"排队时玩不玩手机"（忙等还是让权等待）——玩手机更省电（推荐），但不玩手机干排着也不犯规。

【口诀】 临界三必须，互斥让进有限等；让权只是建议。

【考频】 「临界区」在 2009–2022 年解析中出现 13 次 · 高频

第 33 题 · 协议要素图（正确答案：C，仅 III）

题干与选项：

下图描述的协议要素是（）

- I. 语法
- II. 语义
- III. 时序
- A. 仅 I
- B. 仅 II
- C. 仅 III
- D. I、II 和 III

> 注：本题题干依赖图片（协议要素图），图片未能直接查看，本题标注"部分核对"。答案按答案表写（仅 III）；图中具体画的是哪种顺序关系未查看，图中细节以原卷为准。

入手点： 协议三要素先分清：语法是"话怎么说"（格式），语义是"话什么意思"（含义），时序是"话按什么顺序说"（顺序）——图里画的是顺序关系，就只对应时序。

题干专有名词从零解释：

- 先想两个人约定的规矩：通信双方要约好"怎么说话、什么意思、按什么顺序说"，这套规矩就叫"协议"。
- 再想"语法"：话的格式——先说什么、后说什么、每段多长。举例：数据报格式（先是头部、后是数据，头部每段几位）就是语法。
- 再想"语义"：话的意思——某个位是 1 表示什么、某个字段代表什么控制信息。举例：SYN 位是 1 表示"请求建立连接"，这就是语义。
- 再想"时序"：话的顺序——先发什么、后发什么、等多久、什么时候重发。举例：三次握手先 SYN、后 SYN+ACK、再 ACK，这个顺序就是时序。

考查知识点：

- 先想"三要素怎么区分"：看到格式、结构、字段长度 → 语法；看到含义、控制信息、"表示什么" → 语义；看到顺序、流程、先后、等待 → 时序。
- 再想"本题的图"：按答案表，图中所描述的是顺序关系（先发什么、后发什么的流程），不涉及数据格式（语法），也不涉及某个字段的含义（语义）——所以只对应时序（III）。
- 对照"三个要素齐全的例子"：TCP 三次握手——语法是段的格式（序号字段、标志位字段），语义是 SYN 表示请求建立、ACK 表示确认，时序是先 SYN、后 SYN+ACK、再 ACK 的顺序。三要素各管一块，缺一不可。
- 边界情况：一张图可能同时涉及多个要素（比如既画了格式又画了顺序），那就选多个；本题按答案表只有时序，说明图里只画了顺序关系。

正确选项为什么对： 按答案表，图中所描述的是协议的顺序关系（时序），不涉及格式（语法）和含义（语义）。所以是仅 III。选 C。

错误选项为什么错＋举例：

- A（仅 I）：语法是格式（先说什么、每段多长）；图里没有格式定义，只有顺序关系，所以不是语法。教训：看到"格式、结构"才想语法，看到"顺序、流程"想时序。

- B（仅 II）：语义是含义（某个位表示什么）；图里没有含义说明，只有顺序关系，所以不是语义。教训：看到"表示什么、控制信息"才想语义。
- D（I、II 和 III）：全选，含了图中没有的语法和语义。教训：三要素逐个对图——图里有什么选什么，没有的不选；全选只有在图里三要素齐全时才对。

笔记本一句话： 第 33 题＋协议三要素：语法是格式、语义是含义、时序是顺序；图里只画顺序就只选时序。

【图像】



语法：报文格式、字段划分；语义：字段含义、操作；时序：事件顺序、速度匹配

题干的说法对应三要素 → 正确

服务≠协议：服务是下层给上层的功能，协议是同层之间的约定

接口：上下层之间的交接面（SAP）

点外卖：菜单格式（语法）、菜名含义（语义）、下单→接单→配送（时序）

协议三要素：语法（格式）、语义（含义）、时序（顺序）；图里只画顺序就只选时序

【定位】 计算机网络 · 第 1 章概述 · 协议三要素

【易混对比】

- "语法"和"语义"是两回事：语法是数据的格式（长什么样），语义是数据的含义（表示什么）——格式对含义错，照样通信失败。
- "时序"和"顺序"是两回事：时序特指"先干什么、后干什么"的事件顺序（比如先握手再传数据）；图里只画了事件顺序，就只选时序。
- "协议"和"服务"是两回事：协议是通信双方共同遵守的规则（三要素）；服务是下层给上层的功能——别把"服务"当成三要素之一。

语法是格式，语义是含义，时序是顺序；图里只画顺序就只选时序。

【坑位】

- 坑在"服务"混入：三要素是语法、语义、时序——"服务"是下层给上层的功能，不是协议要素；看到"服务"出现在三要素的选项里，直接排除。
- 坑在"只画顺序就只选时序"：图里画的是事件先后顺序，对应的是时序；别因为"图里有箭头"就联想语法格式——图里有什么才选什么，没画的不选。

【套路】

- 第一步，定三要素：语法（格式）、语义（含义）、时序（顺序）。
- 第二步，看图画了什么：只画了事件先后顺序 → 只有时序。

- 第三步，对号：语法、语义在图里没体现，不选。

定三要素 → 看图 → 对号。

【类比】

像两个人约见面定的规矩：语法是"用微信文字约"（格式），语义是"周三下午三点"表示的时间（含义），时序是"先约好再出门"（顺序）。题干的图只画了"先干什么后干什么"，那它只体现了时序，没体现格式和含义。

【口诀】 语法格式语义义，时序顺序。

【考频】 「协议三要素」在 2009–2022 年解析中出现 1 次 · 低频

第 34 题 · 虚电路网络（正确答案：B，不正确）

题干与选项：

下列关于虚电路网络的叙述中错误的是：

- A. 可以确保数据分组传输顺序
- B. 需要为每条虚电路预分配带宽
- C. 建立虚电路时需要进行路由选择
- D. 依据虚电路号（VCID）进行数据分组转发

入手点： 题干问"错误的是"——抓住"虚"字：虚电路是"约好的路线"，不是"包下来的线"；预分配带宽是电路交换（包场）的做法，不是虚电路的。

题干专有名词从零解释：

- 先想约好的固定路线：发送前先约好一条从起点到终点的路线，之后所有分组都沿这条路线走——这就叫"虚电路"，意思是"虚拟的电路"（看着像一条专线，实际是约好的路线，不是真的独占一条线）。
- 再想包场：通话期间整条线归你一个人，别人不能用，带宽预先分给你——这就叫"电路交换"（比如老式电话）。
- 再想拼车：大家共享带宽，谁有数据谁用，没数据时别人用——这就叫"分组交换的统计复用"，意思是带宽按需共享，不预先分。
- 再想"虚电路号"：每条虚电路在每段链路上的编号（VCID），结点查表按号转发；号只在一段链路上有意义，过了结点可能换号。

考查知识点：

- 先想"虚电路为什么能保序"：所有分组沿同一条路线走，先发的先到、后发的后到，顺序不会乱——所以虚电路可以确保传输顺序。
- 再想"建路时干什么"：建立虚电路时要选路（路由选择）——定下来走哪条路线，沿途每个结点记下虚电路号和下一跳；建好后，分组就不用再选路了，按表转发。
- 再想"转发靠什么"：分组头里带着虚电路号，结点查虚电路表（号 → 下一跳），按号转发；不用查目的地址。
- 再想"带宽怎么分"：虚电路是分组交换，带宽统计复用——有数据时用，没数据时别人用，不预先给每条虚电路分一块固定带宽。预分配带宽是电路交换的做法（包场，独占）。
- 对照"数据报"：数据报是另一种分组交换——不用建路，每个分组独立选路，可能乱序、可能走不同路线；虚电路要建路、保序，两者正好对照。
- 边界情况：虚电路的"可靠"要看具体协议（X.25 提供可靠传输，帧中继、ATM 不保证可靠）——但"保序"是虚电路本身的性质，和可靠是两回事。

正确选项为什么对（即 B 为什么是错述）： B 说虚电路"需要为每条虚电路预分配带宽"。错在"预分配"：预分配带宽是电路交换（包场）的做法；虚电路是分组交换，带宽按需共享、统计复用，不预先分给每条虚电路。所以 B 是错述，是本题的答案。选 B。

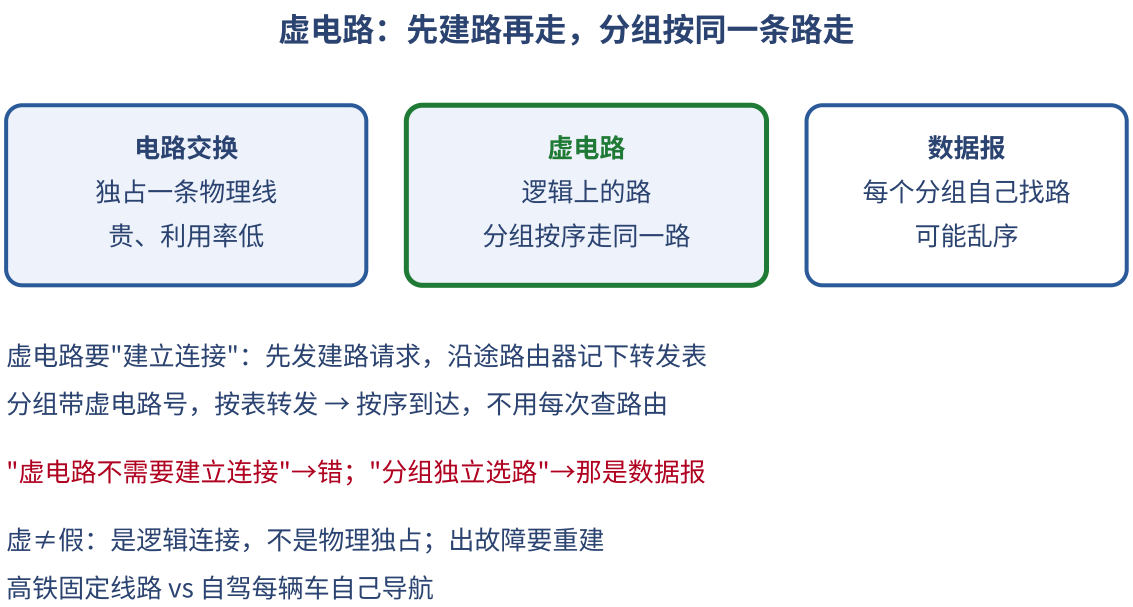
其余选项为什么是对的＋举例：

- A（可以确保数据分组传输顺序）：对。举例：分组 1、2、3 沿同一条虚电路走，先发的 1 先到、后发的 3 后到，顺序不会乱。对照数据报：分组 1 走 A 路、分组 2 走 B 路，2 可能比 1 先到，顺序就乱了。

- C（建立虚电路时需要进行路由选择）：对。举例：建路时从起点到终点选一条路线（比如经过结点 X、Y），沿途结点记下虚电路号；建好后分组就沿这条路走，不用再选路。
- D（依据虚电路号（VCID）进行数据分组转发）：对。举例：分组头里写着虚电路号 5，结点查表"5 号 → 下一跳是 Y"，就把分组发往 Y；转发看的是号，不是目的地址。

笔记本一句话： 第 34 题＋虚电路是约好的路线（建路选路、按号转发、保序），带宽按需共享不预分配；预分配带宽是电路交换（包场）的做法。

【图像】



虚电路是约好的路线：建路选路、按号转发、保序；带宽按需共享不预分配

【定位】 计算机网络 · 第 4 章网络层 · 虚电路

【易混对比】

- "虚电路"和"电路交换"是两回事：虚电路是"约好的路线"（分组按号转发、保序），但带宽按需共享、不预分配；电路交换是"包场"（通话期间带宽独占、预分配）。
- "虚电路"和"数据报"也是两回事：虚电路要先建路（连接），分组按同一条路走、保序；数据报不建路，每个分组独立选路、可能乱序。
- "预分配带宽"是谁的做法：电路交换（包场）的；虚电路不预分配——说"虚电路预分配带宽"是把电路交换的特征安到了虚电路头上。

虚电路是约好的路线（建路、按号转发、保序），带宽按需共享不预分配。

【坑位】

"预分配带宽"是电路交换（包场）的做法，不是虚电路的——别张冠李戴。

最容易踩的坑，是把"虚"字理解成"假的电路交换"，以为它也包场。虚电路的"虚"是指"不是物理独占"：它确实要先建立连接（约好走哪条路），分组按虚电路号转发、按序到达；但带宽是按需共享的——没数据时带宽给别人用，不像电路交换那样通话期间独占一条线。说"虚电路预分配带宽"，是把电路交换"包场"的特征安到了虚电路头上，所以是错的。

【套路】

- 第一步，定虚电路三特征：建路、按号转发、保序。
- 第二步，定带宽：按需共享，不预分配。
- 第三步，见"预分配"判错：那是电路交换的特征。

三特征 → 带宽共享 → 见"预分配"判错。

【类比】

像约好的拼车路线 vs 包车：虚电路是"约好每天走这条路拼车"（路线固定、按序到达），但车上空位谁都能坐（带宽共享）；电路交换是"包下一整辆车"（带宽独占、预分配）。说"拼车也要包场"，是把包车的规矩安到了拼车头上。

【口诀】 虚电路是约好路，按号转发保序。

【考频】 「虚电路」在 2009–2022 年解析中出现 1 次 · 低频

第 35 题 · 冲突域和广播域个数（正确答案：C，4，2）

题干与选项：

下图所示的网络冲突域和广播域的个数分别是：

- A. 2，2
- B. 2，4
- C. 4，2
- D. 4，4

> 注：本题题干依赖图片（网络拓扑图），图片未能直接查看，本题标注"部分核对"。个数按答案表写（冲突域 4 个，广播域 2 个）；图中具体设备连接方式未查看，图中细节以原卷为准。

入手点： 先记住两条隔离规则：冲突域看"谁隔离冲突"，广播域看"谁隔离广播"——再数图中有几个。

题干专有名词从零解释：

- 先想一间屋子只能一个人说话：一个范围内，一次只能一方发数据，两方同时发就撞了（冲突）——这个范围叫"冲突域"，意思是一次只能一方说话的范围。
- 再想广播喊话能传多远：一个广播帧（发给所有人的帧）能传到的范围叫"广播域"，意思是广播能到达的范围。
- 再想三种设备的隔离本事：
- 集线器：傻转发型——一个口收到的数据往所有口发，既不隔离冲突域（所有口一个冲突域），也不隔离广播域（广播传到所有口）。
- 交换机：聪明转发——每个口一个冲突域（隔离冲突域，两个口可以同时说话不撞），但广播还是往所有口泛洪（不隔离广播域）。
- 路由器：两个都隔离——每个口一个冲突域，也每个口一个广播域（广播不跨路由器）。

考查知识点：

- 先想"数冲突域的方法"：交换机、路由器的每个口各是一个冲突域；集线器的所有口合起来是一个冲突域；数的时候按"口"数，不按"设备"数。
- 再想"数广播域的方法"：路由器的每个口各是一个广播域；交换机、集线器不隔离广播域，整个交换机/集线器连着的一片是一个广播域。
- 再想"本题的结果"：按答案表，图中数出来是冲突域 4 个、广播域 2 个——说明图中路由器隔出了 2 个广播域，交换机/集线器的口隔出了 4 个冲突域（具体设备连接方式以原卷为准）。
- 对照"常见错误"：把交换机当成隔离广播域（错，交换机不隔离广播）；把集线器各口当成各一个冲突域（错，集线器所有口一个冲突域）；把路由器各口当成一个广播域（错，路由器每个口一个广播域）。
- 边界情况：交换机下挂集线器时，集线器那一片（所有口）算一个冲突域；路由器下挂交换机时，交换机那一片（所有口）算一个广播域。

正确选项为什么对： 按答案表，图中冲突域 4 个、广播域 2 个。选 C。

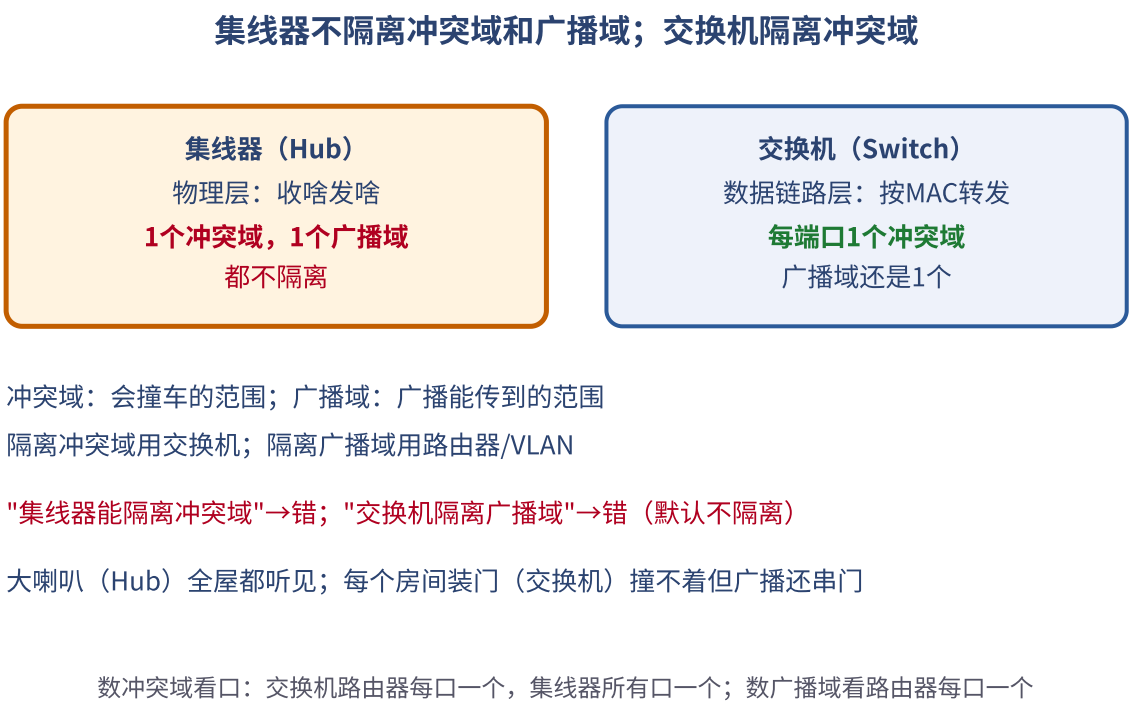
错误选项为什么错＋举例：

- A（2，2）：冲突域数少了——把交换机各口算成了一个冲突域。教训：交换机每个口一个冲突域，不是整个交换机一个；按口数，不按设备数。

- B（2，4）：两个数都错了，还像是把冲突域和广播域的数位弄反了方向——冲突域只数出 2 个（漏了交换机各口），广播域数出 4 个（把路由器隔开的算多了，或把交换机当成了隔离广播域）。教训：冲突域和广播域分开数，隔离规则不一样（交换机隔冲突不隔广播，路由器两个都隔）。
- D（4，4）：冲突域对了，广播域数多了——把交换机当成了隔离广播域的设备。教训：交换机不隔离广播域，广播会泛洪到交换机所有口；只有路由器隔离广播域。

笔记本一句话： 第 35 题＋数冲突域看口（交换机、路由器每口一个，集线器所有口一个）；数广播域看路由器（每口一个，交换机、集线器不隔离）。

【图像】



【定位】 计算机网络 · 第 3 章数据链路层 · 局域网设备

【易混对比】

- "冲突域"和"广播域"是两回事：冲突域看"会不会撞车"（数口），广播域看"广播能不能过去"（看路由器）。
- "集线器"和"交换机"是两回事：集线器所有口一个冲突域（傻瓜转发）；交换机每个口一个冲突域（隔离冲突）——但两者都不隔离广播。
- "路由器"最特殊：每个口既是一个冲突域，又是一个广播域（隔离广播）——数广播域只看路由器。

数冲突域看口（交换机、路由器每口一个，集线器所有口一个）；数广播域看路由器。

【坑位】

交换机隔离冲突域，但不隔离广播域——别把"隔离冲突"当成"隔离广播"。

最容易踩的坑，是觉得"交换机隔离了冲突域，广播域也隔离了"。分开记：冲突域是"会不会撞车"——集线器是傻瓜，所有口撞在一起（1 个）；交换机每个口独立（每口 1 个）；路由器每口也独立。广播域是"广播能传多远"——广播能穿过集线器和交换机，只有路由器挡得住（每口 1 个）。所以数冲突域看"口"，数广播域看"路由器"。

【套路】

- 第一步，数冲突域：集线器所有口算 1 个，交换机/路由器每口算 1 个。

- 第二步，数广播域：只看路由器，每口 1 个；交换机、集线器不隔离，不计数。
- 第三步，加总：本题 4 个冲突域、2 个广播域。

冲突看口，广播看路由器。

【类比】

像小区的门和墙：冲突域是"会不会撞车"——集线器是没隔断的大厅（大家挤一起），交换机是每人一间小屋（各不相干）；广播域是"大喇叭能传多远"——喇叭声能穿过大厅和小屋（集线器、交换机挡不住），只有小区的围墙（路由器）挡得住，一道墙隔出一片。

【口诀】 冲突看口，广播看路由器。

【考频】 「冲突域」在 2009–2022 年解析中出现 5 次 · 中频

第 36 题 · 停-等协议最大信道利用率（正确答案：D，40%）

题干与选项：

假设主机甲采用停-等协议向主机乙发送数据帧，数据帧长与确认帧长均为 1000B。数据传输速率是 10kbps，单向传播延时是 200ms。则甲的最大信道利用率：

- A. 80%
- B. 66.7%
- C. 44.4%
- D. 40%

入手点： 一轮走四段：发数据、数据在路上跑、发确认、确认在路上跑——四段时间加起来是一轮总时间，利用率 = 发数据的时间 ÷ 一轮总时间。

题干专有名词从零解释：

- 先想发一封、等回信、再发下一封：发完一个数据帧就停下来等对方回确认，收到确认才发下一个——这就叫"停-等协议"，意思是 stop-and-wait。
- 再想"信道利用率"：真正用来发数据的时间占一轮总时间的比例——等回信的时间、发确认的时间都不算"发数据"，但都算在总时间里。
- 再想"传输速率 10kbps"：每秒能传 10000 位（k 是 1000，b 是位）；"单向传播延时 200ms"：信号从甲到乙在路上跑 200 毫秒。

考查知识点：

- 先算"发数据的时间"：数据帧 1000B = 8000 位（1 字节 = 8 位），8000 ÷ 10000 = 0.8 秒 = 800ms。
- 再算"数据在路上跑的时间"：单向传播延时 200ms。
- 再算"发确认的时间"：确认帧也是 1000B = 8000 位，同样是 800ms——关键陷阱：很多题里确认帧很小（可以忽略），但本题确认帧和数据帧一样大（1000B），发送时间 800ms 不能省。
- 再算"确认在路上跑的时间"：同样是单向传播延时 200ms。
- 一轮总时间 = 发数据 800ms + 数据传播 200ms + 发确认 800ms + 确认传播 200ms = 2000ms。利用率 = 800 ÷ 2000 = 40%。
- 对照"确认帧可忽略的情况"：如果确认帧很小（发送时间 ≈ 0），一轮 = 800 + 200 + 200 = 1200ms，利用率 = 800 ÷ 1200 = 66.7%（选项 B）——本题确认帧不可忽略，所以不是 B。
- 边界情况：利用率是"最大"值（假设没有出错重发）；实际有出错重发时，利用率更低。

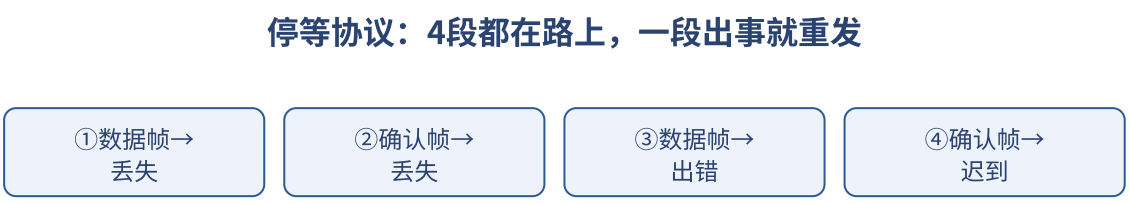
正确选项为什么对： 一轮四段：发数据 800ms + 数据传播 200ms + 发确认 800ms + 确认传播 200ms = 2000ms；利用率 = 800 ÷ 2000 = 40%。选 D。

错误选项为什么错＋举例：

- A（80%）：800 ÷（800 + 200）= 80%——只算了发数据和数据传播，忘了确认的一半（发确认 + 确认传播）。教训：一轮是四段，不是两段；确认的发送和传播都要算。
- B（66.7%）：800 ÷（800 + 200 + 200）= 66.7%——省了确认帧的发送时间 800ms。教训：确认帧很小时才能省；本题确认帧 1000B 和数据帧一样大，发送时间 800ms 必须算。
- C（44.4%）：800 ÷（800 + 200 + 800）= 44.4%——省了确认传播时间 200ms。教训：确认在路上跑也要 200ms，四段一段不能少。

笔记本一句话： 第 36 题＋停-等一轮四段全加（发数据、数据传播、发确认、确认传播）；确认帧和数据帧一样大时，确认发送时间不能省。

【图像】



【定位】 计算机网络 · 第 3 章数据链路层 · 流量控制

【易混对比】

- "一轮四段"和"一轮两段"是两回事：一轮 = 发数据 + 数据传播 + 发确认 + 确认传播，四段时间全加；只算前两段就漏了确认的一半。
- "确认帧可忽略"和"确认帧不可忽略"是两回事：确认帧很小时发送时间可省；本题确认帧和数据帧一样大（1000B），发送时间 800ms 必须算——这是本题的题眼。
- "利用率"和"吞吐量"是两回事：利用率是"发数据的时间占一轮的比例"（800/2000 = 40%）；不是"一秒发多少数据"。

停-等一轮四段全加；确认帧和数据帧一样大时，确认发送时间不能省。

【坑位】

确认帧和数据帧一样大时，确认帧的发送时间 800ms 必须算——这是本题的题眼。

最容易踩的坑，是按"确认帧很小可忽略"的惯性思维，直接得 66.7%（800/(800+200+200)）。但本题题干明确说了"数据帧长与确认帧长均为 1000B"——确认帧发送也要 800ms。一轮四段：发数据 800ms + 数据传播 200ms + 发确认 800ms + 确认传播 200ms = 2000ms，利用率 = 800/2000 = 40%。看到"确认帧长"和"数据帧长"一样，先把"不可忽略"四个字圈出来。

【套路】

- 第一步，算发数据时间：1000B = 8000 位，8000/10000 = 800ms。
- 第二步，看确认帧能不能省：和数据帧一样大 → 不能省，也是 800ms。
- 第三步，四段相加：800 + 200 + 800 + 200 = 2000ms，利用率 = 800/2000 = 40%。

算发送 → 看确认能不能省 → 四段相加。

【类比】

像发一封、等回信、再发下一封：一轮 = 你写信寄出（发数据）+ 信在路上（数据传播）+ 对方写回信寄出（发确认）+ 回信在路上（确认传播）。如果回信和去信一样厚（一样大），对方写回信的时间就不能省——四段时间全加，才是一轮。

【口诀】 一轮四段全加，确认帧大不能省。

【考频】 「停-等协议」在 2009–2022 年解析中出现 4 次 · 中频

第 37 题 · CSMA/CA 帧间间隔（正确答案：A，IFS1）

题干与选项：

某 IEEE 802.11 无线局域网中主机 H 与 AP 之间发送或接收 CSMA/CA 帧的过程如下图所示，在 H 或 AP 发送帧前所等待的帧间间隔时间（IFS）中最长的是：

- A. IFS1
- B. IFS2
- C. IFS3
- D. IFS4

> 注：本题题干依赖图片（CSMA/CA 帧过程图），图片未能直接查看，本题标注"部分核对"。答案按答案表写（IFS1 最长）；图中 IFS1～IFS4 各自的位置未查看，图中细节以原卷为准。

入手点： 帧间间隔分三种，大小关系固定：SIFS 最短、PIFS 中等、DIFS 最长——发送帧前竞争信道等的是最长的 DIFS。

题干专有名词从零解释：

- 先想说话前先听一会儿：无线局域网里，发帧前要先等一会儿、确认信道没人说话再开口——这段等待时间叫"帧间间隔"，意思是 interframe space，简称 IFS。
- 再想三种等待：最短的叫 SIFS（回话专用：收到帧后马上回确认 ACK、回 CTS 时等最短时间）；中等的叫 PIFS（AP 点名轮询时用）；最长的叫 DIFS（普通站点抢信道前等的完整时间，还要再加随机退避）。
- 再想"CSMA/CA"：先听后说 + 冲突避免——发之前先听信道空不空，空了等 DIFS + 随机退避再发；和有线网的 CSMA/CD（冲突检测）不同，无线网检测冲突难，所以用"避免"（退避）代替"检测"。

考查知识点：

- 先想"大小关系"：SIFS < PIFS < DIFS——这是固定的。SIFS 最短（回话不能等太久，不然别人以为信道空了就抢话）；PIFS 中等（AP 点名优先于普通站点）；DIFS 最长（普通站点竞争前等最久，优先级最低）。
- 再想"为什么回话等最短"：ACK、CTS 是对刚收到的帧的回应，必须尽快发——如果等 DIFS 那么久，别的站点可能以为信道空了就插话，回应就被打断了。所以回应类帧等 SIFS。
- 再想"为什么竞争等最长"：普通站点发新帧前等 DIFS + 随机退避——等得久是为了给回应类帧（SIFS）和 AP 轮询（PIFS）让路，优先级最低。
- 对照"时隙"：PIFS = SIFS + 1 个时隙，DIFS = SIFS + 2 个时隙——具体微秒数因标准而异，但大小关系不变。
- 边界情况：退避时间是随机的（竞争窗口内随机选），所以"等待时间"说的是 IFS 本身（固定部分），不是 IFS + 退避的总和；本题问的是 IFS 中最长的，不是总等待最长的。

正确选项为什么对： 按答案表，IFS1 是发送帧前等待时间最长的那个（竞争信道前的 DIFS）。选 A。

错误选项为什么错＋举例：

- B（IFS2）：IFS2 是更短的等待（回应类或轮询用的间隔，比竞争前等待短）。教训：回应（ACK/CTS）等 SIFS 最短，竞争等 DIFS 最长——先分清"回话"还是"抢话"。
- C（IFS3）：同上，IFS3 是更短的等待，不是竞争前最长的 DIFS。教训同上。
- D（IFS4）：同上，IFS4 是更短的等待。教训：四个 IFS 里只有一个是 DIFS（最长），其余都是更短的；按答案表，最长的是 IFS1。

笔记本一句话： 第 37 题＋IFS 大小固定：SIFS（回话）< PIFS（AP 点名）< DIFS（竞争）；发送帧前竞争等的是最长的 DIFS。

【图像】



SIFS < PIFS < DIFS：确认帧最急，等最短
收到数据发ACK前等SIFS；发普通数据前等DIFS+退避

别把"等得久"当成"优先级高"：等得越短越优先

CSMA/CA是"避免"碰撞（先听后说+预约），不是"检测"碰撞

无线收发不能同时 → 撞了也听不见 → 只能尽量避免
急诊（ACK）走绿色通道，普通门诊（数据）排队等

IFS大小固定：SIFS（回话）< PIFS（AP点名）< DIFS（竞争）；发送帧前竞争等最长的DIFS

【定位】 计算机网络 · 第 3 章数据链路层 · 介质访问控制

【易混对比】

- "SIFS/PIFS/DIFS"是三回事：SIFS 最短（收到帧后回话用），PIFS 中等（AP 点名用），DIFS 最长（竞争信道用）——大小固定，SIFS < PIFS < DIFS。
- "发送数据帧前"和"收到帧后回话"是两回事：发送前要竞争，等的是最长的 DIFS；收到后回 ACK/CTS，等的是最短的 SIFS——别把"回话"和"发送"用的 IFS 搞反。
- "CSMA/CA"和"CSMA/CD"是两回事：CA（无线）用 IFS + 退避来"避免"碰撞；CD（有线）是发完才发现"检测"到碰撞——一个避、一个检。

IFS 大小固定：SIFS < PIFS < DIFS；发送帧前竞争等的是最长的 DIFS。

【坑位】

发送数据帧之前等的是 DIFS（最长），不是 SIFS——SIFS 是"回话"用的。

最容易踩的坑，是把 SIFS 和 DIFS 的用途记反。SIFS（短帧间间隔）是"收到帧之后马上回话"的等待——比如收到 RTS 回 CTS、收到数据回 ACK，因为回话优先级最高，所以等得最短。DIFS（长帧间间隔）是"想发送数据之前竞争信道"的等待——大家都要发，公平起见等得最长。本题问的是"发送帧之前"，所以是 DIFS。记住这个顺序：回话（SIFS）< AP 点名（PIFS）< 竞争发送（DIFS）。

【套路】

- 第一步，定场景：发送帧之前 → 竞争 → DIFS；收到帧回话 → SIFS。
- 第二步，背顺序：SIFS < PIFS < DIFS，固定不变。
- 第三步，对号：发送前竞争 → 最长的 DIFS。

定场景 → 背顺序 → 对号。

【类比】

像开会发言的等待规矩：被点名回答（回话）不用等太久（SIFS 最短）；主持人点名（PIFS）等一会儿；自己想抢着发言（竞争）要等最久（DIFS 最长）——想插话的人等得最长，公平。把"抢着发言"和"被点名回答"的等待搞反，就选错了。

【口诀】 SIFS回话PIFS点名DIFS竞争，发送等最长。

【考频】 「CSMA/CA」在 2009–2022 年解析中出现 3 次 · 中频

第 38 题 · 拥塞窗口增长最长时间（正确答案：C，24ms）

注：本题答案曾出现双源不一致——公开版答案表为 C（24ms），夸克解析版答案表为 D（48ms）。按慢启动／拥塞避免标准算法裁定 C 正确（12 轮 × 2ms = 24ms），夸克版 D 疑为答案表笔误。本题题干按原卷写，未为迁就某一来源改题干。

题干与选项：

若主机甲与主机乙已建立一条 TCP 连接，最大段长（MSS）为 1KB，往返时间（RTT）为 2ms，则在不出现拥塞的前提下，拥塞窗口从 8KB 增长到 20KB 所需的最长时间是：

- A. 4ms
- B. 8ms
- C. 24ms
- D. 48ms

入手点： 拥塞窗口两种长法：慢启动翻倍（快），拥塞避免每次加 1KB（慢）——"最长"对应全程用最慢的长法；先算两种长法各要几轮，再乘 RTT。

题干专有名词从零解释：

- 先想一次能发多少没确认的数据：发送方一次能发出去、还没收到确认的数据量叫"拥塞窗口"，意思是 congestion window，简称 cwnd；窗口越大，一次发得越多。
- 再想"最大段长"：一个 TCP 段最多带多少数据，叫 MSS，意思是 maximum segment size；本题 MSS 是 1KB，所以窗口每次加 1KB 就是加 1 个段。
- 再想"往返时间"：发出去到收到确认走一圈的时间叫 RTT，意思是 round-trip time；本题 RTT 是 2ms——窗口每长一轮（每收到一轮确认）要花 1 个 RTT。
- 再想两种长法：慢启动是每轮翻倍（8→16→32，像滚雪球）；拥塞避免是每轮加 1 个 MSS（8→9→10，像爬楼梯，一次上一级）。

考查知识点：

- 先想"最快是多快"：全程慢启动翻倍——第 1 轮 8→16，第 2 轮 16→32（超过 20KB 就够了），2 轮 × 2ms = 4ms（选项 A）。这是最短时间，不是本题问的最长。
- 再想"最慢是多慢"：全程拥塞避免每次加 1KB——8→9→10→…→20，一共加 12KB，要 12 轮，12 × 2ms = 24ms（选项 C）。按慢启动/拥塞避免的标准算法，"最长时间"算出来是 24ms。
- 再想"双源不一致"：公开版答案表写 C（24ms），与上述标准算法一致；夸克解析版答案表写 D（48ms，等于 24 轮 × 2ms，或 12 轮 × 4ms），与题干 RTT = 2ms 下的标准算法不符。裁定：C 正确，夸克版 D 疑为答案表笔误。
- 对照"慢启动门限"：实际 TCP 里，慢启动到门限值后转拥塞避免；本题没给门限值，所以"最长"按全程拥塞避免算（最慢的长法），"最快"按全程慢启动算（最快的长法）。
- 边界情况："不出现拥塞"的意思是窗口按规则正常长，不用减半重来；如果中途出现拥塞（丢包），窗口要减半，时间就另算了——本题不考虑这种情况。

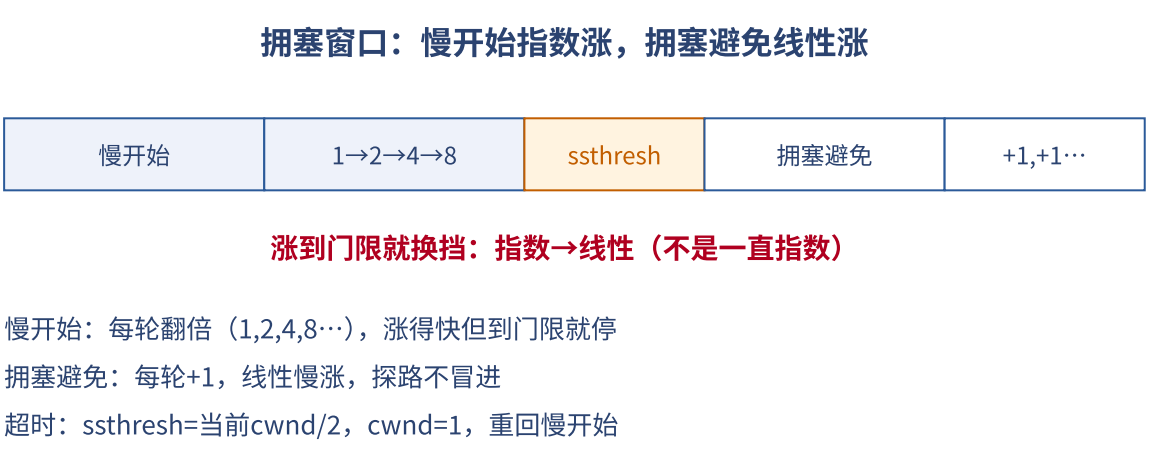
正确选项说明： C（24ms）正确。最慢是全程拥塞避免：8KB 到 20KB 加 12KB、12 轮、12 × 2ms = 24ms。夸克解析版答案表写 D（48ms），与题干 RTT = 2ms 下的标准算法不符，疑为笔误；公开版答案表写 C，与标准算法一致。

其余选项说明＋举例：

- A（4ms）：这是最快时间——全程慢启动翻倍，8→16（第 1 轮）、16→32（第 2 轮），2 轮 × 2ms = 4ms。本题问的是最长时间，不是 A。
- B（8ms）：4 轮 × 2ms——既不是最快（2 轮）的结果，也不是标准算法下最慢（12 轮）的结果。教训：先定"最快"和"最慢"两种长法各要几轮，再对号入座。
- D（48ms）：48ms 相当于 12 轮 × 4ms，或 24 轮 × 2ms；题干 RTT = 2ms，全程拥塞避免只需 12 轮，12 × 2ms = 24ms，不是 48ms。错在把一轮的时间算成了 4ms（把往返时间当成了单向时间的两倍），或把轮数算成了 24 轮。

笔记本一句话： 第 38 题＋慢启动翻倍最快（2 轮 4ms），拥塞避免每次加 1KB 最慢（12 轮 24ms），答案 C；夸克版写 D 疑为笔误。

【图像】



【定位】 计算机网络 · 第 5 章传输层 · TCP 拥塞控制

【易混对比】

- "最快"和"最慢"是两回事：最快是全程慢启动翻倍（8→16→32，2 轮 4ms）；最慢是全程拥塞避免每次加 1KB（8→9→…→20，12 轮 24ms）——题干问的是"最长"时间。
- "慢启动"和"拥塞避免"是两回事：慢启动每轮翻倍（滚雪球，快）；拥塞避免每轮加 1 个 MSS（爬楼梯，一次上一级，慢）。
- "一轮的时间"是 RTT（2ms），不是单向传播时间——12 轮 × 2ms = 24ms；把一轮算成 4ms 就会得 48ms（错）。

"最长"对应全程用最慢的长法：拥塞避免每轮加 1KB，12 轮 24ms。

【坑位】

先定"最快"和"最慢"两种长法各要几轮，再对号入座——别把"最快"的时间当成"最长"的答案。

最容易踩的坑，是算出 4ms（最快）就选 A。题干问的是"最长时间"：慢启动翻倍最快，2 轮就到（8→16→32 超过 20KB），2 × 2ms = 4ms——这是最短，不是所问。最慢是全程拥塞避免：从 8KB 每次加 1KB 到 20KB，要加 12KB，即 12 轮，12 × 2ms = 24ms。另一个坑是把一轮算成 4ms（把 RTT 当成单向时间的两倍），得 48ms——RTT 本身就是往返一圈，一轮就是 2ms。

【套路】

- 第一步，定两种长法：慢启动翻倍（快），拥塞避免 +1MSS（慢）。
- 第二步，看问"最长"还是"最短"：最长 → 全程拥塞避免；最短 → 全程慢启动。
- 第三步，算轮数乘 RTT：12 轮 × 2ms = 24ms。

定长法 → 看问长问短 → 轮数乘RTT。

【类比】

像往桶里加水：慢启动是每轮倒的水翻倍（8 勺→16 勺→32 勺，滚雪球）；拥塞避免是每轮只加 1 勺（爬楼梯）。问"最慢多久能加满"，当然按"每轮只加 1 勺"算——12 勺、12 轮。按"翻倍"算出来的是"最快"，答非所问。

【口诀】 慢启动翻倍快，拥塞避免加一慢。

【考频】 「拥塞窗口」在 2009–2022 年解析中出现 8 次 · 中频

第 39 题 · TCP 已发送字节数（正确答案：C，4000）

题干与选项：

若主机甲与主机乙建立 TCP 连接时发送的 SYN 段中的序号为 1000，在断开连接时，甲发送给乙的 FIN 段中的序号为 5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为：

- A. 4002
- B. 4001
- C. 4000
- D. 3999

入手点： 序号是每个字节的门牌号：SYN 占 1 个号，数据从 SYN 的下一个号开始，FIN 的号是"数据最后一个号 + 1"——数据字节数 = FIN 的号 − SYN 的号 − 1。

题干专有名词从零解释：

- 先想每个字节的门牌号：TCP 给每个数据字节编一个号，叫"序号"——第 1 个字节是 1001，第 2 个是 1002，依此类推；接收方按号拼顺序、查缺了哪个。
- 再想"SYN"和"FIN"：建立连接时发的 SYN（请求建立）和断开时发的 FIN（请求断开）本身不带应用层数据，但各占 1 个序号——占号是为了让对方确认"我收到你的 SYN/FIN 了"。
- 再想"无任何重传"：每个字节只发了一次，没有重复发——所以序号差直接等于字节数，不用扣重传的。

考查知识点：

- 先想"序号怎么排"：SYN 占 1000 号；数据从 1001 开始；FIN 的号是 5001，说明数据的最后一个字节是 5000 号（因为 FIN 紧跟在数据后面，FIN 的号 = 数据最后字节的号 + 1）。
- 再想"数据字节数"：从 1001 到 5000，一共 5000 − 1001 + 1 = 4000 字节。公式：FIN 的号 − SYN 的号 − 1 = 5001 − 1000 − 1 = 4000（减 1 是减掉 SYN 占的那个号）。
- 对照"ACK 占不占号"：纯 ACK（只确认、不带数据）不占序号；SYN 和 FIN 各占 1 个号；数据按字节数占号。
- 边界情况：序号是 32 位循环的（到 2^32 − 1 后回 0），本题数字小，不用考虑回绕；如果有重传，序号差就不能直接等于字节数（重传的字节序号重复），本题说了无重传，所以直接算。

正确选项为什么对： SYN 占 1000 号，数据从 1001 开始，FIN 是 5001 号说明数据最后字节是 5000 号。数据字节数 = 5000 − 1001 + 1 = 4000（或 5001 − 1000 − 1 = 4000）。选 C。

错误选项为什么错＋举例：

- A（4002）：5001 − 1000 + 1 = 4002——把 FIN 的号也算成了数据字节。教训：FIN 的号是"下一个待发"的号，不是数据字节；FIN 占的那个号不算数据。
- B（4001）：5001 − 1000 = 4001——忘了减 SYN 占的 1 个号。教训：SYN 占 1000 号，数据从 1001 开始，不是 1000 开始；不减 SYN 就多算 1 个。
- D（3999）：5001 − 1000 − 2 = 3999——多减了 1 个，把 FIN 占的号也减了。教训：公式只减 SYN 的 1 个号（5001 − 1000 − 1）；FIN 的号本来就不在数据里，不用再减一次。

笔记本一句话： 第 39 题＋TCP 数据字节数 = FIN 的号 − SYN 的号 − 1（只减 SYN 占的 1 个号，FIN 的号本来就不是数据）。

【图像】

TCP序号：SYN/FIN占1号，ACK不占号



题：SYN(seq=x) → ACK(seq=x+1) → 数据(Seq=x+1,100B) → ACK(seq=x+101)
发方下一个序号 = x+101；收方确认号 = x+101

纯ACK不占号：它的序号还是x+1，别写成x+2

确认号 = 期望收到的下一个序号（不是"已收到的最后一个"）

SYN/FIN像挂号信占一个号；纯ACK像回执条不占号

TCP序号：SYN占1000号，数据1001~5000，FIN是5001号；数据字节数=5001−1000−1=4000

【定位】 计算机网络 · 第 5 章传输层 · TCP 序号

【易混对比】

- "SYN 占号"和"ACK 占号"是两回事：SYN 和 FIN 各占 1 个序号；纯 ACK（只确认不带数据）不占序号。
- "FIN 的号"和"数据字节"是两回事：FIN 的号是"数据最后一个号 + 1"，它本身不是数据——把 FIN 的号也算成数据就多算 1 个（4002 的陷阱）。
- "从 1001 开始"和"从 1000 开始"是两回事：SYN 占了 1000 号，数据从 1001 开始；不减 SYN 占的 1 个号就多算 1 个（4001 的陷阱）。

数据字节数 = FIN 的号 − SYN 的号 − 1（只减 SYN 占的 1 个号）。

【坑位】

公式只减 SYN 的 1 个号：FIN 的号本来就不是数据，不用再减一次（3999 的陷阱）。

最容易踩的坑，是多减或少减。拆开看：SYN 占 1000 号，数据从 1001 开始；FIN 是 5001 号，说明数据最后一个字节是 5000 号（FIN 紧跟在数据后面）。数据字节数 = 5000 − 1001 + 1 = 4000。公式 5001 − 1000 − 1 = 4000：减的这 1 是 SYN 占的号。4002 是把 FIN 的号也算成了数据；3999 是多减了一次（把 FIN 占的号又减了一遍）——FIN 的号本来就不在数据里，不用减。

【套路】

- 第一步，定 SYN 的号：SYN 占 1000 号，数据从 1001 开始。
- 第二步，定 FIN 的含义：FIN 的号 = 数据最后字节的号 + 1 → 数据最后是 5000 号。
- 第三步，套公式：FIN 的号 − SYN 的号 − 1 = 5001 − 1000 − 1 = 4000。

SYN 占号 → FIN 号减一是数据尾 → 套公式。

【类比】

像电影票座位号：SYN 占了 1000 号座（工作人员座），观众从 1001 号开始坐；FIN 是 5001 号——它是"下一位观众该坐的号"，不是观众。观众人数 = 5001 − 1000 − 1（减掉工作人员那 1 个）= 4000。把 5001 号也算成观众

（4002），或把工作人员减两遍（3999），都错。

【口诀】 FIN减SYN再减一，只减SYN占的号。

【考频】 「序号」在 2009–2022 年解析中出现 17 次 · 高频

第 40 题 · DNS 最短与最长时间（正确答案：D，20ms，50ms）

题干与选项：

假设下图所示网络中的本地域名服务器只提供递归查询服务，其他域名服务器均只提供迭代查询服务；局域网内主机访问 Internet 上各服务器的往返时间（RTT）均为 10ms，忽略其他各种时延。若主机 H 通过超链接 http://www.abc.com/index.html 请求浏览纯文本 Web 页 index.html，则从点击超链接开始到浏览器接收到 index.html 页面为止，所需的最短时间与最长时间分别是（ ）。

- A. 10ms，40ms
- B. 10ms，50ms
- C. 20ms，40ms
- D. 20ms，50ms

> 注：本题题干依赖图片（DNS 网络图），图片未能直接查看，本题标注"部分核对"。答案按答案表写（最短 20ms，最长 50ms）；图中服务器布局（根、顶级域、权威三级）按常规结构与答案反推，图中细节以原卷为准。

入手点： 两段时间相加：先解析域名要多久（分缓存命中和不命中两种），再取页面要多久（固定 10ms）——最短对应缓存命中，最长对应一级一级问到底。

题干专有名词从零解释：

- 先想电话簿查询：域名（www.abc.com）要翻译成 IP 地址才能访问——这个翻译叫"域名解析"，管翻译的服务器叫"域名服务器"。
- 再想"本地域名服务器"：离主机 H 最近的服务器（局域网里的），H 先问它；它有缓存（以前查过的记录），命中就直接回答。
- 再想"递归查询"：你帮我问到底——H 问本地服务器，本地服务器替 H 一级一级问下去，问到了再把结果带回来；本题本地服务器只提供递归服务。
- 再想"迭代查询"：你告诉我去问谁——根服务器不替你问到底，它告诉你"去问顶级域服务器"，顶级域告诉你"去问权威服务器"，你自己一级一级问；本题其他服务器只提供迭代服务。
- 再想"纯文本 Web 页"：页面只有文字（index.html），没有图片等内嵌对象——取一次页面就够了，不用再取别的东西。

考查知识点：

- 先想"最短时间"：本地服务器缓存命中——H 问本地（RTT 10ms）直接拿到 IP，不用再问各级；再向 Web 服务器取页面（RTT 10ms）。最短 = 10 + 10 = 20ms。
- 再想"最长时间"：本地服务器缓存没有——H 问本地（10ms），本地替 H 跑腿：一级一级问根服务器（10ms）、顶级域服务器（10ms）、权威服务器（10ms），三级迭代共 30ms；拿到 IP 后，H 再向 Web 服务器取页面（10ms）。最长 = 本地 10 + 三级迭代 30 + 取页面 10 = 50ms。
- 再想"为什么是三级迭代"：域名从右往左分级——根（.）→ 顶级域（.com）→ 权威（abc.com）→ 主机（www）——本地服务器要问三级（根、顶级、权威）才能拿到 www.abc.com 的 IP。
- 对照"递归和迭代的分工"：H 和本地之间是递归（本地替 H 跑腿）；本地和其他服务器之间是迭代（各级告诉本地下一步问谁，本地自己一级一级问）。
- 边界情况：如果页面有内嵌图片等对象，取完 index.html 还要再取图片，时间更长；本题是纯文本页，只取一次，所以不用加。

正确选项为什么对： 最短：缓存命中，本地解析 10ms + 取页面 10ms = 20ms。最长：缓存没有，本地 10ms + 三级迭代 30ms + 取页面 10ms = 50ms。选 D。

错误选项为什么错＋举例：

- A（10ms，40ms）：最短 10ms 只算了解析，没算取页面（取页面也要 10ms）；最长 40ms 少算了 10ms（漏了本地查询或取页面中的一段）。教训：总时间 = 解析时间 + 取页面时间，两段都要算。
- B（10ms，50ms）：最长对了，最短 10ms 错在只算了解析、没算取页面。教训：最短也要加取页面的 10ms（缓存命中省的是迭代时间，不是取页面时间）。
- C（20ms，40ms）：最短对了，最长 40ms 少算了 10ms（可能是少算了一级迭代，或漏了取页面）。教训：最长是本地 10 + 三级迭代 30 + 取页面 10 = 50ms，五段一段不能少。

笔记本一句话： 第 40 题＋DNS 最短 = 缓存命中（本地 10 + 取页面 10）；最长 = 本地 10 + 三级迭代 30 + 取页面 10；取页面那 10ms 永远要加。

【图像】



【定位】 计算机网络 · 第 6 章应用层 · DNS

【易混对比】

- "递归查询"和"迭代查询"是两回事：递归是"你帮我问到底"（H 和本地之间，本地替 H 跑腿）；迭代是"你告诉我去问谁"（本地和其他服务器之间，本地自己一级一级问）。
- "缓存命中"和"缓存没有"是两回事：命中，本地直接回答（只花本地 10ms）；没有，本地替 H 跑三级迭代（多花 30ms）。
- "解析时间"和"总时间"是两回事：总时间 = 解析时间 + 取页面时间——取页面那 10ms 永远要加，缓存命中也省不掉它。

最短 = 缓存命中（本地 10 + 取页面 10）；最长 = 本地 10 + 三级迭代 30 + 取页面 10。

【坑位】

取页面那 10ms 永远要加——缓存命中省的是迭代时间，不是取页面时间。

最容易踩的坑，是最短时间只算解析（10ms）忘了取页面。完整拆开：最短是本地缓存命中——H 问本地（10ms）直接拿到 IP，再向 Web 服务器取页面（10ms），共 20ms。最长是本地缓存没有——H 问本地（10ms），本地替 H 一级一级问根、顶级域、权威三级（30ms），拿到 IP 后再取页面（10ms），共 50ms。五段（本地 10 + 三级 30 + 取页面 10）一段不能少。

【套路】

- 第一步，分两种情况：缓存命中 → 最短；缓存没有 → 最长。
- 第二步，数段：最短 = 本地 10 + 取页面 10；最长 = 本地 10 + 三级迭代 30 + 取页面 10。
- 第三步，验"取页面"：取页面 10ms 两种情况都要加。

分情况 → 数段 → 取页面永远加。

【类比】

像查电话簿再打电话：最短是电话簿就在手边（缓存命中）——翻一下（10ms）再打电话（10ms）；最长是手边没有——先问总机（10ms），总机帮你一级一级问分机（30ms），问到号码再打电话（10ms）。不管哪种情况，"打电话"这一步都省不掉。

【口诀】 最短缓存命中，取页面永远加。

【考频】 「DNS」在 2009–2022 年解析中出现 7 次 · 中频

附注

- 本版只含 2020 年 408 选择题第 1–40 题，不含综合应用题；每题含完整题干与选项，可脱离真题册独立使用。
- 每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。
- 题干问"错误的是"的互换题（第 4 节讲正确选项即那个错述为什么成立，第 5 节讲其余选项为什么是对的）：第 15、18、21、22、29、30、34 题。
- 语言风格：大白话为主，每个术语第一次出现配一句大白话解释，顺序按"从具体到抽象"（先讲生活例子、小数字、具体场景，再贴术语名）；讲解比 2022 版更深，考查知识点与错误选项两节把定义、适用条件、边界情况、易混对子讲到位；每题自成体系，不依赖其他题的解析，不预设前置概念。
- 答案表（逐题值，以逐题表／逐题正文为准）：
- 数据结构：第 1 题 C，第 2 题 D，第 3 题 A，第 4 题 C，第 5 题 B，第 6 题 B，第 7 题 A，第 8 题 B，第 9 题 C，第 10 题 B，第 11 题 A。
- 计组：第 12 题 B，第 13 题 A，第 14 题 D，第 15 题 D，第 16 题 A，第 17 题 B，第 18 题 A，第 19 题 C，第 20 题 C，第 21 题 B，第 22 题 C。
- 操作系统：第 23 题 B，第 24 题 A，第 25 题 D，第 26 题 D，第 27 题 B，第 28 题 D，第 29 题 B，第 30 题 D，第 31 题 B，第 32 题 C。
- 计网：第 33 题 C，第 34 题 B，第 35 题 C，第 36 题 D，第 37 题 A，第 38 题 C（见核对结果双源比对），第 39 题 C，第 40 题 D。

核对结果

- 题干与选项来源：存档文件《2020-408选择题原卷与答案表.md》（转录自 N诺考研 2020 408 试题页），1–40 题题干、选项、答案逐题转录；草稿各题题干与选项与存档文件逐题对照，无改动（未为迁就某一来源改题干）。
- 存档文件笔误：存档文件自身的答案串汇总行在 1–10 题与 11–20 题处与逐题表／逐题正文不一致，属转录笔误；草稿以逐题表／逐题正文为准（逐题值见附注答案表），汇总行不采用。
- 双源比对（答案表级）：公开版（N诺）答案表与用户夸克版《2020年计算机408统考真题解析》第 1 页答案表逐题比对，39 题一致；唯第 38 题不一致——公开版 C（24ms），夸克解析版 D（48ms）。裁定：C 正确（标准算法 12 轮 × 2ms = 24ms），夸克版 D 疑为答案表笔误；题干未改；双源不一致如实记录。
- 40 题答案均与存档逐题表一致（第 38 题裁定为 C，与存档逐题表一致；夸克版 D 疑为笔误，见双源比对）。
- 图题部分核对（6 题，图片未能直接查看，图中细节以原卷为准）：
- 第 5 题：树形由四个输入序列与答案反推（A、C、D 生成同一棵树：根 4，左孩子 2，右孩子 5；2 的左孩子 1，右孩子 3；B 生成另一棵链形树），与答案 B 自洽；原图未直接查看，仍标部分核对。
- 第 7 题：边加入顺序按答案表写，图中各边权值未查看；图中细节以原卷为准。
- 第 33 题：答案按答案表写（仅 III）；图中具体画的是哪种顺序关系未查看，图中细节以原卷为准。
- 第 35 题：个数按答案表写（冲突域 4 个，广播域 2 个）；图中具体设备连接方式未查看，图中细节以原卷为准。
- 第 37 题：答案按答案表写（IFS1 最长）；图中 IFS1～IFS4 各自的位置未查看，图中细节以原卷为准。
- 第 40 题：答案按答案表写（最短 20ms，最长 50ms）；图中服务器布局（根、顶级域、权威三级）按常规结构与答案反推，图中细节以原卷为准。
- 除 6 道图题外，其余 34 题与存档逐题表一致，无改动；各题推理与计算逐题验算，答案与附注答案表一致。
- 语言与结构自查：全文检索禁词（显然／易得／同理／易知）与外指（"见第 X 题""前面讲过""前述"），有则改掉；每题自成体系，不依赖其他题的解析。